Sicurezza dei Sistemi Informativi

01UDUOV

Politecnico di Torino

Prof. Cataldo Basile

A cura degli studenti:
A.A. 2025/26: *Alessandro Annecchini*, *Alessio Giliberti*, *Chiara Mercurio*, *Luca Giacchetta*

Copyright © 2025-2026 Cataldo Basile
Licensed under the Creative Commons Attribution-NonCommercial 3.0 Unported License (the "License"). You may not use this file except in compliance with the License. You may obtain a copy of the License at `http://creativecommons.org/licenses/by-nc/3.0`. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

Indice

Todo list 7

I Lezioni

1 Introduzione 9

1.1 La sicurezza informatica: motivazioni e dati 9

1.1.1 Alcune definizioni di sicurezza 9

1.1.2 I motivi dell'importanza della sicurezza informatica 10

1.1.3 Insicurezza: le cause profonde 11

1.1.4 Verizon Data Breach Investigation Report 11

1.2 Minacce e problemi di base 14

1.2.1 Problemi tecnologici di base 14

1.2.2 Complessità 14

1.2.3 Altri problemi tecnologici 15

1.3 Attacchi e tecniche 15

1.3.1 Malware 17

1.3.2 Ransomware 18

1.3.3 Industria del Cybercrime 19

1.4 Altri problemi e minacce 19

1.4.1 Alcuni esempi di attacchi riconducibili a errori umani 19

1.4.2 Ingegneria sociale 21

1.4.3 Attacchi famosi 22

1.5 Sicurezza: definizioni 24

1.5.1 Riservatezza 24

1.5.2 Integrità 24

1.5.3 Autenticazione della controparte 25

1.5.4 Autenticazione dei dati 25

1.5.5 Non ripudio 25

1.5.6 Autorizzazione (o controllo degli accessi) 26

1.5.7 Protezione dei dati 26

1.6 Principi di sicurezza 26

1.7 I tre pilastri della sicurezza 27

2 Gestione dei rischi informatici 29

2.1 Fondamenti e Terminologia del Rischio 29

2.2 Vulnerabilità e CVE – Common Vulnerabilities and Exposures 29

2.3 CWE e Window of Exposure **30**
2.3.1 Esempio di Vulnerabilità Critica: Meltdown (CVE-2017-5754) 30
2.3.2 EPSS – Exploit Prediction Scoring System 30
2.4 Window of Exposure **31**
2.5 Risk Management Framework (RMF) **32**
2.6 Vulnerability Assessment **33**
2.6.1 Vulnerability Assessment tools 33
2.7 Valutazione del Rischio **35**
2.8 Vulnerability Management e Patch Management **37**
2.8.1 Ruoli e Strategie 38
2.8.2 Affidabilità delle Patch e Problematiche 40
2.8.3 Evoluzione del Modello di Rilascio delle Patch 40
2.8.4 Enterprise Patch Management Tools 41

3 Crittografia parte 1 43
3.1 Terminologia **43**
3.2 Crittografia simmetrica **44**
3.2.1 Principio di Kerckhoffs 45
3.2.2 Forza di un algoritmo simmetrico 46
3.2.3 Algoritmi di tipo stream 47
3.2.4 Algoritmi a blocchi 48
3.3 Integrità **51**
3.3.1 Principi di Controllo di Integrità 52
3.3.2 Digest e Funzioni di Hash Crittografiche 52
3.3.3 Resistenza alle Collisioni e Lunghezza del Digest 52
3.3.4 Message Integrity Code (MIC) 54
3.3.5 Message Authentication Code (MAC) e Message IDentifier (MID) 54
3.4 Funzioni combinate per Integrità e Riservatezza **56**
3.4.1 Authenticated Encryption (AE) e AEAD 57

4 Crittografia parte 2 58
4.1 Limitazioni della crittografia simmetrica **58**
4.2 Crittografia asimmetrica **59**
4.2.1 Chiavi con funzione reciproca 59
4.3 Algoritmi di crittografia asimmetrica **61**
4.3.1 Generazione delle chiavi 61
4.3.2 DSA 63
4.3.3 RSA 63
4.4 Applicazioni della Crittografia Asimmetrica **67**
4.4.1 Scambio di Chiavi Simmetriche 67
4.4.2 Autenticazione mediante Cifratura Asimmetrica 69
4.4.3 Crittografia a Curve Ellittiche (ECC) 71
4.4.4 Key Derivation Function (KDF) 73
4.4.5 Crittografia Post-Quantum PQC 75

4.5 Certificati Digitali **77**
4.5.1 Formati per certificati a chiave pubblica 78
4.5.2 Public-Key Infrastructure 78
4.6 Prestazioni ed acceleratori **79**
4.6.1 Personal Security Environment 79
4.6.2 Smart card 80
4.6.3 Hardware Security Module 80
4.7 Documenti elettronici **80**
4.7.1 Formati dei documenti firmati 81
4.7.2 Firme multiple 81
4.7.3 Timestamping 81
4.7.4 Qualified Certificate 82
4.7.5 Il problema delle "macro" 82

Todo list

(Ch 3.1.0) Nota del docente 43
(Ch 3.1.0) Nota del docente 43
(Ch 3.1.0) Nota del docente 43
(Ch 3.1.0) Nota del docente 44
(Ch 3.2.3) Nota del docente 47
(Ch 3.2.4) Nota del docente 49
(Ch 3.2.4) Nota del docente 51
(Ch 3.3.4) Inserire figura 54
(Ch 3.3.5) Nota per il docente 56
(Ch 3.4.1) Inserire figura 57
(Ch 3.4.1) Nota per i collaboratori 57

Parte I

Lezioni

1. Introduzione

Il presente capitolo si propone di illustrare il ruolo centrale della sicurezza informatica nel contesto odierno, analizzandone l'evoluzione storica, le minacce più diffuse e le principali categorie di malware. Successivamente, si esamineranno i fondamenti della difesa informatica, con particolare attenzione ai principi e alle proprietà che ne costituiscono il nucleo concettuale.

In termini generali, *fare sicurezza* significa **portare i rischi a livelli accettabili**, concetto la cui definizione richiede un'attenta contestualizzazione rispetto agli obiettivi organizzativi e alle risorse disponibili.

1.1 La sicurezza informatica: motivazioni e dati

1.1.1 Alcune definizioni di sicurezza

È opportuno innanzitutto chiarire cosa si intenda per **sicurezza informatica**. Nel corso degli anni sono emerse diverse accezioni di questo concetto, ciascuna con specifiche sfumature di significato:

- Una prima definizione, di matrice aziendale, la identifica come:

  > L'insieme dei prodotti, dei servizi, delle regole organizzative e dei comportamenti individuali che proteggono i sistemi informatici di un'azienda.

  Questa prospettiva adotta un approccio pragmatico, sottolineando l'obiettivo di garantire il corretto funzionamento dei sistemi aziendali e la protezione degli asset anche in presenza di minacce provenienti da attori malevoli.
- Una seconda definizione, emersa successivamente, enfatizza invece:

  > Il compito di proteggere le risorse da accessi indesiderati, garantire la riservatezza delle informazioni, assicurare il funzionamento e la disponibilità dei servizi a fronte di eventi imprevedibili (C.I.A. = Confidentiality, Integrity, Availability).

  In questo caso l'attenzione si concentra sulla prevenzione di accessi non autorizzati, sia da parte di attori criminali sia di soggetti concorrenti. Successivamente introduce quella che viene chiamata *CIA triad*, tre concetti essenziali per poter costruire sistemi sicuri. La confidenzialità riguarda la protezione della segretezza dei dati mentre l'integrità indica la necessità di preservare dati e sistemi da modifiche non autorizzate. Viene inoltre introdotto il concetto di disponibilità, aspetto talvolta considerato al confine della sicurezza informatica in senso stretto, ma rilevante nella misura in cui mira a impedire che terzi possano compromettere l'erogazione dei servizi, causando perdite economiche.
- Una terza definizione riprende l'aspetto applicativo della prima e rappresenta la necessità di riportare il mondo informatico e virtuale all'esperienza fisica:

> L'obiettivo è custodire le informazioni con la stessa professionalità ed attenzione con cui ci si prende cura di gioielli o certificati azionari depositati nel caveau. Il sistema informatico è la cassaforte delle nostre informazioni più preziose; la sicurezza informatica è l'equivalente delle serrature, combinazioni e chiavi che servono a proteggerla.

Questa prospettiva evidenzia la sicurezza informatica come trasposizione digitale dei meccanismi di protezione fisica tradizionalmente impiegati per la tutela di beni di valore.

1.1.2 I motivi dell'importanza della sicurezza informatica

La sicurezza informatica è spesso percepita dai vertici aziendali come un investimento privo di ritorno immediato: l'implementazione di misure di protezione non genera profitto diretto e può, al contrario, ridurre la produttività operativa. Tuttavia, un'analisi più approfondita evidenzia tre motivazioni fondamentali che ne giustificano l'essenzialità:

- **Ragioni economiche**: la progressiva digitalizzazione degli asset aziendali ha reso il patrimonio informativo un bene critico da proteggere. Gli istituti finanziari, ad esempio, operano quasi interamente su piattaforme digitali, rendendo evidente la necessità di tutelare tali risorse per prevenire danni economici. Per le organizzazioni, le conseguenze di una compromissione possono manifestarsi in molteplici forme: sottrazione di segreti industriali da parte di concorrenti, danno reputazionale derivante dalla violazione di dati sensibili dei clienti, interruzione prolungata dei servizi con conseguente perdita di ricavi e pagamento di penali per violazione dei *Service Level Agreement*.
- **Ragioni sociali ed etiche**: la sicurezza informatica riveste un ruolo essenziale nella protezione individuale e nella tutela della privacy. La correttezza nella gestione dei dati personali e la prevenzione dello sfruttamento dei dati costituiscono diritti fondamentali che le organizzazioni sono tenute a garantire. Una compromissione può comportare gravi violazioni della riservatezza, esponendo informazioni sensibili degli utenti a utilizzi impropri, furti di identità o discriminazioni.
- **Ragioni di Stato**: la sicurezza informatica rappresenta oggi un elemento strategico per la difesa nazionale. Il contesto attuale è caratterizzato da una *guerra ibrida*, in cui gli Stati organizzano operazioni offensive e difensive attraverso il cyberspazio. A titolo esemplificativo, la Russia conduce campagne periodiche volte a destabilizzare l'Unione Europea[1], prassi storicamente adottata anche dagli Stati Uniti in contesti geopolitici differenti. Analogamente, la Cina avrebbe istituito un dipartimento dedicato all'*offensive security*, con l'obiettivo di acquisire proprietà intellettuale su scala globale, con particolare attenzione al contesto statunitense.

Nel seguito del corso ci si concentrerà prevalentemente sulle ragioni economiche, in quanto più facilmente inquadrabili da una prospettiva ingegneristica.

In passato, le piccole e medie imprese ritenevano di non rappresentare obiettivi di interesse per gli attaccanti, limitando di conseguenza gli investimenti in sicurezza informatica. Tuttavia, è oggi evidente che la probabilità di subire un attacco è estremamente elevata per qualsiasi organizzazione, indipendentemente dalle dimensioni. Gli attori criminali adottano infatti strategie di tipo *pesca a strascico*, orientandosi non verso singoli obiettivi di alto valore, ma verso un elevato numero di bersagli con difese meno strutturate, massimizzando il profitto attraverso la numerosità delle compromissioni. Questo è reso possibile da una grande abilità nell'automazione nello sfruttamento degli attacchi e a una rete efficiente nel trasformare gli effetti degli attacchi (dati confidenziali, risorse computazionali, accessi non autorizzati) in valore. Parallelamente, le grandi organizzazioni risultano vulnerabili ad attacchi più sofisticati mirati alla loro catena di fornitura (*supply chain attack*). Compromettendo fornitori, partner commerciali o componenti software di terze parti

[1] Consiglio dell'Unione Europea, 2025

utilizzati dall'azienda target, gli attaccanti possono ottenere accesso indiretto a sistemi altrimenti ben protetti. Queste tipologie di attacchi, particolarmente insidiosi, sfruttano le relazioni di fiducia tra organizzazioni e la complessità delle moderne catene di approvvigionamento digitali.

Nel complesso, possiamo dedurre che gli investimenti nella cybersecurity sono incrementati notevolmente nell'ultimo periodo perché, nella maggior parte dei casi, il danno economico conseguente a un attacco supera di gran lunga il costo degli investimenti preventivi in sicurezza.

Le principali conseguenze economiche non si limitano ai danni **diretti** (ad esempio, trasferimenti fraudolenti di fondi), ma si articolano prevalentemente in impatti **indiretti**:

- Deprezzamento del valore azionario;
- Sanzioni, es. quelle comminate dal garante della privacy in caso di violazione delle legge europea sulla privacy (GDPR);
- **Costi di ripristino**, le spese necessarie per riportare il sistema allo stato operativo originale e implementare misure correttive volte a prevenire ulteriori compromissioni;
- **Perdita di produzione**, mancati guadagni dovuti a interruzioni o rallentamenti dei processi produttivi, mancata acquisizione di contratti;
- **Perdita di clienti e commesse**: mancati guadagni dovuti all'erosione della base clienti dovuta alla perdita di fiducia conseguente all'incidente (es. danno reputazionale).

1.1.3 Insicurezza: le cause profonde

In seguito al primo attacco informatico di rilevanza pubblica negli Stati Uniti, all'inizio degli anni 2000, l'amministrazione Clinton commissionò un documento analitico volto a identificare le cause profonde dell'insicurezza informatica. Le principali criticità individuate furono le seguenti:

- Le tecnologie di attacco evolvono rapidamente e secondo un modello *open-source*, risultando ampiamente accessibili e facilmente adattabili;
- Le tecniche e metodologie di difesa adottano un approccio reattivo. Ad esempio, una volta scoperta una vulnerabilità, si può proteggere un sistema che espone tale vulnerabilità solo dopo la pubblicazione di patch di sicurezza. Questo implica una condizione strutturale di ritardo, da parte dei difensori, fino all'implementazione delle correzioni, configurando una dinamica in cui i difensori inseguono costantemente gli attaccanti;
- Milioni di sistemi caratterizzati da livelli di sicurezza ridotti o assenti sono connessi a Internet, ad esempio i sistemi legacy, e rispetto al 2000, data della relazione, dovremmo includere anche i dispositivi IoT (Internet of Things);
- La rapida diffusione di Internet ha reso necessaria l'assunzione di personale privo di competenze specifiche in materia di sicurezza informatica;
- Lo sviluppo di software complessi è spesso affidato a programmatori senza formazione in pratiche di programmazione sicura;
- Gli attacchi e gli strumenti impiegati trascendono i confini geografici e le giurisdizioni nazionali;
- La complessità delle investigazioni sui crimini informatici, unitamente alle problematiche legate al diritto internazionale, rende improbabile il perseguimento effettivo dei responsabili.

L'analisi condotta oltre due decenni fa mantiene piena attualità e, per certi aspetti, descrive una situazione aggravata dall'incremento esponenziale del numero di dispositivi connessi, dalla complessità dei servizi erogati e dall'efficienza delle organizzazioni criminali che investono in cybercrime.

1.1.4 Verizon Data Breach Investigation Report

Diverse organizzazioni pubblicano periodicamente report di cybersecurity per scopi strategici e analitici. Tra queste, Verizon[2], azienda di telecomunicazioni statunitense, redige annualmente

[2] Verizion, 2025

un'analisi degli incidenti di sicurezza **dichiarati** e collezionati dal 1 novembre al 31 ottobre dell'anno precedente, conducendo indagini statistiche scientificamente rilevanti. Le informazioni che seguono sono di carattere generale e non si riferiscono ad una versione specifica del report.

Costi

I grafici contenuti nei report illustrano i costi diretti e indiretti derivanti dagli attacchi dichiarati, suddivisi per categoria di attacco. Dall'analisi emerge la perdita mediana e massima subita dalle organizzazioni per ciascuna categoria di attacco. Una sezione del report può inoltre includere i costi forensi, relativi alle analisi necessarie per determinare le responsabilità dell'incidente, al fine di avviare eventuali azioni legali volte al recupero parziale dei danni. Inoltre, i report indicano che molte aziende non hanno subito perdite a causa degli attacchi. Questa situazione, seppure probabile, potrebbe essere giustificata dal fatto che le aziende tendono a mantenere segrete le conseguenze degli attacchi o a minimizzarle, per evitare gli effetti collaterali (es. perdita di reputazione, perdita di valore delle azioni).

Come esempio, si riporta che negli ultimi anni, tra le tipologie più rilevanti figurano il BEC[3], il CDB[4] e il Ransomware.

Principali attori

Il report classifica le principali categorie di attaccanti, tra cui organizzazioni criminali strutturate, governi e minacce interne all'organizzazione, analizzandone le motivazioni dominanti, quali il profitto economico e l'acquisizione di informazioni riservate per finalità di spionaggio.

Dai report degli ultimi anni emerge inoltre la progressiva evoluzione del crimine informatico verso un modello industriale organizzato secondo una logica di *supply chain*: la produzione e distribuzione di malware, l'erogazione di servizi DDoS su commissione, il commercio di credenziali compromesse, la gestione di strutture di supporto fraudolente e le operazioni di riciclaggio costituiscono segmenti distinti ma interconnessi di un ecosistema criminale altamente specializzato. Tale frammentazione delle competenze ottimizza l'efficacia operativa delle campagne malevole, massimizzando i profitti dei criminali e rendendo al contempo estremamente complesso il tracciamento delle responsabilità individuali e l'azione giudiziaria contro i perpetratori.

Motivazione degli attaccanti esterni

I grafici in questione evidenziano le differenze nelle motivazioni degli attaccanti in funzione delle dimensioni dell'organizzazione bersaglio.

I report più recenti fanno emergere un pattern distintivo: gli attacchi diretti verso grandi organizzazioni sono frequentemente motivati da protesta o curiosità. Nel primo caso, gruppi hacktivisti o individui ideologicamente orientati scelgono obiettivi ad alta visibilità per amplificare il messaggio politico o sociale, mirando a danneggiare la reputazione dell'organizzazione o a rendere pubbliche pratiche ritenute scorrette. Nel secondo caso, attaccanti mossi da curiosità tecnica o dalla ricerca di prestigio nella comunità hacker tendono a sfidare i sistemi di difesa più sofisticati, considerando la compromissione di grandi aziende come una dimostrazione di abilità tecnica superiore. Al contrario, gli attacchi rivolti alle piccole e medie imprese sono quasi univocamente lanciati per motivazioni di natura economica.

[3]Business Email Compromise (BEC): attacco informatico sofisticato in cui un attaccante compromette o falsifica account di posta elettronica aziendali per indurre vittime a effettuare trasferimenti di fondi o a divulgare informazioni sensibili. Tipicamente l'attaccante si finge un dirigente, un fornitore o un partner commerciale fidato.

[4]Computer Data Breach (CDB): violazione della sicurezza che comporta l'accesso non autorizzato, la copia, la trasmissione o l'utilizzo di dati riservati. Include incidenti quali furto di credenziali, esfiltrazione di informazioni sensibili e accessi non autorizzati a database aziendali.

Access vector

L'*access vector* descrive da dove e con quali mezzi un attaccante può operare e descrive di solito anche il livello di privilegi necessario per condurre l'attacco. Può essere considerato il punto di partenza di un attacco.

L'analisi dei dati più recenti mostra l'evoluzione dei vettori di accesso utilizzati negli attacchi. Mentre l'abuso delle credenziali è in diminuzione, probabilmente grazie all'adozione dell'autenticazione a più fattori (MFA) e delle passkey, lo sfruttamento delle vulnerabilità risulta in crescita, indicando persistenti criticità nella qualità del codice. Il phishing, invece, mantiene un andamento costante, suggerendo che gli utenti non abbiano sviluppato maggiori capacità di discernimento, che l'uso degli smartphone faciliti errori di interazione a causa di click accidentali e che gli strumenti di AI generativa permettono di generare testi sempre più credibili e privi di errori grammaticali e inconsistenze (che insospettiscono gli utenti).

Asset

Un asset, in ambito di sicurezza informatica, è qualsiasi risorsa di valore per un'organizzazione che deve essere protetta. Può trattarsi di elementi tangibili, come server, dispositivi di rete o database, oppure intangibili, come software, dati, proprietà intellettuale o reputazione.

I grafici relativi alle sezioni sugli asset illustrano quali risorse aziendali siano maggiormente prese di mira e quali tipologie di informazioni vengano più frequentemente sottratte. Tra gli obiettivi principali figurano i server web, i più facilmente raggiungibili, e i server di posta elettronica, utilizzati per acquisire informazioni interne o dati personali.

Tipologie di attacco

Gli attaccanti conducono di solito campagne standard di attacco, non inventano ogni volta tecniche nuove. Per questo motivo, conoscere e categorizzare le tipologie di attacco può aiutare a capire come proteggersi meglio.

I grafici mostrano quali tipologie di attacco risultino più efficaci, evidenziando una significativa variabilità nel tempo. Tra le cause principali si individuano:

- Debolezze tecnologiche e del software (sfruttamento di vulnerabilità)
- Debolezze umane (phishing, errata consegna di informazioni, uso di credenziali compromesse)
- Debolezze organizzative (presenza di backdoor, diffusione di ransomware)

Tempo per rendersi conto

Non tutti gli attacchi sono finalizzati a creare danni immediatamente visibili. Anzi, alcune campagne di attacco puntano a rimanere nascoste subdolamente anche per lunghi periodi. Questo ci fa capire che è anche difficile, in alcuni casi, capire che siamo stati attaccati.

Le analisi riportano spesso la distribuzione del tempo necessario alle organizzazioni per rilevare un attacco subito. Questo fenomeno è parzialmente attribuibile al fatto che gli attaccanti non hanno sempre interesse a rendere pubblica la compromissione: mantenere un accesso persistente all'interno dell'organizzazione può generare maggiori profitti nel tempo. Mediamente, le organizzazioni impiegano diverse settimane per identificare un attacco, con estremi che si estendono a mesi o addirittura anni in alcuni casi.

Un ulteriore tipo di grafico si propone di illustrare i metodi più frequentemente impiegati dalle organizzazioni per identificare un attacco, tra cui spicca il monitoraggio dell'infrastruttura. Un accurato monitoraggio consente di rilevare anomalie comportamentali indicative di compromissioni, facilitando una risposta tempestiva ed efficace.

Tempo di recupero

Le conseguenze degli attacchi informatici possono essere vaste, quindi non è possibile rimediare a tutte le conseguenze in tempi necessariamente brevi.

Per questo motivo, vengono usati dei grafici che riportano i tempi mediani necessari per ripristinare le piene funzionalità operative in seguito a un attacco, nonché i tempi richiesti per rimuovere chiavi pubblicate erroneamente su repository pubblici.

Anche in questo caso, dai dati degli anni passati, si può osservare una significativa variabilità, con tempi di recupero che possono estendersi da giorni a settimane, a seconda della gravità dell'attacco e dell'efficacia delle misure di risposta implementate dall'organizzazione.

1.2 Minacce e problemi di base

L'analisi che segue si propone di illustrare i fattori che hanno condotto all'attuale scenario di vulnerabilità sistemica.

1.2.1 Problemi tecnologici di base

La sicurezza informatica è una problematica relativamente recente. Durante la progettazione delle prime reti, negli anni Settanta, l'obiettivo primario era garantirne il funzionamento operativo (e lo scopo era principalmente militare); la sicurezza non era considerata una priorità, anche in ragione del fatto che tali infrastrutture non erano accessibili al pubblico generale. Inoltre, l'uso di meccanismi crittografici comporta un costo computazionale che i sistemi dell'epoca non erano in grado di sostenere.

Per accedere a Internet, utenti privati e organizzazioni stipulano contratti con Internet Service Provider (ISP), che forniscono l'accesso alla propria infrastruttura di rete. In passato si sono verificati casi in cui dipendenti di ISP hanno sottratto o rivenduto informazioni transitate sulla propria rete, oppure hanno applicato politiche di instradamento anomale per frodare determinati utenti. In generale, dalla prospettiva di un'azienda, un ISP non è necessariamente un ente fidato o può addirittura essere un competitor. Tutto il traffico che attraversa l'infrastruttura di un ISP è tecnicamente accessibile ai suoi operatori, rendendo possibile, ad esempio, la lettura di segreti industriali da parte del personale tecnico di rete.

Un ulteriore problema riguarda l'autenticazione degli utenti, che risulta inadeguata sia per debolezze strutturali dei sistemi sia a causa di comportamenti utente imprudenti, quali l'adozione di password semplici, riutilizzate o già note agli attaccanti.

Infine, come già evidenziato, il software continua a essere sviluppato senza adeguati miglioramenti in termini di sicurezza.

1.2.2 Complessità

Storicamente, la sicurezza informatica era circoscritta alla protezione del sistema operativo. L'architettura tradizionale prevedeva un elaboratore centrale e N postazioni terminali prive di capacità computazionale autonoma (costituite da monitor e tastiera). In tale contesto, proteggere il sistema operativo equivaleva a proteggere l'intera infrastruttura aziendale.

Con l'evoluzione tecnologica, il panorama è mutato radicalmente: si è assistito alla proliferazione di calcolatori sempre più potenti, compatti e accessibili, fino all'avvento degli smartphone. Parallelamente, le infrastrutture di rete hanno subito una trasformazione profonda, con l'emergere di data center, soluzioni cloud, edge computing, fog computing e liquid computing. Tale evoluzione ha reso spesso indeterminabile il luogo fisico di esecuzione delle applicazioni, rendendo complessa l'identificazione del *perimetro di sicurezza* da presidiare.

Nell'attuale scenario, il tentativo di implementare misure di sicurezza si scontra con il seguente principio empirico:

Primo assioma dell'ingegneria. Più un sistema è complesso, più è difficile verificarne la correttezza (implementazione, gestione, funzionamento).

Il paradigma progettuale da adottare è pertanto il **KISS** (*Keep It Simple, Stupid*), che prescrive di mantenere il sistema il più semplice possibile: maggiore è la complessità, maggiore è la probabilità che un attaccante individui vulnerabilità sfruttabili.

Un'analisi interessante sulla correlazione inversa tra crescente potenza computazionale e prestazioni software è disponibile al seguente indirizzo:

`https://tonsky.me/blog/disenchantment/it/`

1.2.3 Altri problemi tecnologici

Ulteriori criticità di natura tecnologica possono derivare da:

- *Problemi di programmazione*: il software contiene bug o non è sviluppato secondo i principi del *secure coding* (ad esempio, validazione insufficiente dei dati di input, gestione inadeguata della memoria, assenza di controlli di accesso);
- *Problemi di progettazione*: la sicurezza non viene considerata come requisito fin dalle fasi iniziali di progettazione, ma viene aggiunta solo in una fase successiva. L'esperienza ha dimostrato che ignorare la sicurezza nelle fasi di concezione del sistema ha storicamente condotto a vulnerabilità gravi e alla necessità di riprogettazioni complete;
- *Configurazione di dispositivi e servizi*: una configurazione inadeguata di server, servizi o sistemi operativi può esporre vulnerabilità verso l'esterno. L'attività di *hardening*[5] richiede competenze tecniche specifiche;
- *Configurazione dei controlli di sicurezza*: molti dispositivi di sicurezza presentano elevata complessità gestionale. In determinati contesti, la configurazione delle politiche di sicurezza aziendali richiede la definizione di un numero considerevole di regole. Alcuni firewall analizzati per identificare errori di configurazione contenevano decine di migliaia di regole [6]. La gestione di tale complessità eccede le capacità cognitive umane, rendendo estremamente difficile prevedere le conseguenze delle interazioni tra le regole.

1.3 Attacchi e tecniche

L'analisi che segue esamina ad alto livello una serie di attacchi fondamentali, comunemente diffusi, presentando, ove disponibili, contromisure note ed efficaci.

Packet sniffing

Il *packet sniffing* è una tecnica di attacco che consiste nell'intercettazione e lettura di pacchetti di rete non destinati all'attaccante. In assenza di cifratura, il contenuto dei pacchetti risulta interamente leggibile, permettendo a chiunque di estrarre informazioni sensibili. Tali informazioni possono essere sfruttate in modo diretto, ad esempio per sottrarre credenziali di accesso e accedere ad account protetti, oppure in modo indiretto, rivelando dettagli interni di un'organizzazione utili per individuare ulteriori vulnerabilità da sfruttare. La contromisura principale consiste nella cifratura del payload dei pacchetti e nella limitazione dell'uso di reti broadcast. Le reti sono state sempre progettate per essere broadcast per ridurre i costi e semplificare la comunicazione; in questo tipo di reti, il packet sniffing risulta particolarmente semplice da realizzare. Reti che prediligono l'uso di switch al posto degli hub riducono l'impatto di questo problema, ma hanno costi decisamente maggiori.

[5] L'hardening è un processo di rafforzamento della sicurezza di un sistema mediante la riduzione della superficie di attacco. Include la disabilitazione di servizi non necessari, l'applicazione di patch, la configurazione restrittiva dei permessi e l'implementazione di controlli di sicurezza aggiuntivi.

[6] Wool, 2010

IP spoofing

Per *IP spoofing* si intende la falsificazione dell'indirizzo di rete di livello 3 (IP). La modifica dell'indirizzo IP è tecnicamente agevole per chi dispone di privilegi amministrativi (basta usare un tool a riga di comando o accedere alle finestre di configurazione della rete), motivo per cui nelle organizzazioni tali privilegi non vengono concessi indiscriminatamente. Questa tecnica consente di falsificare dati o accedere a sistemi in modo non autorizzato. Anche la falsificazione dell'indirizzo di livello 2 (MAC) è semplice, mediante i driver delle schede o manipolando le informazioni di rete delle macchine virtuali. Quindi, sarebbe più corretto parlare di *source address spoofing*. Ne consegue che non è possibile fare affidamento sugli indirizzi di livello 3 o di livello 2 per garantire l'autenticità della comunicazione, rendendo necessario l'impiego di meccanismi di autenticazione non basati su tali informazioni.

Shadow (or fake) server

Nell'attacco di tipo *shadow server*, un elaboratore non autorizzato finge di essere un server legittimo e può anche interporsi nella comunicazione tra un client e un server legittimo. Ciò può avvenire mediante intercettazione della richiesta tramite *packet sniffing*, seguita da *IP spoofing* per impersonare il server legittimo, oppure attraverso la manipolazione delle risposte DNS, in modo da indirizzare il client verso il server falso. In altri casi, sfrutta la mancanza di qualunque forma di autenticazione (es. DHCP) per rispondere al posto del server legittimo senza neanche dover impersonare altre entità. In tal caso, affinché l'attacco abbia successo nel primo caso, il server falso deve rispondere più rapidamente di quello autentico; per questo motivo, talvolta il server legittimo viene deliberatamente rallentato mediante attacchi DDoS. Gli attacchi di questo tipo mirano tipicamente a fornire un servizio fraudolento o apparentemente legittimo alla vittima, oppure a intercettare pacchetti destinati al server autentico. Attualmente, questo tipo di attacco viene contrastato mediante l'impiego di TLS, un protocollo di livello 6 dello stack OSI che stabilisce un'autenticazione unilaterale del server, preservando l'anonimato del client.

Denial of Service (DoS)

Con *Denial of Service* si identificano gli attacchi volti a saturare un host o un server di richieste o messaggi, impedendone la capacità di rispondere in tempi adeguati e compromettendone l'utilizzo per un periodo di tempo più o meno prolungato. Non esiste una soluzione definitiva contro questo tipo di attacchi: il sovradimensionamento dell'infrastruttura costituisce un approccio efficace ma non definitivo, poiché il volume di richieste può sempre eccedere le capacità disponibili. Esistono tuttavia sistemi in grado di rilevare un attacco in corso e, in caso positivo, disconnettere temporaneamente il sistema da Internet, rendendolo irraggiungibile. Al momento, lo strumento più efficace pare essere ancora lo sfruttamento delle relazioni tra persone (es. tra aziende e ISP) che possono collaborare per mitigare gli effetti di questi tipi di attacchi.

Distributed Denial of Service (DDoS)

Con l'aumento della capacità dei sistemi di gestire elevati volumi di traffico, gli attacchi DoS tradizionali, provenienti da un singolo elaboratore, sono diventati meno efficaci (es. un singolo utente non può saturare le risorse di grossi gestori o corporation). Per questo motivo si è sviluppata una variante più sofisticata: il *DDoS*. In questo scenario vengono coinvolti milioni di nodi, denominati *daemon* o *malbot*, che costituiscono una *botnet*. Questi nodi, che possono essere dispositivi comuni compromessi all'insaputa dei proprietari, vengono coordinati per inviare simultaneamente pacchetti verso un unico server bersaglio, moltiplicando l'effetto del DoS per il numero di nodi coinvolti. Un host diventa *daemon* attraverso il download di contenuti non sicuri o la navigazione su siti compromessi, installando processi latenti che vengono attivati successivamente da un computer *master* remoto.

Uno dei primi attacchi di questo tipo ad acquisire rilevanza pubblica portò alla redazione, da parte dell'amministrazione Clinton, di un documento contenente linee guida per prevenire simili eventi. Un caso notevole risale al 2016, quando, oltre ai dispositivi tradizionali, una grande quantità di dispositivi IoT venne infettata dal malware Mirai, riuscendo a compromettere il blog di Krebs, nonostante fosse protetto da Akamai, azienda statunitense specializzata in servizi CDN. Si ritiene che l'attacco fosse collegato alle analisi pubblicate dal blog riguardo ad attacchi simili ai danni di server di gioco online.

Va sottolineato che, secondo importanti studi di avvocati, il coinvolgimento involontario (anche se di solito dovuto a mancanza di misure idonee di sicurezza) di dispositivi nella botnet può consentire alla vittima dell'attacco di intraprendere azioni legali per il recupero dei danni.

Al giorno d'oggi, secondo Cloudflare, che rappresenta la principale fonte di report sul fenomeno DDoS[7], questi attacchi non sono soltanto più frequenti, ma anche più sofisticati e potenti rispetto al passato. Nel 2025 sono stati registrati attacchi con picchi di traffico di 7.3 Tbps, un aumento significativo rispetto ai 3,5 Tbps del 2024, con un flusso di 4.8 miliardi di pacchetti per secondo.

Tra le categorie di attaccanti più attive emergono i **competitor diretti**, che sfruttano il DDoS come strumento di sabotaggio economico per compromettere la disponibilità dei servizi rivali. L'obiettivo è causare perdite economiche dirette, erodere la fiducia dei clienti e danneggiare la reputazione dell'avversario. In settori altamente competitivi quali e-commerce, streaming e gaming online, anche brevi interruzioni del servizio possono spingere la clientela a migrare verso competitor che paiono garantire maggiore affidabilità. Tali attacchi sono frequentemente orchestrati in concomitanza con eventi critici come lanci di prodotti, picchi di vendite o campagne promozionali, al fine di massimizzarne l'impatto economico. La crescente disponibilità di servizi DDoS-as-a-Service sul dark web ha inoltre reso questa forma di sabotaggio accessibile anche ad attori privi di competenze tecniche avanzate.

Man In The Middle (o connection hijacking)

Negli attacchi di tipo *Man In The Middle* (**MITM**) l'attaccante si interpone nella comunicazione tra due host, acquisendo la capacità di leggere il traffico, manipolarlo, eliminare messaggi o aggiungerne di nuovi in tempo reale, compatibilmente con i tempi fisici di trasmissione. Una sottocategoria di questi attacchi è anche nota come *data spoofing*. Per contrastare questo tipo di attacchi è necessario garantire: riservatezza (solo il destinatario legittimo può leggere il messaggio), autenticazione (certezza che nessuno si stia fingendo il destinatario), integrità (il messaggio non deve essere modificato prima di raggiungere il destinatario) e serializzazione (assegnazione di un identificativo univoco a ogni pacchetto).

1.3.1 Malware

Il termine *malware* identifica l'insieme dei software progettati per arrecare danno ai sistemi informatici. Tra questi si distinguono:

- **Virus**: software malevolo caratterizzato da elevata aggressività, il cui obiettivo è danneggiare il sistema e replicarsi. La propagazione avviene spesso con la collaborazione involontaria degli utenti (es. aprire file sospetti, disabilitare i controlli dei sistemi operativi, scaricare crack).
- **Worm**: software malevolo il cui scopo principale è la replicazione e la propagazione verso il maggior numero possibile di sistemi, senza necessità di intervento umano. Sfrutta vulnerabilità di rete per diffondersi, saturando le risorse disponibili. Il fatto che si replichi senza controllo saturando le risorse rappresenta il danno arrecato.
- **Backdoor**: punto di accesso non autorizzato a un sistema. Può essere inserita intenzionalmente dal produttore dell'hardware per consentire aggiornamenti di firmware senza notificare

[7] The Cloudflare Blog, 2025

l'utente, oppure può essere creata da un trojan per permettere all'attaccante di accedere facilmente al sistema in qualsiasi momento.

- **Trojan**: programma apparentemente innocuo che nasconde al suo interno codice malevolo e che ha il ruolo di aprire una breccia nei sistemi da compromettere[8]. È progettato per colpire principalmente host individuali piuttosto che server, generalmente più protetti e dove si può contare mediamente meno sull'impreparazione del personale. Può rimanere latente per periodi prolungati mentre gli attaccanti individuano un punto di accesso al sistema. Una volta attivato può svolgere azioni dannose, aprire backdoor, esfiltrare dati, iniettare altri programmi che servono ad acquisire privilegi di amministrazione o bot per DDoS.
- **Rootkit**: insieme di strumenti nascosti per l'accesso privilegiato, installati nel sistema per consentirne l'amministrazione remota non autorizzata.

In generale, i malware continuano a dimostrarsi efficaci poiché sfruttano l'anello debole della catena di sicurezza: il fattore umano. Gli utenti contribuiscono involontariamente alla propagazione del malware. Per contrastare questa minaccia è necessario prestare particolare attenzione alla configurazione dei software e mantenere costantemente aggiornati i sistemi, senza rimandare indefinitamente tali operazioni.

1.3.2 Ransomware

Il *ransomware* è una categoria particolare di malware il cui obiettivo consiste nel cifrare i dati del sistema bersaglio, in tutto o in parte, al fine di richiedere un riscatto per la sua decifratura. In alcuni casi, gli attaccanti si fingono rappresentanti delle forze dell'ordine per aumentare la pressione psicologica sulla vittima. Come evidenziato dai dati, l'utilizzo di questo tipo di attacco mostra una tendenza crescente nel tempo, poiché molte vittime preferiscono pagare il riscatto senza denunciare l'accaduto per evitare ripercussioni reputazionali, sebbene il pagamento non garantisca il recupero dei dati. Sono comunque dei criminali e anche se non decifrare i dati crea loro danno reputazionale, non si esporranno né si preoccuperanno troppo nel caso non riuscissero a decifrare i dati.

Ransomware vs. organizzazione

Nelle organizzazioni, specialmente in quelle di grandi dimensioni, è prassi consolidata effettuare backup periodici dei dati, data l'impossibilità di tollerare la perdita di informazioni critiche. Tuttavia, nel caso ci servisse ripristinare i dati in seguito ad un attacco ransomware, possono emergere un insieme di criticità rilevanti. Prima di tutto, è importante capire quanto tempo prima dell'attacco è stato effettuato l'ultimo backup. Il backup potrebbe infatti contenere il ransomware in stato latente, non ancora attivato (*ransomware silente*). Ciò genera un circolo vizioso: il sistema viene compromesso, si ripristina il backup, ma il ransomware presente in esso si riattiva dopo un certo periodo, causando una nuova compromissione. L'utilizzo del cloud per il backup non cambia la situazione. Il ransomware può infiltrarsi e cifrare tutti i dischi di rete e le risorse cloud raggiungibili dalla macchina infetta. Quello che è importante è avere anche un backup offline su cui riversare periodicamente (es. una volta al giorno) i backup online (server sempre raggiungibili). In questo modo si riduce drasticamente la finestra di esposizione al rischio che il ransomware cifri anche quei dati.

Un caso emblematico di un problema di gestione dei backup riguarda un'organizzazione svedese che effettuava backup su nastro magnetico (molto più economico dei dischi per grandi quantità di dati), trasportato manualmente in un archivio situato in un edificio separato. Durante il trasporto, il nastro veniva appoggiato sui sedili riscaldati dell'auto, causando il danneggiamento del supporto. Il problema non fu rilevato tempestivamente poiché nessuno aveva mai verificato l'effettiva funzionalità dei backup. È quindi indispensabile verificare periodicamente le procedure di ripristino e la qualità dei dati salvati.

[8] Per approfondire: Fortinet, 2025

Un altro caso significativo coinvolse un archivio televisivo, in cui l'infezione da ransomware non era stata scoperta per un periodo talmente prolungato da compromettere decenni di contenuti audiovisivi. Questi episodi dimostrano che un backup eseguito senza criteri rigorosi non costituisce una protezione efficace contro i ransomware. Solo backup periodici, costantemente verificati e testati, possono garantire un recupero tempestivo in caso di compromissione.

1.3.3 Industria del Cybercrime

La diffusione degli attacchi informatici ha favorito lo sviluppo di una vera e propria industria del crimine informatico, strutturata come una *supply chain* in cui vengono commercializzati prodotti e servizi quali: malware progettati per attacchi specifici, servizi DDoS, credenziali sottratte, vulnerabilità e relativi exploit, call center impiegati per fornire i codici OTP in sistemi di autenticazione a due fattori comuni (più frequenti in USA) e *money mule*. Quest'ultima categoria si riferisce a individui che partecipano a operazioni di riciclaggio: quando vengono sottratti fondi da conti bancari, questi vengono trasferiti su conti intermediari gestiti dai *money mule*, i quali prelevano il contante tramite bancomat per renderlo non tracciabile. Per questo motivo, gli attori criminali tendono a raccogliere grandi quantità di dati, incluse le credenziali di accesso, per rivenderli sul *dark web*. I prezzi di tali servizi sono mantenuti relativamente contenuti per renderli accessibili a un'ampia gamma di acquirenti, privilegiando la numerosità delle transazioni rispetto al valore unitario elevato.

Come se non bastasse, questo fenomeno del Crime-as-a-Service (CaaS) ha reso il crimine informatico accessibile anche a individui privi di competenze tecniche avanzate, ampliando significativamente la base di potenziali attaccanti e contribuendo alla crescita esponenziale degli incidenti di sicurezza informatica a livello globale.

1.4 Altri problemi e minacce

Nel contesto della cybersecurity, l'essere umano rappresenta notoriamente l'elemento più vulnerabile della catena difensiva. Nonostante l'evoluzione delle tecnologie di protezione, quali firewall, crittografia e sistemi di rilevamento delle intrusioni, l'efficacia complessiva di un sistema informatico dipende in larga misura dal comportamento degli utenti, degli amministratori e degli operatori coinvolti.

Le principali aree in cui gli errori umani producono il maggior impatto sono:

- **Abuso di credenziali**: la scelta di password banali, l'utilizzo improprio, la condivisione o la gestione non sicura di credenziali (password, token, accessi) rappresentano una delle principali cause di compromissione dei sistemi. Attacchi quali il *credential stuffing* sfruttano credenziali sottratte o riutilizzate per accedere a servizi sensibili.
- **Azioni sociali**: tecniche di manipolazione psicologica, come il phishing o lo spear phishing, inducono gli utenti a rivelare informazioni riservate o a compiere azioni dannose, spesso sfruttando la fiducia o la pressione emotiva.
- **Errori**: configurazioni errate, mancati aggiornamenti, gestione superficiale delle risorse o semplici disattenzioni possono aprire vulnerabilità sfruttabili dagli attaccanti.
- **Malware**: l'esecuzione involontaria di software malevolo, spesso tramite allegati o link ricevuti via email o messaggi, può compromettere dati, sistemi e reti, propagandosi rapidamente a causa di comportamenti poco attenti degli utenti.

1.4.1 Alcuni esempi di attacchi riconducibili a errori umani

I successivi paragrafi riportano alcuni esempi di attacchi famosi. Analizzarli è utile per capire quali debolezze sono state sfruttate e come provare a rimediare.

Caso di studio 1: Wind Infostrada, ottobre 2017

Un caso emblematico della vulnerabilità dei sistemi digitali causata da errori umani è l'incidente Wind Infostrada del 2017[9]. Nei primi giorni di ottobre, migliaia di utenti della rete in fibra subirono un'interruzione totale della connettività: i modem forniti dall'operatore risultarono *bricked*, ovvero completamente inutilizzabili, resi inaccessibili da remoto e irrimediabilmente compromessi.

La causa fu una grave falla di sicurezza: i dispositivi avevano la porta Telnet (8023) aperta e accessibile da Internet, con credenziali di default impostate su `admin/admin`. Questa configurazione, pensata per permettere l'aggiornamento trasparente dei dispositivi, permise al malware BrickerBot di accedere ai modem e sovrascriverne il firmware con codice casuale. L'attacco, seppur distruttivo, ebbe l'effetto collaterale di impedire che i dispositivi venissero arruolati in botnet o sfruttati per attività malevole come furto di dati o mining illecito.

Un anno prima, un episodio simile ma su scala più ampia aveva coinvolto Deutsche Telekom, dove una vulnerabilità nei router aveva causato disservizi a centinaia di migliaia di utenti. Anche in quel caso, decisioni incaute, configurazioni errate, protocolli obsoleti e mancanza di valutazione dell'impatto di sicurezza di determinate scelte avevano aperto la porta a tentativi di compromissione su vasta scala.

Attualmente, per contrastare i rischi legati alla sicurezza informatica e alla gestione dei dati, molte organizzazioni hanno adottato sistemi di autenticazione significativamente più robusti. Tra le misure più efficaci si annovera l'**utilizzo di password uniche e complesse**, generate automaticamente e mai lasciate ai valori di default, al fine di evitare accessi non autorizzati dovuti all'uso di credenziali banali. A queste si affianca l'autenticazione a più fattori (**MFA**, Multi-Factor Authentication), che aggiunge un ulteriore livello di protezione per l'accesso alle aree riservate e ai sistemi critici. Un altro passo fondamentale è la **disabilitazione dei protocolli legacy** come Telnet, ormai considerati insicuri, e la loro sostituzione con soluzioni più moderne e cifrate come SSH o VPN. Infine, le organizzazioni più mature dal punto di vista della sicurezza implementano **audit periodici e sistemi di monitoraggio attivo**, capaci di rilevare tempestivamente anomalie e modifiche non autorizzate. Queste pratiche, se integrate in una governance chiara e accompagnate da una responsabilizzazione contrattuale dei fornitori, costituiscono oggi il fondamento di una resilienza operativa consapevole, capace di rispondere con efficacia alle minacce emergenti.

Caso di studio 2: i mercatini di Peshàwar, 2009

Nel 2009, nei mercatini di Peshàwar, città pakistana situata al confine con l'Afghanistan, furono rinvenuti diversi computer portatili appartenenti ai Marine statunitensi, messi in vendita da commercianti locali[10]. Questi dispositivi, presumibilmente smarriti o sottratti durante operazioni sul campo, contenevano documenti riservati, mappe operative e comunicazioni interne, spesso non protetti da alcuna forma di crittografia. La facilità con cui tali informazioni finirono in circuiti commerciali informali evidenziò quanto sia vulnerabile l'infrastruttura digitale quando non supportata da misure di sicurezza robuste e da una formazione adeguata del personale.

L'episodio andò oltre il danno economico di rimpiazzare l'hardware perso, poiché espose dati sensibili che avrebbero potuto essere sfruttati da gruppi ostili per pianificare attacchi o sabotaggi. La mancanza di protocolli di sicurezza adeguati, quali l'uso di crittografia forte per i dati a riposo (es. cifratura dei dischi) e l'implementazione di sistemi di tracciamento e cancellazione remota, evidenziò una lacuna significativa nella gestione della sicurezza informatica in contesti operativi ad alto rischio.

[9] securityinfo.it, 2017

[10] Corriere della Sera, 2009

1.4.2 Ingegneria sociale

Una delle vulnerabilità più insidiose nella sicurezza informatica è la predisposizione umana alla fiducia, specialmente in contesti professionali o sociali. Gli attaccanti sfruttano questa tendenza attraverso tecniche di **ingegneria sociale**, mirate a manipolare le vittime affinché compiano azioni che compromettono la sicurezza o rivelino informazioni riservate.

Phishing

Il *phishing* è una tecnica fraudolenta che consiste nell'invio di comunicazioni ingannevoli (tipicamente email, ma anche SMS o messaggi su piattaforme digitali) che simulano provenienza da enti affidabili, quali istituti bancari, organizzazioni o colleghi. L'obiettivo è indurre l'utente a fornire informazioni sensibili, come credenziali di accesso, dati personali o finanziari, o a compiere azioni che compromettono la sicurezza del sistema.

Esistono due varianti principali del phishing:

Spear phishing: attacco mirato a un individuo specifico (o un piccolo gruppo omogeneo), con contenuti personalizzati basati su informazioni raccolte in precedenza (ad esempio, ruolo aziendale, progetti in corso). L'elevato grado di verosimiglianza aumenta la probabilità di successo.

Whaling: forma di spear phishing rivolta a figure apicali (ad esempio, CEO, CFO), con comunicazioni che simulano richieste urgenti o transazioni strategiche. Data la posizione della vittima, il potenziale impatto è particolarmente elevato.

Pretexting

Il *pretexting* si basa sulla costruzione di un **pretesto credibile** per ottenere informazioni sensibili. L'attaccante si presenta con un'identità falsa e una narrazione coerente, spesso supportata da dettagli plausibili, per giustificare la richiesta. A differenza del phishing, il pretexting può avvenire anche tramite telefono o di persona.

Esempi comuni includono:

- Fingersi un tecnico IT che necessita di verificare le credenziali;
- Impersonare un collega del reparto risorse umane per aggiornare dati personali;
- Simulare una chiamata da parte della banca per confermare movimenti sospetti.

Tutte queste tecniche si basano sull'abilità dell'attaccante nel stabilire un rapporto di fiducia e sfruttare contesti familiari o l'urgenza. La consapevolezza e la formazione degli utenti sono strumenti fondamentali per contrastare questi attacchi.

Caso di studio 3: il caso Confindustria, 2017

Nel 2017, Confindustria fu vittima di una truffa informatica che portò alla perdita di circa 500.000 euro[11].

L'episodio, rimasto in parte avvolto nel mistero, rappresenta un caso emblematico di *Business Email Compromise*, una tecnica di attacco basata sull'ingegneria sociale e sull'inganno psicologico. I truffatori si finsero fornitori abituali dell'associazione, utilizzando un indirizzo email contraffatto ma verosimile. Il messaggio, redatto in modo professionale e contestuale (sapevano che il manager era impegnato in una riunione; hanno simulato lo stile di scrittura), richiedeva il pagamento urgente di una fattura su un nuovo conto bancario estero (in paradisi fiscali). La comunicazione faceva leva su elementi chiave dell'ingegneria sociale: l'urgenza dell'operazione, l'apparente autorevolezza del mittente e la familiarità con le procedure interne. Il bonifico fu eseguito senza ulteriori verifiche.

Questo evento dimostra quanto siano cruciali la formazione del personale, l'adozione di protocolli di verifica e la consapevolezza dei rischi legati alla manipolazione psicologica. L'ingegneria sociale non si basa sulla forza bruta, ma sulla capacità di sfruttare le debolezze umane e organizzative.

[11] Il Fatto Quotidiano, 2017

Caso di studio 4: phishing via Transformer 3, 2010

Nel 2010, la base aerea statunitense di Andersen (Guam) condusse un'esercitazione di sicurezza informatica per sensibilizzare il personale militare sui rischi del phishing[12]. L'esercizio consistette nell'invio di email false che annunciavano la presunta produzione del film Transformers 3 sull'isola, invitando i destinatari a candidarsi come comparse o membri dello staff. Il sito collegato alle email richiedeva informazioni personali e militari, simulando un attacco di phishing realistico.

L'obiettivo era duplice: testare la reattività del personale e generare consapevolezza sui rischi delle comunicazioni fraudolente. Tuttavia, l'esercizio ebbe effetti collaterali imprevisti. Molti destinatari condivisero l'informazione sui forum online, contribuendo alla diffusione virale della notizia. Il risultato fu un'ondata di speculazioni mediatiche locali sull'arrivo della produzione cinematografica, il che rese necessario il rilascio di una dichiarazione ufficiale per chiarire l'equivoco.

1.4.3 Attacchi famosi

Stuxnet

Nel 2009, l'attacco informatico alla centrale nucleare iraniana di Natanz segnò un punto di svolta nella storia della sicurezza informatica globale. Il malware Stuxnet[13], sviluppato congiuntamente dagli Stati Uniti e da Israele (secondo fonti non ufficiali), rappresenta il primo esempio documentato di arma cibernetica progettata per causare danni fisici a infrastrutture critiche. Non si trattava più di furto di dati o spionaggio digitale, ma di sabotaggio industriale su scala nazionale.

L'obiettivo era rallentare il programma nucleare iraniano senza ricorrere a un attacco militare convenzionale: una volta introdotto nei sistemi della centrale tramite una chiavetta USB, il worm si attivava solo in presenza di una configurazione specifica di controllori logici programmabili (PLC) Siemens, utilizzati per regolare la velocità delle centrifughe per l'arricchimento dell'uranio. Il malware alterava la velocità dei rotori senza che questa venisse riportata dagli strumenti di monitoraggio dell'infrastruttura, causando danni meccanici progressivi, mentre forniva feedback falsi agli operatori, rendendo difficile individuare l'origine del malfunzionamento.

La sofisticazione dell'attacco è testimoniata dall'impiego di almeno cinque vulnerabilità zero-day, un numero senza precedenti per un singolo malware. Inoltre, circa la metà del codice era dedicata esclusivamente ad evitare il rilevamento.

Brutal Kangaroo

Brutal Kangaroo[14], rivelato nel 2017 attraverso i leak di WikiLeaks, rappresenta una delle più sofisticate strategie di infiltrazione sviluppate dalla CIA per compromettere sistemi **air-gapped**, ovvero computer isolati fisicamente da qualsiasi rete. L'obiettivo era superare il paradigma della sicurezza per isolamento, dimostrando che anche i dispositivi scollegati da Internet possono essere bersagliati con successo. A differenza di Stuxnet, che mirava a sabotare infrastrutture critiche, Brutal Kangaroo era progettato per l'esfiltrazione di dati sensibili da reti altamente protette, con un controllo invisibile e duraturo.

Il framework si compone di una serie di strumenti modulari che operano in sinergia. Il punto di ingresso è solitamente un computer connesso alla rete, infettato tramite un payload nascosto in documenti o file eseguibili. Una volta compromesso, questo sistema agisce come *Primary Host* e prepara chiavette USB contenenti un malware secondario, chiamato *Drifting Deadline*, progettato per attivarsi quando la chiavetta viene inserita in un sistema air-gapped.

Il malware è in grado di raccogliere informazioni, esfiltrare dati e persino propagarsi ulteriormente, sfruttando vulnerabilità locali e tecniche di steganografia per nascondere i dati all'interno di

[12] The Register, 2010
[13] ICT Security Magazine, 2021
[14] securityinfo.it, 2017

file apparentemente innocui. L'intero processo è pensato per operare in modo silenzioso, con un alto grado di persistenza e capacità di evasione.

Black Energy

BlackEnergy[15] è un malware modulare adattato per colpire infrastrutture critiche. Originariamente sviluppato per attacchi DDoS e campagne di spam, ha subito una profonda evoluzione fino a diventare uno strumento di sabotaggio industriale.

La sua notorietà è legata in particolare all'attacco del dicembre 2015 contro la rete elettrica ucraina, in cui BlackEnergy fu utilizzato per compromettere sistemi SCADA e causare blackout in diverse regioni per diverse ore. L'operazione, attribuita a gruppi APT legati alla Russia, ha dimostrato come un malware possa essere integrato in una campagna multi-vettoriale.

BlackEnergy si distingue per la sua architettura modulare: il payload principale può essere esteso con plugin specifici per la raccolta di credenziali, l'esfiltrazione di dati, il controllo remoto e la distruzione di file. La sua capacità di persistenza e la compatibilità con ambienti industriali lo rendono particolarmente pericoloso in contesti SCADA e ICS.

L'attacco alla rete ucraina ha evidenziato anche l'uso di tecniche di spear phishing per la diffusione iniziale, con allegati malevoli mascherati da documenti legittimi. Una volta installato, il malware agiva in modo silente, raccogliendo informazioni e preparando l'ambiente per l'interruzione dei servizi.

Proprio per la sua natura, Black Energy è un esempio di **Advanced Persistent Threat (APT)**, una minaccia (Threat) sviluppata con tecniche avanzate (Advanced) che resta nascosta ed operativa per periodi molto lunghi (Persistent).

Mirai

Il malware Mirai[16] ha segnato una svolta epocale nella storia delle botnet, dimostrando il potenziale distruttivo dell'Internet of Things (IoT) quando sfruttato come vettore di attacco. Scoperto nel 2016, Mirai è progettato per infettare dispositivi IoT basati su processori ARC che eseguono versioni ridotte di Linux. Una volta compromessi, questi dispositivi diventano *zombie* all'interno di una botnet controllata da remoto, utilizzata principalmente per attacchi DDoS su larga scala.

Il meccanismo di infezione è semplice: Mirai effettua una scansione automatica della rete alla ricerca di dispositivi con credenziali di default non modificate. Una volta individuato un target vulnerabile, il malware si installa e stabilisce una connessione con un server di comando e controllo (C&C), pronto a ricevere istruzioni per lanciare attacchi coordinati. La categoria di IoT infetta era costituita da sistemi relativamente complessi ed in grado di trasmettere una grande quantità di dati (es. diversi tipi di baby monitor), quindi molto interessante per attacchi DDoS.

Il primo attacco noto fu diretto contro il sito di un esperto di sicurezza (Krebs on Security), ma il vero impatto si manifestò nell'ottobre 2016, quando un attacco DDoS contro il provider DNS Dyn causò l'interruzione di servizi come Twitter, Netflix e Reddit. L'attacco coinvolse oltre 100.000 dispositivi IoT compromessi.

Dopo l'arresto degli autori originali, il codice sorgente di Mirai fu pubblicato online e reso open source, dando origine a numerose varianti. Inoltre, la semplicità d'uso dei kit Mirai ha favorito la proliferazione del malware anche tra attori con competenze tecniche limitate.

Attacchi noti con Mirai includono:

- Krebs on Security (2016) - fino a 620 Gbps
- Ars Technica (2016) - fino a 100 Gbps
- OVH (2016) - fino a 1 Tbps
- Dyn (2016) - fino a 1.2 Tbps

[15] cybersecurity360.it, 2021
[16] Cloudflare, 2025

BlueBorne

BlueBorne[17] è una minaccia informatica emersa nel 2017 che ha evidenziato il protocollo Bluetooth come una nuova superficie d'attacco. BlueBorne non richiede che il dispositivo target sia connesso a Internet o che accetti connessioni Bluetooth in ingresso; è sufficiente che il Bluetooth sia attivo per comprometterlo in meno di dieci secondi.

Il malware è classificato come Trojan-Worm ibrido: una volta infettato, il dispositivo diventa a sua volta vettore di propagazione, cercando attivamente altri host vulnerabili nelle vicinanze. BlueBorne colpisce una vasta gamma di sistemi operativi, tra cui Android, iOS, Windows e Linux, rendendo vulnerabile qualsiasi dispositivo con Bluetooth abilitato.

Sfrutta diverse vulnerabilità, tra cui buffer overflow e problemi di gestione della memoria, per eseguire codice arbitrario con privilegi elevati. Una volta installato, può effettuare MITM, esfiltrare dati, installare ulteriori malware e persino consentire il controllo completo da remoto del dispositivo. Avendo colpito diversi tipi di sistemi di infotainement di veicoli l'impatto anche sulla sicurezza fisica (*safety*) è evidente.

1.5 Sicurezza: definizioni

La sicurezza informatica è l'insieme dei processi, delle tecnologie e delle politiche che mirano a proteggere le informazioni e i sistemi informatici da accessi non autorizzati, alterazioni, distruzioni o interruzioni dei servizi. L'obiettivo è ridurre i rischi entro limiti accettabili, garantendo la disponibilità, l'integrità e la riservatezza delle risorse informative.

Di seguito sono riportate le principali proprietà che caratterizzano la sicurezza dei sistemi informatici.

1.5.1 Riservatezza

La riservatezza è la proprietà che assicura che le informazioni siano accessibili esclusivamente a soggetti, entità o processi autorizzati, prevenendo così la divulgazione o l'esposizione dei dati a terze parti non autorizzate. Questo principio si applica tanto alle informazioni digitali (come file e dati trasmessi in rete) quanto a quelle fisiche e personali, incluse preferenze, dati relativi alla posizione geografica e dati sensibili, come l'origine etnica, le opinioni politiche, le convinzioni religiose o filosofiche, l'orientamento sessuale, dati biometrici, genetici e sanitari.

La riservatezza viene garantita attraverso un insieme coordinato di misure tecniche e organizzative, come crittografia end-to-end, protocolli sicuri, controlli degli accessi, sistemi di autenticazione e gestione delle credenziali.

La riservatezza è una proprietà distinta dalla privacy. Sebbene i metodi per garantirle siano spesso comuni, la privacy si focalizza più sulla protezione dei dati personali e sull'autonomia di ogni singolo individuo nell'uso e nella concessione di questi dati. La privacy è regolata da normative sulla protezione dei dati, come il GDPR, che definiscono modi accettabili di uso dei dati personali e sensibili degli utenti, limitano la profilazione e disciplinano la raccolta e il trattamento delle informazioni personali.

1.5.2 Integrità

L'integrità assicura che le informazioni e i sistemi rimangano corretti, completi e coerenti nel tempo. In una comunicazione, ciò significa che il destinatario deve ricevere esattamente i dati inviati dal mittente, senza modifiche, omissioni (*cancellazione/filtraggio*) o duplicazioni non autorizzate (*attacchi di tipo replay*). L'integrità si tutela attraverso tecniche che prevedono il calcolo di codici di controllo (resistenti agli attacchi), es., con funzioni di hash, firme digitali e sistemi di versionamento.

[17] ICT Security Magazine, 2017

Questi codici vengono calcolati sui dati al momento in cui il loro stato di integrità è certo (es. prima della trasmissione). Ricalcolando gli stessi a posteriori, quando i dati potrebbero aver subito modifiche, si riesce a capire se le modifiche ci sono state o meno.

Invece, omissioni e duplicazioni possono essere identificate numerando tutti i dati.

1.5.3 Autenticazione della controparte

L'autenticazione è il processo mediante il quale un'entità verifica l'identità di un'altra. Essa permette di accertare che chi o ciò con cui si sta comunicando sia effettivamente chi dichiara di essere. L'autenticazione può essere:

- *singola*, quando solo una delle due entità verifica l'identità dell'altra;
- *mutua*, quando entrambe le entità si autenticano reciprocamente.

L'autenticazione può basarsi su uno o più fattori: qualcosa che l'utente conosce (ad esempio, password), possiede (ad esempio, token o smart card) o rappresenta (ad esempio, dati biometrici). I principi dell'autenticazione e i meccanismi per ottenerli verranno trattati più in dettaglio nel capitolo **??**.

1.5.4 Autenticazione dei dati

L'autenticazione dei dati è la proprietà che consente di verificare l'origine e l'autenticità delle informazioni, garantendo che esse provengano effettivamente dal soggetto dichiarato. È strettamente legata all'integrità, poiché non avrebbe senso verificare l'origine dei dati se essi potessero essere modificati da terzi. Gli strumenti comunemente impiegati per l'autenticazione dei dati includono codici MAC (*Message Authentication Code*) e firme digitali.

L'autenticazione dei dati è necessaria tutte le volte in cui i dati vengono trasmessi. Infatti, dopo aver autenticato la controparte, è necessario controllare che i dati della comunicazione provengano ancora dalla stessa entità precedentemente autenticata (per come sono fatte le reti, un MITM potrebbe interferire con le comunicazioni). Inoltre, è necessaria quando si devono preservare dei documenti (es. atti o contratti).

1.5.5 Non ripudio

Il non ripudio è la proprietà che impedisce a un soggetto o a un'entità di negare in seguito di aver compiuto un'azione o generato determinate informazioni. Questa caratteristica garantisce un valore probatorio, spesso riconosciuto anche a livello legale, nelle comunicazioni digitali, nelle transazioni elettroniche e nei contratti informatici.

I meccanismi tecnici che lo supportano comprendono tipicamente firme digitali, protocolli crittografici e sistemi di tracciamento sicuro. Ma il non ripudio non è solo una questione tecnologica: riveste un ruolo centrale anche in ambito giuridico, dove leggi, regolamenti e la consulenza di avvocati definiscono come i dati e le prove digitali possano essere raccolti, conservati e validati affinché abbiano efficacia legale.

Ad esempio, oltre agli aspetti legali, per avere il non ripudio è richiesta anche la dimostrazione che da parte degli utenti ci fosse comprensione chiara delle operazioni che stavano svolgendo: un'azione digitale deve essere consapevole per poter attribuire responsabilità. È anche necessario che le interfacce informatiche siano ben fatte, prive di errori involontari o dolosi (es., cambiare il testo dei pulsanti di un'interfaccia grafica da Sì a No per ingannare gli utenti). In assenza di ciò, possono insorgere rischi come estorsioni digitali, phishing o l'uso di interfacce fraudolente che simulano operazioni legittime per ottenere firme o autorizzazioni ingannevoli.

In pratica, garantire il non ripudio richiede un approccio integrato che unisca strumenti tecnici e considerazioni normative, procedure contrattuali e responsabilità legali.

1.5.6 Autorizzazione (o controllo degli accessi)

L'autorizzazione è il processo che stabilisce quali operazioni un utente o un processo può eseguire su una risorsa. Si concretizza attraverso politiche di controllo degli accessi, che determinano i diritti di lettura, scrittura o esecuzione in base a ruoli, livelli di sicurezza o altre regole dell'organizzazione. La gestione del controllo di accessi può essere modellata con enormi matrici (multidimensionali) che uniscono, tra le altre cose, utenti, ruoli, risorse, privilegi e azioni permesse. Tuttavia, per migliorarne l'usabilità e ottimizzare lo sfruttamento (es. con dei workflow dedicati), sono stati proposti in letteratura diversi modelli. Tra i principali modelli di controllo degli accessi si distinguono:

- *Role-Based Access Control (RBAC)*: i permessi vengono assegnati in base ai ruoli degli utenti (es. amministratore, utente, sviluppatore), che vengono solo successivamente assegnati agli utenti, facilitando la gestione centralizzata dei privilegi;
- *Identity-Based Access Control (IBAC)*: i privilegi sono assegnati direttamente a utenti specifici, senza mediazione di ruoli o attributi; questo modello è semplice ma poco scalabile;
- *Attribute-Based Access Control (ABAC)*: le decisioni di accesso vengono prese in base a funzioni che verificano un insieme di attributi assegnati a utenti e risorse, rispetto ad azioni richieste e contesto (es. reparto, classificazione dei dati, luogo, orario di accesso);
- *Relationship-Based Access Control (ReBAC)*: i permessi derivano dalle relazioni tra entità; questi modelli sono tipicamente usati nei social network e in sistemi collaborativi (es. un utente può vedere i dati solo dei suoi "amici" o "colleghi");
- *Mandatory Access Control (MAC)*: l'accesso è regolato da una politica di sicurezza centralizzata implementata direttamente dai sistemi (hardcoded e quindi non configurabile o disattivabile) basata su classificazioni e livelli di sensibilità delle informazioni (es. sistemi militari in cui i documenti classificati come "riservati", "segreti", "top secret" sono accessibili solo da processi che abbiano etichette compatibili);

1.5.7 Protezione dei dati

La protezione dei dati comprende l'insieme delle misure tecniche e organizzative volte a preservare riservatezza, integrità e autenticità delle informazioni, sia durante la trasmissione (*dati in movimento*) sia durante la memorizzazione (*dati a riposo*). Essa può anche includere pratiche di cifratura, gestione sicura delle credenziali, backup periodici e controlli di accesso. A livello normativo, la protezione dei dati è strettamente connessa al rispetto della privacy e della conformità legislativa (ad esempio, GDPR).

1.6 Principi di sicurezza

I principi di sicurezza sono un metodo per riassumere in maniera chiara ed efficace metodologie e approcci corretti o specifici rispetto alla sicurezza informatica.

In diversi standard, è possibile trovare questi tre principi progettuali fondamentali:

- *Security by design*: indica il desiderio o la necessità di integrare la sicurezza sin dalle prime fasi di progettazione di un sistema. In contesti di questo tipo le soluzioni necessarie a garantire proprietà di sicurezza diventano parte integrante dell'architettura, dei processi e del ciclo di sviluppo. L'obiettivo è anticipare o prevenire vulnerabilità e minacce in modo proattivo, garantendo che i requisiti di sicurezza siano trattati come componenti essenziali del sistema e non come interventi correttivi successivi (approccio che negli anni si è rivelato fonte di diversi problemi e attacchi), rafforzando così la responsabilità di produttori e sviluppatori;
- *Security by default*: indica un approccio all'implementazione delle misure di sicurezza di un sistema che prevede che il sistema sia sicuro (o al livello di sicurezza più alto possibile) nella sua configurazione iniziale. Ciò implica che impostazioni, permessi e servizi predefiniti siano configurati con criteri restrittivi, trasferendo il compito di configurare correttamente la

sicurezza sul produttore e consentendo anche agli utenti meno esperti di operare in modo protetto, senza richiedere modifiche o interventi complessi da parte dell'utente finale. Questo principio prova a mitigare i rischi dovuti ai precedenti approcci, in cui i sistemi venivano preparati per essere più performanti e minimizzare i 'fastidi' dati dalle restrizioni di sicurezza all'usabilità. Per eliminare delle misure di sicurezza è necessario un intervento esplicito, che determina anche una responsabilità esplicita, es. dell'utente, nel caso il sistema fosse compromesso a causa di una vulnerabilità sfruttabile in seguito a tale eliminazione.

- *Defense in depth*: indica un approccio in cui si prova a rafforzare la sicurezza stratificando le protezioni su più livelli, combinando soluzioni tecniche, procedurali e organizzative e facendo uso di ridondanza. Questo approccio presuppone, correttamente, che nessuna singola misura sia sufficiente a garantire la protezione completa: se un livello viene violato, gli altri dovrebbero continuare a funzionare come barriere aggiuntive, riducendo la probabilità di una compromissione ed i conseguenti impatti.

Insieme, questi principi definiscono un approccio sistematico e robusto alla progettazione della sicurezza, che mira a prevenire, mitigare e gestire le minacce in modo efficace.

Il NIST[18] aggiunge altri due principi fondamentali:

- *Need-to-know*: un utente di un sistema informatico deve accedere esclusivamente alle informazioni strettamente necessarie per svolgere le proprie attività. Questo principio limita il numero di utenti che possono accedere a informazioni e di conseguenza l'esposizione ai dati confidenziali o sensibili; riduce quindi la probabilità che informazioni riservate vengano divulgate involontariamente o abusivamente;
- *Least privilege*: un processo o un utente deve disporre solo dei privilegi minimi indispensabili per portare a termine le operazioni perviste dal suo ruolo e dalle sue mansioni. Questo principio è pensato per evitare concessioni eccessive come diritti amministrativi non necessari, privilegi di sistema elevati o accessi permanenti a risorse critiche. L'implementazione di questo principio consente di limitare i potenziali danni causati da errori, compromissioni o comportamenti malevoli, e rende più semplice individuare attività anomale.

Sebbene strettamente collegati, i due principi operano su ambiti complementari: il *need-to-know* si concentra sul controllo dell'accesso ai dati e alle informazioni, mentre il *least privilege* si applica ai permessi operativi di utenti, applicazioni e processi. Insieme, costituiscono una base essenziale della sicurezza moderna, riducendo la superficie di attacco e favorendo una gestione granulare e responsabile delle risorse.

1.7 I tre pilastri della sicurezza

Il ciclo di gestione della sicurezza informatica è un processo continuo e iterativo che mira a garantire un livello di protezione adeguato e costante nel tempo. Anche in questo ambito sono state proposte diverse maniere per approcciare operativamente la sicurezza in maniera corretta. Uno di questi propone di strutturare le proprie attività secondo i tre pilastri della sicurezza[19].

Secondo questo approccio, prima di attivare le fasi operative, è essenziale una fase preliminare di *planning*, in cui si identificano i rischi, si definiscono politiche e requisiti di sicurezza, si stabiliscono priorità e si pianificano le procedure di risposta agli incidenti. Successivamente, il ciclo si articola in tre fasi principali:

1. *Avoidance*: implementazione di misure preventive per ridurre la probabilità e la superficie d'attacco, come firewall, segmentazione di rete, VPN, formazione degli utenti e politiche di aggiornamento. Questa fase non elimina completamente il rischio: anche i controlli

[18] **National Institute of Standards and Technology**: agenzia del governo degli Stati Uniti che pubblica linee guida e standard in materia di sicurezza informatica.

[19] Attenzione, esiste anche una versione, leggermente diversa, che propone quattro pilastri.

più robusti possono fallire o essere aggirati (anche se la defense in depth è implementata), rendendo necessario prevedere sempre misure di supporto, indicate dai pilastri successivi;

2. *Detection*: monitoraggio continuo del sistema per identificare tempestivamente anomalie, attività sospette o indicatori di compromissione (che potrebbero rivelare il fallimento dei sistemi di *avoidance*), tramite strumenti come sistemi IDPS, analisi dei log, monitoraggio del traffico di rete e tecniche di threat intelligence. Anche qui è possibile il fallimento: non tutte le minacce vengono rilevate immediatamente e alcune potrebbero rimanere latenti o mascherate;
3. *Investigation*: attività di analisi e risposta agli incidenti, come analisi forense, auditing e incident response, per comprenderne origine, dinamiche e impatti, ripristinare la normale operatività e adottare misure correttive. L'analisi forense è inoltre indispensabile in tutti quei casi in cui si configura una violazione delle leggi/norme/standard o se è necessario acquisire informazioni da usare in sede processuale. Non sempre l'indagine produce risultati completi o immediati, e possono emergere lacune informative o operative da colmare.

Ogni fase può rivelare criticità, rendendo necessaria una revisione delle misure adottate. Per questo il ciclo è ricorsivo: gli esiti dell'investigazione e della rilevazione alimentano un nuovo processo di pianificazione, migliorando continuamente procedure, controlli e strategie difensive. In altre parole, la sicurezza non è mai definitiva, ma richiede un aggiornamento dinamico per adattarsi all'evoluzione delle minacce e dei sistemi.

2. Gestione dei rischi informatici

La gestione del rischio informatico (*Risk Management*) si configura come un'attività essenziale in ogni azienda, con l'obiettivo primario di **quantificare i rischi e ridurli entro limiti tollerabili** tramite l'applicazione di un **processo ripetibile e oggettivo**. Questo approccio metodologico è necessario per minimizzare i costi diretti e indiretti derivanti da incidenti e attacchi. Ma diventa indispensabile nel caso si fosse interessati ad aderire a standard (come la famiglia ISO/IEC 27000) e normative (come il GDPR) che richiedono esplicitamente un'analisi del rischio. Il processo deve essere il più possibile scientifico e rigoroso per evitare che la valutazione del rischio sia arbitraria o soggettiva e, in ultima istanza, inutile.

2.1 Fondamenti e Terminologia del Rischio

Il processo di gestione del rischio richiede la precisa definizione e distinzione dei seguenti elementi:

- **Asset**: qualsiasi risorsa che abbia valore per l'azienda e sia necessaria all'erogazione di un servizio IT, inclusi beni materiali (es. server) e immateriali (es. reputazione, proprietà intellettuale).
- **Debolezza (*Weakness*)**: una predisposizione del sistema a manifestare problemi, come codice scritto male o configurazione non attenta.
- **Vulnerabilità (*Vulnerability*)**: una debolezza scoperta e **sfruttabile direttamente dagli attaccanti** per violare le proprietà di sicurezza di un sistema o di una rete.
- **Minaccia (*Threat*)**: Un evento (intenzionale o accidentale) che, se sfruttato, può causare la perdita di una o più proprietà di sicurezza (es. Confidenzialità, Integrità, Disponibilità).
- **Attacco**: il verificarsi di una minaccia di tipo intenzionale.
- **Evento Negativo**: il verificarsi di una minaccia di tipo accidentale (es. guasto hardware o errore umano).

2.2 Vulnerabilità e CVE – Common Vulnerabilities and Exposures

Le vulnerabilità di sicurezza informatica note e rese pubbliche sono descritte in modo standardizzato nel database **Common Vulnerabilities and Exposures (CVE)**, mantenuto dalla **MITRE Corporation**. Le CVE riportano un insieme standard di attributi che servono a caratterizzarle:

- **CVE-ID**: ogni vulnerabilità ottiene un ID univoco (es. CVE-2017-5754), una descrizione testuale che e una serie riferimenti per approfondimenti. L'ID è composto da una stringa fissa (CVE), l'anno in cui la CVE è stata resa pubblica, e un numero incrementale azzerato all'inizio di ogni anno.
- **CVSS (Common Vulnerability Scoring System)**: è un punteggio da **0 a 10** che fornisce un'idea del **danno potenziale** che lo sfruttamento della CVE può causare. Questo punteggio è calcolato mediante una formula standard a partire dai valori di **metriche base** che vengono esplicitamente associati alla CVE:
  - *Attack Vector*: da dove può essere lanciato l'attacco (ad esempio, accesso fisico, rete locale o remoto).

  - *Attack Complexity* e *Privileges Required*: La difficoltà che un attaccante può incontrare nello sfruttamento e le competenze che servono, e i permessi necessari allo sfruttamento (da nessun privilegio a privilegi di root).
  - *Impatto*: l'impatto potenziale descritto da tre metriche distinte per Confidenzialità, Integrità e Disponibilità (CIA).

Le vulnerabilità sono classificabili come **Non note** (se ancora nessuno l'ha scoperta) o **Note**; in quest'ultimo caso si può specializzare la classificazione in:

- *zero-day*, da quando viene scoperta (da un hacker o un ricercatore) fino a quando non viene pubblicata una *patch*.
- *unpatched*, da quando viene pubblicata la patch a quando viene installata sul dispositivo vulnerabile.
- *patched*, successivamente all'installazione della patch.

Una **patch** è una porzione di codice che rende innocua una vulnerabilità [1].

2.3 CWE e Window of Exposure

La **CWE (Common Weakness Enumeration)** è una tassonomia che cataloga gli errori di programmazione, configurazione e gestione che potrebbero essere rilevati nel software e nell'hardware. Queste debolezze devono essere monitorate da chi sviluppa il codice poiché potrebbero trasformarsi in vulnerabilità sfruttabili (CVE). Un esempio comune è la **CWE-20** (*Improper Input Validation*). Non c'è la certezza che una debolezza riuscirà ad essere sfruttata, ma di sicuro va monitorata ed eliminata se possibile per anticipare i problemi.

2.3.1 Esempio di Vulnerabilità Critica: Meltdown (CVE-2017-5754)

Un esempio notevole di vulnerabilità catalogata è **Meltdown** (CVE-2017-5754)[1][2][3]. Questa vulnerabilità è associata ai sistemi dotati di microprocessori (Intel) che utilizzano l'**esecuzione speculativa** e la predizione di salto (*indirect branch prediction*) per ottimizzare le prestazioni. Sfruttando questa tecnica, un attaccante con accesso fisico all'apparecchiatura può eseguire un attacco di **side-channel** della *data cache* per avere accesso non autorizzato ad informazioni riservate. Meltdown è correlata alla debolezza del codice **CWE-200** (*information exposure*). Le mitigazioni software/microcodice introdotte per correggere Meltdown hanno portato a una **riduzione drastica delle performance** (anche oltre il 30%) in molti processori affetti.

2.3.2 EPSS – Exploit Prediction Scoring System

Per completare le informazioni su quanto possono essere pericolose le vulnerabilità per un'azienda interessata a svolgere un'analisi dei rischi, per completare il punteggio CVSS, che descrive il massimo danno, è stato introdotto l'**EPSS (Exploit Prediction Scoring System)**. Questo parametro stima la **probabilità** che la vulnerabilità possa essere sfruttata nel mondo reale in un determinato momento basandosi su dati dinamici e osservazioni ottenute (*in the wild*), cioè dati reali raccolti da Internet. L'organizzazione nonprofit FIRST (Forum of Incident Response and Security Teams) mantiene aggiornate le statistiche ed i punteggi EPSS.

Esempio: EPSS di Meltdown

Considerando la vulnerabilità Meltdown (CVE-2017-5754), è interessante osservare come l'EPSS sia variato nel tempo. Al momento della divulgazione pubblica (gennaio 2018), il punteggio EPSS era relativamente basso (circa `0.02`, ovvero probabilità del 2% di sfruttamento nei successivi 30

[1] NIST, 2017
[2] MITRE, 2017
[3] Google Project Zero, 2017

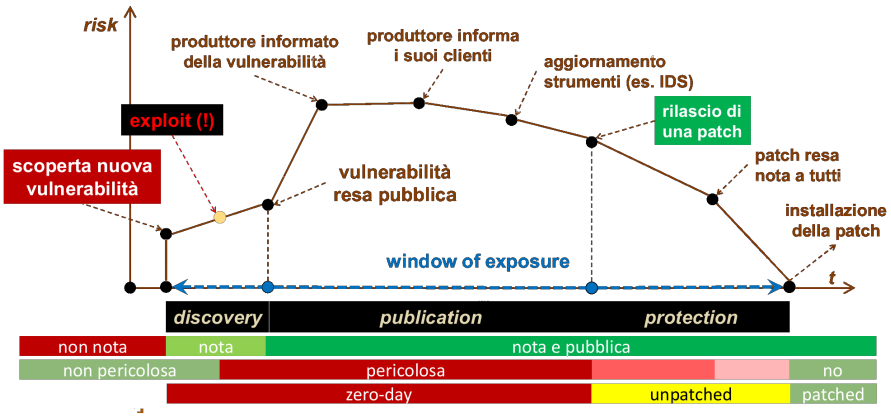


Figura 2.1: Finestra di esposizione

giorni) nonostante l'elevato CVSS di 5.6, poiché lo sfruttamento richiedeva competenze tecniche avanzate e accesso locale al sistema. Tuttavia, con la pubblicazione di *proof-of-concept* pubblici e l'aumento della consapevolezza nella comunità di sicurezza, l'EPSS è gradualmente aumentato nelle settimane successive, raggiungendo picchi superiori al 5% quando sono stati rilasciati tool automatizzati per lo sfruttamento. Dopo la disponibilità delle patch e delle mitigazioni software, l'EPSS è progressivamente diminuito, stabilizzandosi attorno allo 0.5% nei mesi seguenti. Questo andamento dimostra come l'EPSS catturi dinamicamente l'evoluzione del rischio reale, fornendo informazioni complementari al CVSS statico e permettendo una prioritizzazione più accurata degli interventi di patching.

2.4 Window of Exposure

Il concetto di **window of exposure** (finestra di esposizione) definisce l'intervallo temporale ad alto rischio che intercorre tra la **scoperta di una vulnerabilità** e la sua **correzione**.

- La vulnerabilità passa dallo stato *non nota* a *nota* quando viene scoperta.
- Diventa **pericolosa** quando viene trovato un **exploit**, cioè un pezzo di codice che la sfrutta. Un exploit dimostra che si tratta in effetti di una vulnerabilità.
- Il rischio aumenta drasticamente quando la vulnerabilità viene **resa pubblica** (disclosure), perché il numero di attaccanti potenzialmente in grado di trovare exploit e sfruttarla aumenta.
- Durante l'attesa del rilascio della patch, è necessario aggiornare gli strumenti di monitoring (es. IDS – Intrusion Detection System) per rilevare tentativi di sfruttarla.
- Secondo il diagramma proposto, dopo il rilascio della patch, il livello globale di rischio cala, fino al momento in cui la patch viene installata su tutti i dispositivi affetti. In realtà, qualche studioso propone una visione diversa. Infatti, una volta rilasciata una patch, gli attaccanti possono farne reverse engineering, scoprire il problema, la vulnerabilità e preparare un exploit più facilmente. È quindi importante applicare la patch prima possibile perché dopo il rilascio il numero di attaccanti in grado di poterla sfruttare potrebbe aumentare.
- Le vulnerabilità **zero-day** sono quelle scoperte per le quali non esistono ancora patch. Si può venire a conoscenza delle zero-day sul mercato illegale nel dark web oppure perché sono state rese pubbliche tramite canali ufficiali (di solito dopo la disponibilità di una patch).

L'organizzazione **Zero Day Initiative (ZDI)** propone una gestione responsabile delle vulnerabilità, operando come intermediario tra i ricercatori di sicurezza e i fornitori di software. Una volta scoperta una vulnerabilità, ZDI notifica privatamente il vendor interessato, stabilendo una *deadline* di 120 giorni per il rilascio di una patch correttiva. Qualora il fornitore non rilasci la correzione entro tale termine, ZDI procede con la divulgazione pubblica della vulnerabilità (*full disclosure*),

con conseguenze negative per la reputazione del vendor e un incremento sostanziale del rischio per gli utenti del software vulnerabile.

2.5 Risk Management Framework (RMF)

Il modello di riferimento consigliato dal **NIST** per la gestione dei rischi è il **RMF**, articolato in quattro fasi:

1. **Risk Framing (Preparazione)**. Questa fase iniziale stabilisce il contesto e i presupposti per l'intero processo di gestione del rischio. Si stabilisce l'ambito del sistema da proteggere, definendo chiaramente i confini organizzativi e i dettagli tecnologici. Si identificano gli asset materiali e immateriali (come la reputazione aziendale) e si determina il loro valore per l'organizzazione. L'output principale è la descrizione dettagliata della **metodologia di valutazione** che verrà applicata, includendo le metriche da utilizzare e tutte le formule per il calcolo dei rischi, i criteri per caratterizzare quando dei rischi sono accettabili e le strategie di gestione. La metodologia definita durante il framing deve essere **ripetibile** per garantire coerenza nel tempo e tra diverse valutazioni, oltre a preservare il *know-how* organizzativo anche in caso di turnover del personale tecnico.
2. **Risk Assessment (Valutazione)**. In questa fase si procede all'identificazione sistematica delle minacce (sia intenzionali che accidentali), delle debolezze architetturali e implementative, e delle vulnerabilità tecniche (interne ed esterne al perimetro aziendale). Per ciascun rischio identificato si stimano gli **impatti** potenziali, che possono includere danni finanziari diretti e indiretti, interruzione o blocco dei processi critici, perdita di reputazione, e conseguenze legali o regolamentari. Parallelamente, si valuta la **probabilità** di accadimento (*likelihood*) di ogni scenario di rischio, considerando fattori quali la presenza di controlli esistenti, le capacità degli attaccanti, e l'attrattività del target. L'obiettivo è valutare tutti (o la maggior parte) dei rischi rilevanti, producendo una mappatura completa del profilo di rischio delle parti dell'organizzazione, coerentemente con quanto deciso in fase di framing.
3. **Risk Response/Mitigation (Risposta e Mitigazione)**. Sulla base dei rischi identificati e quantificati, si individuano e valutano le possibili soluzioni di mitigazione. Ogni soluzione viene analizzata in termini di **costo** di implementazione e mantenimento, **tempo necessario** per la realizzazione e messa in opera, e **impatto** effettivo sulla riduzione del rischio (sia in termini di probabilità che di conseguenze). La valutazione deve considerare diverse alternative, tra cui l'accettazione del rischio (quando il costo della mitigazione supera il danno potenziale), il trasferimento del rischio (ad esempio tramite assicurazioni), l'annullamento del rischio (es. eliminando l'attività che lo genera), o la sua riduzione attraverso controlli tecnici e organizzativi. La decisione finale deve essere motivata oggettivamente e documentata, anche nel caso si scelga di accettare o ignorare il rischio. Una volta selezionate le contromisure, si procede all'implementazione delle soluzioni scelte secondo le priorità definite.
4. **Risk Monitoring (Monitoraggio)**. Questa fase non è conclusiva ma continuativa. Essa valuta l'efficacia delle mitigazioni implementate sul campo, verificando che i controlli funzionino come previsto e raggiungano effettivamente gli obiettivi di riduzione del rischio. Si controlla la validità delle assunzioni fatte nelle fasi precedenti, poiché il contesto operativo, il panorama delle minacce e le tecnologie evolvono costantemente. Il monitoraggio include la verifica della conformità alle policy di sicurezza, l'analisi degli indicatori di compromissione, e la raccolta di metriche sull'efficacia dei controlli. L'obiettivo è mantenere il processo aggiornato e adeguato ai cambiamenti del panorama delle minacce e dell'infrastruttura IT. Quando si rilevano scostamenti significativi, modifiche architetturali, nuove minacce o inefficacia delle mitigazioni, il processo viene iterato: si può ricominciare dal *Risk Framing* per una revisione completa, oppure tornare a una fase precedente specifica per aggiustamenti mirati, mantenendo così il profilo di gestione del rischio sempre aggiornato.

2.6 Vulnerability Assessment

Il **Vulnerability Assessment** è un processo che valuta l'esposizione aziendale alle vulnerabilità, focalizzandosi prevalentemente sulle minacce di rete. Esso non è da considerarsi una fase indipendente dell'RMF, è infatti un'attività che potrebbe essere necessaria come sottofase del Risk Assessment. Il Vulnerability Assessment consiste in quattro fasi:

1. **Planning**. Definire lo *scope* (cosa valutare) e lo *schedule* (quando svolgerlo).
2. **Information Gathering**. Acquisire informazioni sugli obiettivi (topologia, host, software installati). Per quanto riguarda gli host, è fondamentale identificare il sistema operativo e la loro versione per associare le vulnerabilità note.
3. **Scanning**: Cercare vulnerabilità note (CVE). Le tecniche di scansione includono:
   - *host discovery*[4]: ad esempio con ICMP echo request, ARP request, TCP SYN, UDP.
   - *port scanning*: per classificare le porte in *open* (c'è un servizio raggiungibile in ascolto), *closed* (non in ascolto), *filtered* (non raggiungibile, ad es. un firewall blocca la connessione).
   - *OS detection*: alcuni scanner sono in grado di determinare il sistema operativo (e alcune volte la versione) tramite delle euristiche in base a come gli host rispondono a determinate sequenze di pacchetti.
   - *TCP Sequence Predictability Classification*: lo scanner cerca di determinare il prossimo sequence number che userà il TCP, per testare la bontà del generatore di numeri casuali; in caso sfavorevole, un attacco di tipo spoofing potrebbe essere attuabile.
4. **Report**: Stesura di un documento che presenti i risultati e includa le proposte per le mitigazioni in base a vari fattori (la migliore, la più conveniente e quella suggerita).

2.6.1 Vulnerability Assessment tools

Esistono dei software in grado di automatizzare questo processo, almeno nei punti 3 e 4: difatti, questi programmi si occupano di trovare i target raggiungibili, identificare e categorizzare le vulnerabilità e produrre dei report complessivi. Tra questi tool, alcuni tra i più famosi sono:

- **nmap**, CLI (*Command-Line Interface*) può essere usato anche da attaccanti per effettuare qualunque tipo di scansione; ha una documentazione molto ricca, ma per usarlo in maniera consistente ed esaustiva in ambienti aziendali, è necessario creare script che automatizzino vari tipi di ricerche e che supportino scansioni in diverse parti della rete aziendale. Inoltre, genera report molto sintetici o disorganizzati. È particolarmente adatto per scansioni mirate e personalizzate, ma risulta meno pratico in scenari aziendali complessi.
- **GVM (OpenVAS)**, è una soluzione open source per l'automazione dello scanning in ambienti aziendali. Rappresenta una soluzione integrata che combina discovery, scanning e reporting in un'unica piattaforma, particolarmente indicata per ambienti enterprise con reti articolate e numerosi asset da monitorare. La versione free è accessibile a tutti, ma se si vogliono identificare le vulnerabilità più recenti o quelle per sistemi non Linux è necessario pagare. Esistono anche delle *appliance*, più costose, pensate per facilitare le scansioni.
- **Nessus**, una volta il più utilizzato, anche perché open source e gratis. Dopo averlo reso chiuso e a pagamento (anche se si può installare e si possono lanciare alcune scansioni gratis), è stato creato OpenVAS come suo fork. Al contrario delle prime versioni, che permettevano di lanciare gli exploit solo premendo un tasto, le versioni più recenti e i loro fork hanno disabilitato le funzioni per evitare danni fatti da script kiddy[5]. Offre funzionalità avanzate di

[4] questo passo viene spesso associato anche all'Information Gathering

[5] Script kiddy è un termine dispregiativo usato per indicare individui con scarse competenze tecniche che utilizzano strumenti automatizzati sviluppati da altri per condurre attacchi informatici, spesso senza comprendere i meccanismi sottostanti né le conseguenze delle loro azioni.

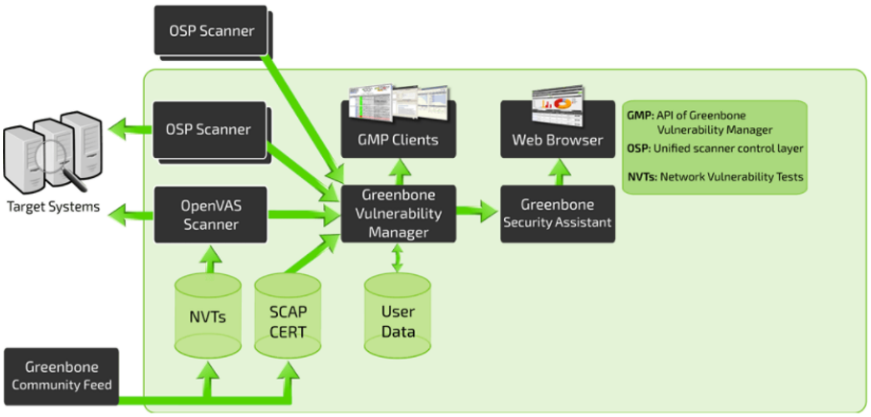


Figura 2.2: Architettura di GVM (Fonte: Medium)

gestione centralizzata e reportistica, rendendolo appropriato per organizzazioni di medie e grandi dimensioni.

- **Acunetix**, software proprietario, completo, molto rifinito anche nelle interfacce grafiche e quindi molto costoso, che offre anche un'analisi approfondita delle vulnerabilità delle applicazioni web, con capacità di scanning e correlazione automatica dei risultati.

La scelta tra questi strumenti dipende dalle dimensioni e dalla complessità dell'infrastruttura da proteggere: mentre **nmap** è eccellente per analisi puntuali e test di sicurezza mirati, i tool integrati come **GVM** e **Nessus** sono più adatti ad ambienti enterprise con reti vaste e complicate, dove è necessario gestire sistematicamente migliaia di dispositivi, consolidare i risultati da multiple sorgenti, mantenere database storici delle vulnerabilità, e generare report conformi agli standard di compliance. Questi strumenti integrati offrono inoltre funzionalità di orchestrazione, scheduling automatico delle scansioni, gestione centralizzata delle policy, oltre ad integrarsi con i sistemi di ticketing e workflow aziendali.

Esempio: OpenVAS

In Figura 2.2 viene presentata l'architettura di **OpenVAS** (Open Vulnerability Assessment System), che recentemente ha cambiato nome in GVM. GVM è composto da più componenti modulari che collaborano per eseguire la scansione e l'analisi delle vulnerabilità di sistemi e reti. Gli *OpenVAS Scanner*, sono i componenti responsabili dell'esecuzione effettiva dei test di vulnerabilità e per questo motivo devono essere posizionati in maniera tale da riuscire a svolgere le scansioni senza interferenze (es. nella stessa sottorete per evitare i filtri imposti dai firewall). In contesto GVM, i test sono definiti Network Vulnerability Tests (NVTs), e sono basati sul linguaggio NASL (Nessus Attack Scripting Language). Questi test sono forniti dal *Greenbone Community Feed*, un repository costantemente aggiornato di script di scansione, sebbene gli aggiornamenti più recenti siano a pagamento.

In alternativa agli scanner nativi OpenVAS, è possibile collegare altri tool di scansione; l'importante è che possano essere configurati tramite il protocollo *OSP (Open Scanner Protocol)*.

La gestione e il controllo delle attività di scansione sono demandati al *Greenbone Vulnerability Manager*, che coordina l'esecuzione delle scansioni interfacciandosi con gli scanner perché analizzino i vari target, e gestisce la memorizzazione dei risultati. Il *Greenbone Security Assistant (GSA)* è il server che espone un'API che permette la configurazione del Greenbone Vulnerability Manager mediante un'interfaccia web. Tramite browser gli utenti possono configurare le scansioni, visualizzare i report e gestire i risultati in modo interattivo. Tutti questi componenti comunicano tramite protocolli dedicati, in particolare l'*OpenVAS Management Protocol (OMP)* e il *Greenbone Management Protocol (GMP)*, che garantiscono un'interazione sicura e strutturata tra le diverse

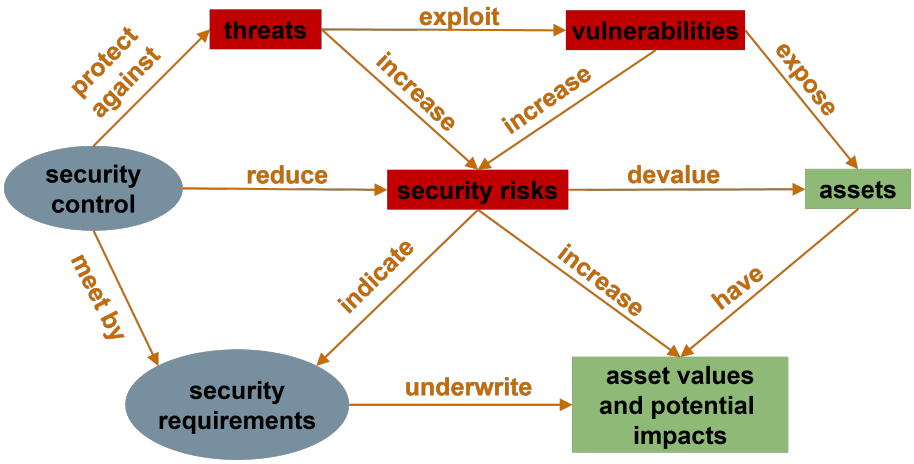


Figura 2.3: Un possibile modello per rappresentare le relazioni tra i concetti legati al rischio

entità del sistema. L'architettura modulare consente un'elevata scalabilità e flessibilità, rendendo OpenVAS adatto sia a piccoli ambienti di test che a infrastrutture di sicurezza di livello enterprise.

Infine, le funzionalità del Greenbone Vulnerability Manager sono anche accessibili in maniera programmatica tramite una libreria Python; è quindi possibile usare dei *GMP Client* dedicati ed automatizzare tramite script la gestione delle scansioni ed il processamento dei risultati.

2.7 Valutazione del Rischio

Dare un peso al rischio è necessario anche per *stabilire un ordine di priorità* di intervento. L'obiettivo è sempre ridurre i rischi a livelli accettabili applicando mitigazioni. Tre aspetti sono stati considerati fondamentali per la valutazione del rischio:

- Il *danno* massimo (e medio) in caso di manifestazione del rischio e le categorie di asset interessate. Questo parametro è importante perché permette di valutare le conseguenze peggiori che potrebbero verificarsi nel caso il rischio si manifestasse. Serve per valutazioni sia finanziarie che di funzionamento delle infrastrutture.
- I *tempi di recupero* della piena funzionalità (in caso di attacco). Questa metrica è anche utile per stimare le conseguenze indirette, es. i mancati guadagni o i problemi al personale che non potrà lavorare.
- La *probabilità* che esso si verifichi. Naturalmente, essendo stime probabilistiche, non tutti i rischi si manifesteranno, ma è importante sapere se sono rischi concreti (probabilità alte) oppure sono fenomeni che potrebbero accadere con probabilità talmente trascurabile che potrebbe essere anche opportuno accettarli.

La valutazione può essere *quantitativa*, se ad ogni rischio viene associato un numero, ad esempio come risultato del calcolo di una formula. Anche se i numeri vengono rappresentati con molta precisione, va ricordato che tali valori devono essere valutati in termini di ordini di grandezza (un punteggio di 5.55 non indica un rischio davvero maggiore di un altro valutato 5.33, mentre è sicuramente più basso di un rischio valutato 112.34). La valutazione può anche essere *qualitativa*, se i rischi vengono associati a un numero limitato di categorie (es. 3 valori ordinati tipo low, moderate, high). Esiste anche la valutazione *semi-quantitativa* se i rischi vengono associati a valori numerici indicativi di categorie qualitative; ad es. 1:low, 2:moderate, 3:high.

I numeri ottenuti dall'approccio quantitativo sono spesso ingannevoli, quindi si preferisce mapparli su approcci semi-quantitativi, analoghi a quelli qualitativi, ma con il vantaggio di poter lavorare con i numeri, rendendo possibile definire delle formule.

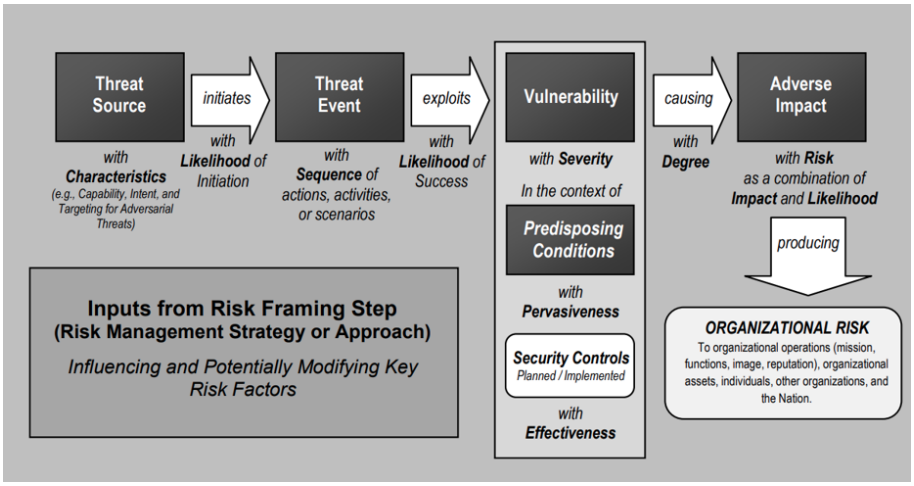


Figura 2.4: Esempio di modello di rischio (Fonte: NIST RMF)

In passato, si è pensato di usare anche l'approccio della *monetization*. Questo approccio proponeva di associare un costo ai rischi, invece di un punteggio numerico. Di solito, questo punteggio indicava il valore medio (e spesso anche quello massimo) associato al rischio (che quindi considera anche la probabilità di accadimento). Tale approccio è stato proposto per rendere la valutazione immediatamente comprensibile (o spaventare) chi, a livello di business, doveva decidere se finanziare spese per la mitigazione dei rischi informativi.

Per calcolare il valore finale associato ai rischi, lo *score*, sono stati proposti tre approcci principali:

- *Threat-Oriented* (cfr. Figura 2.4): Si parte dalle minacce, che vengono caratterizzate ad esempio da obiettivi, competenze, budget. Dalla caratterizzazione delle minacce si può capire che operazioni sarebbero in grado di svolgere (*threat event*) e con quale probabilità. Dagli eventi si può capire quali vulnerabilità potrebbero sfruttare anche in base alle debolezze (*predisposing conditions*). A partire da questi dati si può stimare l'impatto ed il rischio tecnico (tipicamente come impatto $\times$ probabilità che si verifichi), che deve successivamente essere trasformato in un rischi organizzativo globale. In questo modo, quello proposto dal NIST nel RMF, è possibile valutare l'impatto delle mitigazioni (*security control*), sia quelli già usati sia quelli che si deve valutare di applicare per portare il rischio a livelli tollerabili.
- *Asset/Impact-Oriented*: facendo riferimento alla Figura 2.3, si parte dagli asset critici (*mission/business critical*) e si valutano le conseguenze della loro compromissione. Successivamente si identificano le minacce che potrebbero essere interessate a comprometterli, le loro capacità, budget, in modo da poter stimare le probabilità che riescano a sfruttare. Sulla base di queste informazioni si calcolano i rischi e si valutano le mitigazioni. In sostanza, si parte da punti diversi, ma è equivalente a quello threat-oriented.
- *Vulnerability-Oriented*: metodo un po' naif e semplificato, parte dall'identificazione delle vulnerabilità già identificate (es. tramite *Vulnerability Assessment*) e, usando punteggi standard come EPSS e CVSS, valuta l'*exploitability* per stimare costi, probabilità e conseguenze. Chi adotta questo approccio non sempre poi si impegna a identificare le parti interessate a volerle sfruttare, capacità o motivazioni. Per questo motivo, questo metodo è meno rigoroso e più approssimativo, ma veloce e meno costoso.

Strumenti come il *MITRE ATT&CK*[6] forniscono una *knowledge base* delle principali tecniche di attacco per costruire *threat model* affidabili e completi: è una grande tassonomia di attacchi (divisi in 14 macrocategorie) con migliaia di esempi, con riferimenti su come mitigare i rischi. Analogamente,

[6] MITRE, 2013

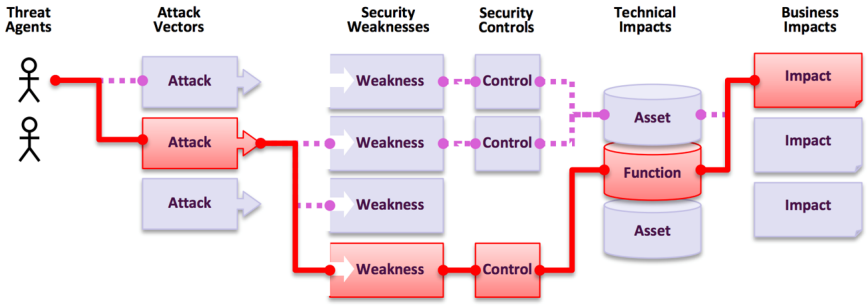


Figura 2.5: Esempio di modello di rischio (Fonte: OWASP)

c'è il *MITRE D3FEND*[78] che costituisce una base di conoscenza per tutte le contromisure attraverso dei grafi, a partire da quasi un migliaio di *artefatti* (asset, oggetti o tipologie), che si collegano agli attacchi, piattaforme ed eventuali contromisure. Questi modelli sono estremamente utili per aiutare a svolgere l'analisi dei rischi e stimare il livello di copertura (e quindi affidabilità) delle fasi di assessment.

L'ente *OWASP (Open Web Application Security Project)* fornisce diverse risorse su come effettuare l'assessment delle proprie web app e delle Top 10 periodiche in cui si specificano i rischi più alti per le applicazioni web (recentemente anche per l'uso degli LLM). OWASP utilizza un approccio semi-qualitativo per la valutazione dei rischi nelle applicazioni web, calcolando il punteggio come likelihood $\times$ impact. L'approccio proposto è di tipo threat-oriented, in cui le fasi seguono un flusso standard, come descritto dal grafico Figura 2.5. Si parte dalle minacce (i *threat agent*) che sono in grado di lanciare determinati attacchi, tra una serie di attack vector noti, che sfruttano debolezze (*security weaknesses*) del sistema. Questi attacchi causano impatti a livello tecnico che si ripercuotono anche a livello di business. I security control, già applicati o da implementare come mitigazione, riducono le conseguenze tecniche.

2.8 Vulnerability Management e Patch Management

Il **Vulnerability Management** è il processo aziendale continuo che mira a minimizzare la *window of exposure*. Partendo dall'assunzione che il software ha sicuramente delle debolezze e spesso delle vulnerabilità note. L'obiettivo di questo processo è **automatizzare la gestione delle vulnerabilità**, in modo da affrontare il problema delle vulnerabilità in maniera organizzata, con l'obiettivo ultimo di minimizzare il rischio informativo, lasciando il sistema esposto alle vulnerabilità il minor tempo possibile e, in caso di vulnerabilità multiple, eliminare prima quelle che potrebbero creare i danni maggiori.

L'approccio proposto dal NIST, nel documento IR 8011 Vol. 4, propone di definire un *desired state*, cioè uno stato di esposizione ai rischi dati dalle vulnerabilità caratterizzato da metriche ben precise, ad esempio, il massimo numero di vulnerabilità per classe di CVSS che possono essere accettate, la durata massima del processo di patching (da quando una vulnerabilità diventa nota a quando verrà corretta). Questo desired state deve poi essere confrontato con *actual state*, mantenuto sempre aggiornato mediante strumenti di vulnerability assessment. L'uso del **Information Security Continuous Monitoring (ISCM)** è essenziale per accorgersi subito di nuove vulnerabilità.

Il **Patch Management** è il processo specifico di identificazione, acquisizione, installazione e verifica delle patch. È un compito di sicurezza fondamentale per mitigare i rischi. Anche in questo caso, un documento di riferimento è il NIST SP 800-40, "Guide to Enterprise Patch Management".

[7] MITRE, 2013
[8] MITRE, 2013

2.8.1 Ruoli e Strategie

Il NIST sottolinea che ogni task, specie quelle essenziali per la sicurezza, richieda che ci sia personale dedicato e che le attività da svolgere siano ben definite. A tal fine individua dei ruoli specifici per gestire questo processo:

- **Software Flaw Manager (SFM)**: questo ruolo è stato proposto per affrontare, a livello di sviluppatori e venditori di software, tutte le problematiche relative a bug e problemi di sicurezza. È tipicamente ricoperto dal personale che ha maturato molta esperienza nello sviluppo e gestione di software e ha la responsabilità di coordinare l'intero ciclo di vita delle vulnerabilità dal punto di vista dello sviluppo. Le sue attività principali includono:
  - Coordinare i processi di *discovery* delle debolezze, sia attraverso audit interni del codice che tramite programmi di *bug bounty* o collaborazioni con ricercatori di sicurezza esterni.
  - Gestire la ricezione e la validazione delle segnalazioni di vulnerabilità provenienti da fonti esterne (es. tramite programmi di *responsible disclosure*).
  - Supervisionare lo sviluppo delle patch correttive, coordinando i team di sviluppo per garantire che le correzioni siano efficaci e non introducano nuove problematiche.
  - Stabilire le priorità di sviluppo delle patch in base alla gravità delle vulnerabilità e all'impatto potenziale sugli utenti.
  - Comunicare tempestivamente con i clienti e le organizzazioni di sicurezza riguardo alle vulnerabilità scoperte e alle patch disponibili.

  La presenza di questo ruolo è fondamentale per garantire che il processo di gestione delle vulnerabilità sia strutturato e non lasciato alla casualità, evitando ritardi pericolosi nel rilascio delle correzioni che hanno di solito impatti devastanti sulla reputazione degli sviluppatori.
- **Patch Manager (PM)**: questo ruolo è stato proposto per le organizzazioni che utilizzano del software (cioè tutte) e ha lo scopo di organizzare tutte le responsabilità relative alla gestione e all'applicazione delle patch nell'infrastruttura aziendale. Le sue attività principali comprendono:
  - Monitorare costantemente i bollettini di sicurezza dei fornitori e le fonti di intelligence sulle minacce per identificare tempestivamente le nuove patch disponibili.
  - Valutare quali patch siano applicabili all'ambiente aziendale, considerando i sistemi in uso e le loro configurazioni specifiche.
  - Stabilire le priorità di applicazione delle patch basandosi su criteri come CVSS, EPSS, criticità degli asset affetti e potenziale impatto sul business.
  - Coordinare il testing delle patch in ambienti di pre-produzione per verificarne la compatibilità e l'assenza di effetti collaterali indesiderati.
  - Pianificare e orchestrare il rollout delle patch nei vari ambienti produttivi, minimizzando le interruzioni di servizio.
  - Documentare tutte le attività di patching e mantenere un inventario aggiornato dello stato di patching di tutti i sistemi aziendali.
  - Gestire le eccezioni, ovvero i casi in cui una patch non può essere applicata immediatamente e devono essere implementate contromisure compensative.

  Questo ruolo è essenziale perché l'applicazione indiscriminata e non coordinata delle patch può causare interruzioni di servizio, incompatibilità e malfunzionamenti. Al contempo, ritardi nell'applicazione espongono l'organizzazione a rischi concreti di compromissione.

La separazione netta di questi ruoli riflette la diversità di prospettive tra sviluppatori e utenti. Il Software Flaw Manager si concentra sulla creazione di correzioni sicure ed efficaci, mentre il Patch Manager si focalizza sull'integrazione sicura di tali correzioni nell'ecosistema IT aziendale. Entrambi i ruoli richiedono competenze tecniche approfondite, ma anche capacità di coordinamento, comunicazione e gestione del rischio.

Le risorse limitate e la necessità di evitare interruzioni del servizio impongono la **prioritizzazione** e il **testing** delle patch. Intuitivamente, la priorità di correzione aumenta in funzione della gravità (CVSS/EPSS) e dell'importanza/valore dell'asset.

Il NIST ha proposto delle strategie di alto livello per razionalizzare il modo in cui il patch management viene affrontato:

- **Subsets**: consiglia di avere un elenco preciso delle categorie di macchine da "patchare". All'interno di ognuno di questi gruppi, conviene scegliere un piccolo sottoinsieme di macchine a cui iniziare ad applicare una patch, prima di procedere con l'intero insieme di macchine nella stessa categoria. L'obiettivo è testare la compatibilità e l'affidabilità della patch su pochi elementi ed evitare che eventuali problematiche causate dalla patch (malfunzionamenti, incompatibilità, regressioni) si propaghino sull'intera infrastruttura. Questo approccio incrementale permette di identificare tempestivamente eventuali effetti collaterali dannosi e di interrompere il rollout prima che l'intera categoria di sistemi venga compromessa, limitando il raggio d'azione di patch difettose.
- **Standard first**: questa strategia raccomanda di applicare le patch prioritariamente ai sistemi con configurazioni standard e ben documentate, che rappresentano tipicamente la maggior parte dell'infrastruttura aziendale. I vantaggi di questo approccio sono molteplici: i sistemi standard sono più facili da testare poiché le loro configurazioni sono note e riproducibili in ambienti di test; le patch per questi sistemi hanno maggiore probabilità di essere state testate dal fornitore in scenari tipici; eventuali problemi vengono rilevati più rapidamente grazie alla maggiore omogeneità dei sistemi; e soprattutto si riesce a coprire una larga porzione dell'infrastruttura in tempi relativamente brevi, riducendo significativamente l'esposizione complessiva al rischio. Questa strategia massimizza l'efficienza del processo, permettendo di proteggere rapidamente il maggior numero di asset con il minor sforzo.
- **Difficult ones later**: i sistemi complessi, *legacy*, o con configurazioni particolari (ad esempio sistemi SCADA, apparati medicali, dispositivi embedded, o sistemi con dipendenze software specifiche) vanno affrontati nelle fasi finali del processo di patching. Questi sistemi richiedono un'attenzione particolare perché potrebbero non supportare le patch più recenti senza modifiche sostanziali, potrebbero avere dipendenze critiche da versioni specifiche di librerie o componenti, i test di compatibilità richiedono ambienti specializzati e competenze tecniche approfondite, e il rischio di interruzione del servizio è spesso più elevato. Per questi sistemi è necessario pianificare collaudi manuali dettagliati, preparare piani di rollback specifici, talvolta aggirando i problemi o sviluppando contromisure compensative quando l'applicazione diretta della patch non è immediatamente possibile. Affrontando questi casi per ultimi, si garantisce che il personale tecnico possa dedicare il tempo e le risorse necessarie senza compromettere la protezione della maggioranza dei sistemi.
- **Cover all**: questa strategia finale sottolinea l'importanza di garantire una copertura completa ed esaustiva di tutti i sistemi aziendali, senza eccezioni. È fondamentale non trascurare categorie di dispositivi che potrebbero sembrare marginali ma che rappresentano potenziali vettori di attacco: macchine virtuali e virtualizzatori (che potrebbero essere dimenticate o considerate temporanee), dispositivi mobili (smartphone, tablet aziendali), firmware di dispositivi, in particolare di quelli di rete (router, switch, firewall), IoT e dispositivi embedded, sistemi di backup e ambienti di test o sviluppo. Ogni sistema non aggiornato rappresenta un potenziale punto di ingresso per gli attaccanti, che potrebbero sfruttarlo per penetrare nella rete aziendale e muoversi lateralmente verso asset più critici. È quindi necessario mantenere un inventario completo e aggiornato di tutti i dispositivi, implementare processi di discovery automatici per identificare nuovi asset e stabilire policy che obblighino l'applicazione delle patch anche ai sistemi considerati non critici.

2.8.2 Affidabilità delle Patch e Problematiche

Nonostante le patch siano fondamentali per la sicurezza, non sono esenti da rischi. Le patch possono introdurre nuove problematiche che impattano negativamente sui sistemi:

- **Incompatibilità software**: una patch può causare malfunzionamenti di applicazioni che dipendono da specifiche versioni di librerie o componenti modificati dalla correzione. Questo è particolarmente critico per software legacy o applicazioni personalizzate non più supportate.
- **Regressioni funzionali**: talvolta le patch introducono bug che compromettono funzionalità precedentemente operative, creando problemi operativi anche più gravi della vulnerabilità che dovevano correggere.
- **Degradazione delle prestazioni**: alcune patch, specialmente quelle che correggono vulnerabilità architetturali (come nel caso di Meltdown e Spectre), possono causare riduzioni significative delle performance del sistema.
- **Problemi di stabilità**: patch non sufficientemente testate possono causare crash di sistema, instabilità o comportamenti imprevedibili, richiedendo rollback urgenti.
- **Conflitti con altre patch**: in ambienti complessi, l'applicazione di multiple patch può generare conflitti o dipendenze circolari che compromettono l'intero sistema.

Come esempio più eclatante in tempi recenti di patch difettosa c'è sicuramente l'incidente CrowdStrike del luglio 2024, dove un aggiornamento difettoso del software di sicurezza CrowdStrike Falcon ha causato BSOD (Blue Screen of Death) massivi su milioni di sistemi Windows in tutto il mondo, bloccando aeroporti, ospedali e servizi critici[9]. Altri esempi possono includere patch di sicurezza per processori che hanno ridotto drasticamente le prestazioni (es. Meltdown), e correzioni che hanno reso inutilizzabili intere funzionalità, anche fondamentali, fino al rilascio di patch correttive delle patch stesse (es. Rete cellulare e TouchID in iOS 8.0.1[10]). Questi incidenti sottolineano l'importanza del testing accurato prima del deployment in produzione.

2.8.3 Evoluzione del Modello di Rilascio delle Patch

Storicamente, i fornitori di software rilasciavano patch **singolarmente**, non appena una vulnerabilità veniva scoperta e corretta. Questo approccio, apparentemente vantaggioso per minimizzare la *window of exposure*, creava numerosi problemi a livello aziendale:

- **Sovraccarico operativo**: gli amministratori di sistema dovevano costantemente monitorare, testare e applicare patch, dedicando la maggior parte del loro tempo al patch management invece che ad altre attività strategiche.
- **Interruzioni continue del servizio**: ogni patch richiedeva spesso un riavvio dei sistemi, causando interruzioni frequenti e impattando negativamente sulla disponibilità dei servizi.
- **Costi di testing elevati**: ogni singola patch doveva essere testata indipendentemente, moltiplicando i costi e i tempi necessari per la validazione.
- **Complessità di gestione**: tracciare quali patch erano state applicate a quali sistemi diventava un incubo amministrativo, con alto rischio di errori e dimenticanze.
- **Rischi di incompatibilità**: l'applicazione sequenziale di patch poteva creare situazioni in cui alcune patch dipendevano da altre non ancora installate o entravano in conflitto tra loro.

Per affrontare questi problemi, i fornitori hanno progressivamente adottato il modello delle **aggregated patch** (o *patch bundle*), rilasciando correzioni cumulative a cadenze regolari e prevedibili. Ad esempio, Microsoft ha introdotto il *Patch Tuesday*, rilasciando aggiornamenti di sicurezza il secondo martedì di ogni mese. Questo approccio presenta vantaggi significativi:

- **Prevedibilità**: le organizzazioni possono pianificare finestre di manutenzione programmate, riducendo l'impatto sul business.

[9] Wikipedia, 2025
[10] HWupgrade, 2014

- **Efficienza operativa**: un singolo ciclo di test e deployment sostituisce decine di interventi separati.
- **Testing integrato**: i fornitori possono testare l'interazione tra le varie correzioni prima del rilascio.
- **Riduzione dei riavvii**: invece di riavviare i sistemi più volte al mese, un singolo riavvio programmato può applicare tutte le patch.

Tuttavia, questo approccio presenta anche svantaggi: la *window of exposure* per le singole vulnerabilità può estendersi fino al prossimo rilascio programmato, e patch critiche potrebbero dover attendere settimane prima di essere distribuite (sebbene i fornitori mantengano la possibilità di rilasci immediati per vulnerabilità critiche attivamente sfruttate).

2.8.4 Enterprise Patch Management Tools

Gli **Enterprise Patch Management (EPM) Tools** sono soluzioni software centralizzate che automatizzano e orchestrano l'intero processo di patch management su larga scala. Strumenti come *Microsoft WSUS/SCCM*, *Red Hat Satellite*, *VMware vCenter Update Manager*, *ManageEngine Patch Manager*, e soluzioni cloud-based come *NinjaOne* o *Automox* offrono funzionalità avanzate:

- **Discovery automatica**: identificazione di tutti i sistemi nella rete e del loro stato di patching.
- **Distribuzione centralizzata**: deployment automatico delle patch su migliaia di endpoint da un'unica console.
- **Scheduling flessibile**: pianificazione di finestre di manutenzione diverse per gruppi di sistemi.
- **Reporting e compliance**: generazione di report dettagliati sullo stato del patching per audit e conformità normativa.
- **Rollback automatico**: capacità di annullare patch problematiche e ripristinare lo stato precedente.
- **Gestione multi-piattaforma**: supporto per Windows, Linux, macOS, e talvolta applicazioni di terze parti.

Nonostante i benefici evidenti, l'adozione di EPM introduce rischi significativi che devono essere attentamente gestiti:

- **Single point of failure**: se il server EPM viene compromesso, un attaccante ottiene un vettore perfetto per distribuire malware su tutta l'infrastruttura aziendale, trasformando uno strumento di sicurezza in un'arma devastante. Casi storici hanno dimostrato che attaccanti sofisticati mirano specificamente a questi sistemi.
- **Privileged access**: i tool EPM richiedono credenziali privilegiate per operare su tutti i sistemi gestiti. Se queste credenziali vengono rubate, l'attaccante ottiene accesso amministrativo all'intera rete.
- **Distribuzione di patch malevole**: un attaccante che compromette il repository delle patch o esegue attacchi man-in-the-middle può distribuire patch falsificate o backdoor mascherate da aggiornamenti legittimi.
- **Errori di configurazione**: configurazioni errate possono causare il deployment simultaneo di patch incompatibili su sistemi critici, provocando blocchi massicci.
- **Dipendenza dalla connettività**: sistemi remoti o isolati potrebbero non ricevere aggiornamenti se la connessione al server EPM è compromessa.
- **Complessità e manutenzione**: gli stessi EPM richiedono aggiornamenti, manutenzione e patching, creando un problema metaricorsivo del tipo "chi patcha il patcher?".

Per mitigare questi rischi, il NIST raccomanda diverse best practice, che non si discostano da quello che dovrebbe essere fatto con tutti gli altri sistemi critici o sensibili:

- **Hardening del server EPM**: isolamento in segmenti di rete protetti, applicazione di tutti i controlli di sicurezza, monitoraggio intensivo delle attività.

- **Autenticazione multi-fattore**: richiesta per tutti gli accessi amministrativi agli EPM.
- **Verifica dell'integrità**: uso di firme digitali e checksum per validare l'autenticità delle patch prima della distribuzione.
- **Segregazione dei privilegi**: limitazione dei permessi EPM al minimo indispensabile attraverso principi di least privilege.
- **Logging e auditing**: registrazione dettagliata di tutte le operazioni per investigazioni e forensics.
- **Deployment graduale**: uso di gruppi pilota e rollout progressivi anche con automazione, per identificare problemi prima che impattino l'intera infrastruttura.

Nonostante i rischi, l'analisi costi-benefici condotta dal NIST conclude che, se implementati correttamente con adeguate misure di sicurezza, gli EPM rappresentano una necessità per organizzazioni di medie e grandi dimensioni. I benefici in termini di riduzione della *window of exposure*, efficienza operativa, conformità normativa e riduzione dell'errore umano superano significativamente i rischi, purché questi ultimi siano attivamente mitigati attraverso le best practice di sicurezza.

3. Crittografia parte 1

Questo capitolo introduce i fondamenti della crittografia, concentrandosi sugli algoritmi simmetrici e sui meccanismi di protezione dell'integrità dei dati. Verranno analizzati i principi teorici, le proprietà di sicurezza richieste e le tecniche impiegate per garantire sia la riservatezza sia l'autenticazione delle informazioni in ambienti potenzialmente ostili.

3.1 Terminologia

La crittografia è considerata, a tutti gli effetti, una branca della matematica. Per comprendere come funziona la crittografia è necessario introdurre i concetti di casualità e pseudo-casualità. Un numero si definisce *casuale* se è stato generato da un processo intrinsecamente imprevedibile (*random*), come il rumore termico, i moti browniani o il decadimento. Essendo qualunque sistema a microprocessore intrinsecamente deterministico, perché esegue sequenze di istruzioni definite da un programma, la generazione di numeri casuali mediante un programma è impossibile. Infatti, i numeri casuali possono essere generati solo con particolari dispositivi, capaci di generarli misurando fenomeni fisici[1]. Per questo motivo, è stato necessario introdurre un altro concetto, la *pseudo-casualità*. Una sequenza di numeri è detta *pseudo-casuale* se è generata da un algoritmo deterministico (un pseudo-random number generator, o PRNG), ma, testandola con metodi statistici e probabilistici, risulta indistinguibile da un numero realmente casuale. La casualità si misura attraverso l'*entropia*, che quantifica l'incertezza o l'imprevedibilità di una sequenza di bit. Un'entropia elevata indica una maggiore resistenza agli attacchi di predizione. Se un attaccante riesce a prevedere i valori futuri di una sequenza, può compromettere la sicurezza del sistema crittografico.

qualche parola in più e/o qualche link sull'entropia di Shannon?

La **casualità** riveste un ruolo essenziale nella crittografia: chiavi, nonce e vettori di inizializzazione devono essere imprevedibili per evitare che gli attaccanti possano indovinarli e riuscire a effettuare attacchi.

cercare/confrontare def di generatore di sequenze pseudo-random

I generatori pseudo-casuali producono in output una sequenza. Questa sequenza viene generata mediante particolari algoritmi a partire da un *seed* (seme), che è il valore iniziale che determina l'intera sequenza generata. Poiché l'algoritmo è deterministico, lo stesso seed produce sempre la stessa sequenza: per questo, un attaccante che riuscirà a scoprire il seed riuscirà anche a prevedere l'intera sequenza di bit generata. Per questo motivo, il seed deve essere sufficientemente grande (da non essere indovinabile con pochi tentativi) e scelto con un'entropia adeguata, tipicamente utilizzando sorgenti di casualità hardware o eventi imprevedibili del sistema.

servirebbero degli esempi di casualità HW e di eventi: HW: generatori, eventi di sistema: spostamento mouse, processi e occupazione CPU, temperatura processore, tasti premuti,

[1] `https://www.idquantique.com/random-number-generation/products/quantis-qrng-pcie/`

La crittografia si può definire come un processo di trasformazione. Nel corso della storia, la crittografia è nata e si è evoluta proprio per soddisfare l'esigenza di garantire la proprietà astratta della confidenzialità: sin dalle prime tecniche di cifratura utilizzate in ambito militare e diplomatico, l'obiettivo è stato impedire che informazioni sensibili fossero lette da soggetti non autorizzati. Questa relazione diretta ha fatto della crittografia uno strumento fondamentale per trasformare la confidenzialità da concetto teorico in una caratteristica concretamente implementabile nei sistemi di comunicazione. Con il progredire delle tecnologie e l'aumento della complessità delle minacce, la crittografia ha assunto un ruolo ancora più centrale, codificando in modo matematico la confidenzialità.

legame tra crittografia e confidenzialità

Gli elementi fondamentali del processo di trasformazione sono:

- *testo in chiaro (plaintext, P)*: il messaggio originale, in formato leggibile;
- *testo cifrato (ciphertext, C)*: è il messaggio trasformato, in un formato "illeggibile", cioè che preserva la proprietà della confidenzialità;
- *algoritmo di cifratura/decifratura*: è il procedimento matematico (o logico) che trasforma il testo in chiaro in testo cifrato (cifratura) e, in modo inverso, trasforma testo cifrato in testo in chiaro (decifratura);
- *chiave (key, K)*: è l'informazione segreta necessaria per pilotare il processo crittografico. Solo chi possiede la chiave corretta (quella usata per la cifratura) è in grado di decifrare correttamente il messaggio.

3.2 Crittografia simmetrica

La crittografia simmetrica utilizza un'unica chiave per entrambe le operazioni di cifratura e di decifratura. Naturalmente, questa chiave deve essere condivisa tra le entità (es. utenti, processi) coinvolte. A livello di notazione, nei libri di testo e negli articoli scientifici si possono trovare le seguenti notazioni

$$\text{per la cifratura (encryption)} C = enc(K,P) \quad \textit{oppure} \quad C = E_K(P) \quad \textit{oppure} \quad C = \{P\}_K \tag{3.1}$$

$$\text{per la decifratura (decryption)} \; P = enc^{-1}(K,C) = dec(K,C) \quad \textit{oppure} \quad P = D_K(C) \tag{3.2}$$

Si sottolinea che le chiavi simmetriche usate nei sistemi di calcolatori sono sequenze di bit (es. lunghe da 128 a 256 bit), quindi possono essere a tutti gli effetti considerate numeri (es. considerandoli rappresentati in binario puro).

Sebbene le operazioni crittografiche possano essere computazionalmente intensive, la crittografia simmetrica è significativamente più efficiente rispetto ad altri tipi di crittografia (es. quella asimmetrica). Per questo motivo, è particolarmente adatta alla protezione di grandi quantità di dati. Il principale limite, tuttavia, risiede nella distribuzione della chiave segreta, poiché deve essere condivisa tra le parti in modo sicuro per evitare che venga intercettata o compromessa.

Storicamente, il primo esempio di cifrario simmetrico risale all'epoca romana con il Cifrario di Cesare, un semplice metodo di sostituzione in cui ogni lettera dell'alfabeto veniva spostata di un numero fisso di posizioni (la versione di Cesare usava 3 come spostamento). Oggi, gli algoritmi di crittografia simmetrica si basano su principi matematici molto più complessi. Tra i più noti e diffusi si trovano **AES** (con chiavi da 128, 192 e 256 bit), l'algoritmo scelto dal governo degli Stati Uniti e considerato il *de facto standard* mondiale, il DES (ormai obsoleto), Salsa20 e **ChaCha20**.

Gli algoritmi di crittografia simmetrica possono essere classificati in base a diversi criteri tecnici:

- *nome dell'algoritmo e tipo di trasformazione*: un algoritmo definisce le trasformazioni che verranno applicate al plaintext;

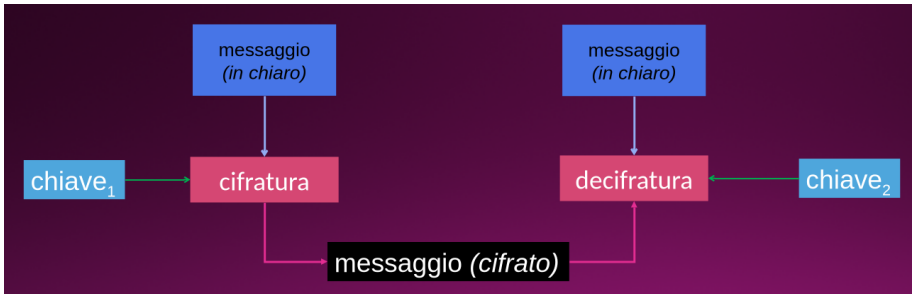


Figura 3.1: Processo di trasformazione.

- *lunghezza della chiave*: varia generalmente da 40 bit fino a 256 bit o più; una chiave più lunga implica una maggiore resistenza agli attacchi a forza bruta;
- *tipologia operativa*: si distinguono in algoritmi a blocchi, che elaborano gruppi di bit di dimensione fissa, e algoritmi stream, che operano su singoli bit o byte;
- *modalità di utilizzo*: lo stesso algoritmo può essere usato in modi diversi, in funzione di alcuni schemi predefiniti. Alcuni sono più efficienti per la cifratura di piccole quantità di dati, altri per grandi volumi o flussi continui, e altri ancora permettono di ottenere più proprietà di sicurezza con una sola operazione di cifratura (es. numerazione dei blocchi per evitare attacchi di tipo replay);
- *modalità di computazione*: possono essere seriali, se elaborano i dati in sequenza, oppure paralleli, se supportano l'elaborazione simultanea di più blocchi;
- *servizi di sicurezza aggiuntivi*: alcuni algoritmi prevedono funzionalità accessorie, come la numerazione dei blocchi, il controllo dell'integrità o la protezione contro il riutilizzo dei dati cifrati.

3.2.1 Principio di Kerckhoffs

Fin dalle origini della crittografia, è stato applicato il principio della *security through obscurity*, ovvero la convinzione che la sicurezza di un sistema possa essere garantita mantenendo segreti i suoi meccanismi interni. Questo approccio era già in uso nel cifrario di Cesare. Anche in tempi più moderni, la *security through obscurity* ha caratterizzato la progettazione di apparati cifranti. Ad esempio, le forze naziste durante la Seconda Guerra Mondiale avevano progettato la macchina Enigma, che avrebbe dovuto proteggere la segretezza delle comunicazioni militari tedesche, nascondendo dettagli dell'algoritmo nell'implementazione della macchina stessa. Dopo essere riusciti ad ottenere una copia di Enigma, i crittografi britannici sono riusciti a capire come decifrare le comunicazioni tedesche. Al di là dell'impatto positivo di tale errore nella guerra contro i nazisti, è chiaro che questo approccio si è rivelato inefficace e pericoloso. Infatti, la possibilità di mantenere segreti i dettagli tecnici di un sistema è limitata, e una volta che tali dettagli vengono scoperti, la sicurezza dell'intero sistema viene compromessa. Già nel 1851, il matematico Hobbs definì questo approccio sbagliato. Il concetto fu ribadito un secolo dopo dal crittografo Shannon, che affermò che, nel valutare un sistema di sicurezza, bisogna assumere che l'attaccante conosca il sistema di cifratura. Nonostante ciò, l'approccio rimane ancora oggi diffuso e, in molti casi, è all'origine di vulnerabilità e attacchi informatici.

Nel 1883, Kerckhoffs affermò il principio che un sistema *non deve essere segreto, deve poter cadere nelle mani del nemico senza inconvenienti*. Dall'applicazione di questo principio deriva che **la forza della crittografia deve risiedere esclusivamente nella gestione corretta delle chiavi crittografiche**: se queste sono tenute segrete, gestite da sistemi fidati e sono di lunghezza adeguata, allora non ha alcuna importanza che gli algoritmi di cifratura e decifratura siano tenuti segreti, anzi è bene che siano pubblici affinché siano studiati ampiamente e siano rilevate eventuali debolezze.

3.2.2 Forza di un algoritmo simmetrico

In accordo con il principio di Kerckhoffs, si può affermare che, se un algoritmo di crittografia è ben progettato e le chiavi, lunghe n bit, sono scelte correttamente e mantenute segrete, l'unico attacco possibile è l'attacco esaustivo (**forza bruta**), che richiede 2^n tentativi. L'attacco a forza bruta è possibile perché, dato che una chiave è un numero rappresentato su un numero fisso di bit, sappiamo che si troverà nell'intervallo $[0, 2^n - 1]$. Quindi, sappiamo che ogni chiave potrà essere indovinata provando tutti i numeri nell'intervallo. Per questo motivo, un algoritmo ben progettato sarà sicuro se, con le risorse computazionali a sua disposizione, un attaccante non riuscirà ad ottenere, se non con probabilità negligibile, la chiave usata per cifrare i dati entro il tempo in cui i dati sono ancora validi.

Per stimare il tempo che servirebbe ad un attaccante per riuscire a tentare tutte le chiavi, bisogna partire da quanto tempo ci mette ad effettuare un'operazione di decifratura con una singola chiave. Inoltre, per ogni tentativo bisogna considerare sia il tempo di calcolo necessario per eseguire l'operazione di decifratura, sia il tempo necessario per verificare se il risultato ottenuto è corretto (ad esempio, se il messaggio decifrato contiene caratteri stampabili o se ci aspettiamo un'immagine, che sia in un formato noto, tipo JPG/PNG/etc.).

Infatti ci sono informazioni che diventano subito irrilevanti (es. la posizione dove verranno lanciati dei missili) e non c'è bisogno di proteggerne la confidenzialità a lungo, e altre possono dover essere mantenute segrete per decenni (es. segreti di stato). Un altro aspetto cruciale da considerare è il *tempo di validità* dell'informazione protetta. La robustezza crittografica richiesta dipende fortemente dal tempo durante il quale i dati devono rimanere segreti, ossia il tempo che lasciamo a disposizione degli attaccanti. Un'informazione militare tattica, ad esempio, potrebbe perdere valore dopo poche ore o giorni, rendendo accettabile una chiave che resista agli attacchi per un periodo limitato. Al contrario, segreti di Stato, progetti strategici o dati personali sensibili possono richiedere protezione per anni o decenni: in questi casi è necessario utilizzare chiavi più lunghe e algoritmi più robusti, in grado di resistere non solo alle capacità computazionali attuali, ma anche a quelle future prevedibili. Infatti, la sicurezza non è immutabile: i progressi tecnologici aumentano la velocità con cui si possono effettuare tentativi nello stesso intervallo di tempo. La ricerca crittanalitica, basata su analisi matematiche e statistiche, può talvolta ridurre lo spazio delle chiavi effettivamente da esplorare. Per queste ragioni è necessario sostituire le chiavi periodicamente, anche se di lunghezza opportuna, e devono essere scelte sempre casualmente.

Caso di studio: le sfide DES

Il DES (Data Encryption Standard), introdotto negli anni '70, utilizza chiavi a 56 bit, offrendo quindi 2^{56} chiavi possibili (circa 72 quadrilioni). Questa lunghezza, adeguata all'epoca, è diventata insufficiente con l'aumento della potenza computazionale.

Per dimostrare questa vulnerabilità, la RSA Security organizzò una serie di sfide pubbliche:

- **DES Challenge I (1997)**: risolta in 96 giorni mediante ricerca distribuita su Internet;
- **DES Challenge II (1998)**: risolta in 41 giorni con tecniche analoghe;
- **DES Challenge III (1998)**: risolta in soli 56 ore utilizzando hardware specializzato (Deep Crack), un sistema ad hoc in grado di decifrare e testare che l'output fosse un messaggio di testo;
- **DES Challenge IV (1999)**: risolta in meno di 23 ore combinando Deep Crack e calcolo distribuito (199Gkey/s[2] di media).

Questi risultati evidenziarono la necessità di un algoritmo più robusto. Come soluzione temporanea fu adottato il 3DES (Triple DES), che applica tre volte l'algoritmo DES in serie con due o tre chiavi diverse (lunghezza effettiva della chiave: 168 bit, numero di tentativi per individuarla è

[2] Gkey/s: miliardi di chiavi testate al secondo (GigaKeys per second).

solo[3] $\leq 2^1 12$). Il processo prevede cifratura-decifratura-cifratura (EDE: Encrypt-Decrypt-Encrypt), aumentando notevolmente la sicurezza rispetto al DES singolo ().

Nonostante la maggiore robustezza, il 3DES è oggi considerato obsoleto per la sua inefficienza computazionale (circa tre volte più lento del DES), che lo rende inutilizzabile su sistemi con scarsa capacità di calcolo e vulnerabile agli attacchi alla disponibilità. È stato definitivamente sostituito da AES (Advanced Encryption Standard), che offre prestazioni superiori e una maggiore sicurezza. AES è stato scelto tramite una *call pubblica* indetta dal governo degli USA[4].

3.2.3 Algoritmi di tipo stream

Gli algoritmi stream operano un bit o un byte alla volta. Questo li rende idonei all'utilizzo su un flusso continuo di dati, come streaming o comunicazioni in tempo reale. Questi algoritmi fanno ampio uso dell'operatore XOR, la cui tavola di verità, bilanciata tra 0 e 1, lo rende un operatore di confusione ideale e reversibile. L'XOR consente di trasferire la casualità della chiave sul messaggio, garantendo sicurezza, ed è disponibile come operazione di base su tutte le CPU.

l'XOR è idempotente e vale a XOR b = c b XOR c = a e a XOR c = b

L'algoritmo ideale in questa categoria è il *one-time pad*, per cui, per ogni messaggio da cifrare si genera una chiave che è una sequenza di bit random della stessa lunghezza in bit del messaggio da proteggere. Tale chiave viene usata per fare un XOR bit-a-bit con il plaintext. Shannon ha dimostrato che è impossibile ricavare informazioni dal ciphertext senza conoscere la chiave. Nei casi reali, la gestione di chiavi così lunghe risulta impraticabile. Infatti, le chiavi per il principio di Kerckhoffs devono essere mantenute segrete e farlo per chiavi grandi quanto i dati cifrati è nella pratica impossibile (sebbene alcune applicazioni della fisica stiano permettendo il Quantum Key Distribution). Per questo motivo, gli algoritmi stream utilizzano generatori di sequenze pseudo-casuali: la chiave funge da seme (*seed*) e viene usata per generare con questi generatori una sequenza pseudo-casuale lunga quanto il messaggio da cifrare. Successivamente viene calcolato l'XOR bit a bit tra la sequenza generata e il plaintext. Il vantaggio è che viene e, ogni volta che viene riutilizzata, produce la stessa sequenza, quindi chi conosce la chiave potrà usarla per decifrare.

Tra gli algoritmi stream più noti figurano **RC4**, **Salsa20** e **ChaCha20**. RC4, sviluppato negli anni '80, è oggi considerato obsoleto a causa di gravi vulnerabilità crittanalitiche che permettono di recuperare la chiave o parti del testo in chiaro (es. l'attacco di Fluhrer, Mantin and Shamir). Salsa20 e ChaCha20, entrambi sviluppati da Daniel J. Bernstein e molto simili tra loro, rappresentano lo stato dell'arte per i cifrari stream. Entrambi utilizzano chiavi di 256 bit. Per ogni operazione di cifratura deve anche essere generato un nonce[5] a 64 bit e counter a 64 bit, e generano blocchi di keystream da 512 bit. Salsa20, sviluppato nel 2005, usa soltanto operazioni semplici, disponibili nativamente nelle CPU (ARX: Add, Rotate, XOR) che garantiscono un'elevata velocità di calcolo. Il suffisso 20 deriva dal numero di passaggi (*round*) di "mescolamento" applicati all'input durante la generazione della sequenza pseudo-casuale. Esistono anche varianti che ripetono meno passaggi, come Salsa20/12 e Salsa20/8, che applicano rispettivamente 12 e 8 passaggi: sono più veloci, rendendoli usabili anche su dispositivi IoT, ma offrono margini di sicurezza inferiori. È infatti noto che, se vengono usati fino a sette passaggi, Salsa è attaccabile; non ci sono prove di attacchi contro

[3] la sicurezza effettiva, pari al numero di tentativi che servirebbe ad un attaccante è al massimo di 112 bit a causa di vulnerabilità crittanalitiche, note come *meet-in-the-middle attacks*. Questo perché si sfrutta il fatto che il risultato intermedio dopo la prima cifratura può essere confrontato con quello della decifratura del terzo step, così da ottenere equazioni che saltino il secondo step.

[4] `https://nvlpubs.nist.gov/nistpubs/jres/106/3/j63nec.pdf`.

[5] Nonce (number used once). Un nonce è un valore casuale o pseudo-casuale, pubblico da utilizzare una sola volta. Esso serve a garantire che, pur usando più volte la stessa chiave (cosa che mi evita di generare e custodire un segreto diverso per ogni cifratura), non venga mai lo stesso keystream (evita gli attacchi di *keystream reuse*). Possiamo immaginare che il nonce sia una sequenza che viene concatenata alla chiave per generare il seed del PRNG (nella pratica è leggermente diverso)

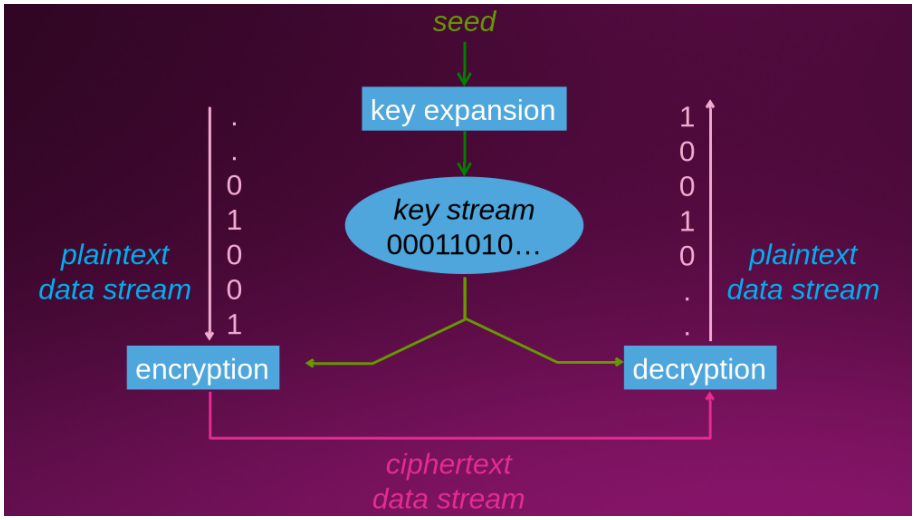


Figura 3.2: Algoritmo stream.

la versione a 8 passaggi, per questo motivo, per sicurezza, si preferisce farne almeno 12 o, meglio ancora, 20. ChaCha20, introdotto nel 2008, rappresenta un'evoluzione di Salsa20: incrementa la diffusione dei bit a ogni iterazione (cioè quanto è efficace la generazione di sequenze pseudo-casuali) e migliora ulteriormente le prestazioni su alcune architetture rispetto al predecessore.

3.2.4 Algoritmi a blocchi

Gli algoritmi a blocchi operano su un blocco di dati di dimensione fissa. La dimensione dei blocchi è 64 bit per gli algoritmi meno recenti (e ormai ritenuti insicuri) oppure di 128 bit per gli algoritmi più moderni (alcuni dei quali ritenuti sicuri). Questi tipi di algoritmi effettuano, sui dati del blocco in input (plaintext), un certo numero di trasformazioni guidate dalla chiave, al termine delle quali si ottiene il blocco in output (ciphertext). Il tipo di trasformazioni che possono essere applicate è definito dall'algoritmo; quelle effettivamente applicate dipendono dalla chiave.

Dato che gli algoritmi a blocchi definiscono con precisione solo ciò che si può fare su un singolo blocco di dati in input, di 64 o 128 bit, rimane il problema di definire come usare questi algoritmi su dati reali, normalmente molto più grandi della dimensione di un blocco. Un altro problema che rimane aperto dalla definizione di algoritmi a blocchi è come fare a cifrare dei dati quando non si riesce a riempire un intero blocco di dati. Quest'ultimo scenario capita con l'ultimo blocco di un file che non abbia lunghezza multipla della dimensione del blocco (es. file da 1034 byte), oppure se ci devono cifrare pochissimi dati (es. se devo cifrare un comando di shell, ad esempio `ls` durante la comunicazione con una macchina remota).

Per poter cifrare un messaggio più lungo di un blocco, oppure simulare un cifrario stream, si usano le *modalità operative di funzionamento*, che definiscono come concatenare i blocchi e quali altri dati serve usare (es. i vettori di inizializzazione (**IV**, Initialization Vector), contatori).

Modalità ECB (Electronic Code Book)

$$\text{formula per il blocco i-esimo :} \quad C_i = enc(K, P_i) \tag{3.3}$$

La modalità ECB consiste nel dividere il plaintext in tanti frammenti di lunghezza pari al blocco dell'algoritmo da usare (blocchi in input) e poi cifrare ciascun blocco in input in chiaro con la stessa chiave. Nella modalità ECB viene cifrato un blocco alla volta indipendentemente da ogni altro blocco. È semplice e parallelizzabile, ma presenta gravi debolezze di sicurezza: non

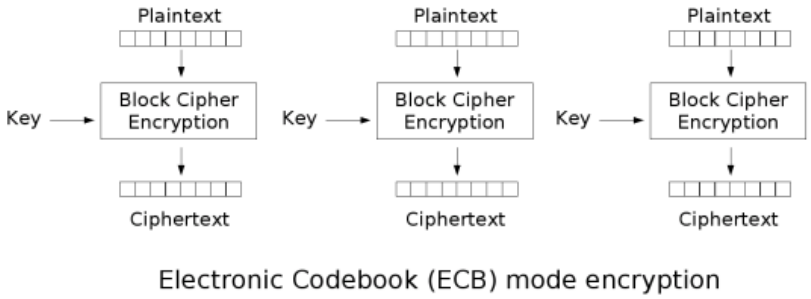


Figura 3.3: Modalità ECB. (Fonte: Wikipedia)

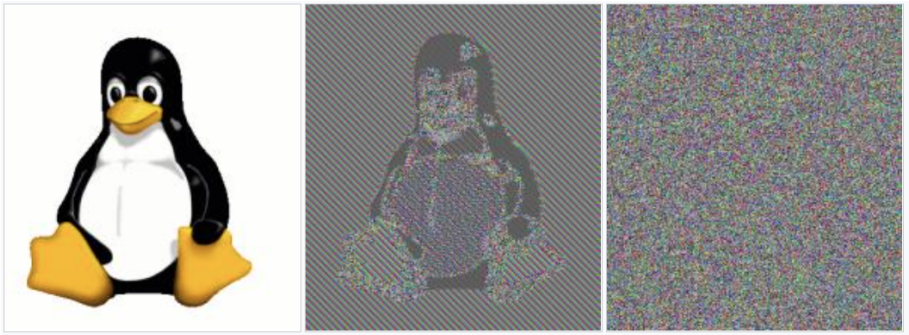


Figura 3.4: Insicurezza della modalità ECB. (Fonte: Wikipedia)

garantisce l'integrità del messaggio[6], perché un attaccante potrebbe scambiare due blocchi e la vittima non se ne accorgerebbe; inoltre è vulnerabile ad attacchi di tipo *known-plaintext*, ovvero se un attaccante conosce anche solo una parte del testo in chiaro e il corrispondente testo cifrato, può costruire un *dizionario* di coppie (blocco in chiaro, blocco cifrato). Successivamente, ogni volta che incontra uno di questi blocchi cifrati, può immediatamente risalire al contenuto del blocco in chiaro corrispondente, senza dover necessariamente recuperare la chiave. Inoltre, poiché blocchi identici generano sempre lo stesso ciphertext, pattern ripetitivi nel testo in chiaro rimangono identificabili nel testo cifrato, rivelando strutture e informazioni statistiche.

ref alla figura? con qualche parola che dica che, sapendo che si tratta di un'immagine, parti di colore uniforme generano blocchi in input tutti uguali e quindi con queste informazioni, non si può risalire al colore, ma si può capire che il colore è uguale e provare a ricostruire una forma

Modalità CBC (Cipher Block Chaining)

$$\text{formula per il blocco i-esimo :} \quad C_i = enc(K, P_i \oplus C_{i-1}) \quad \text{con } C_0 = IV \tag{3.4}$$

La modalità CBC (*Cipher Block Chaining*) consiste nell'eseguire un XOR tra l'IV e il primo blocco di testo in chiaro prima della cifratura. Il blocco di testo cifrato viene utilizzato per eseguire l'XOR sul blocco successivo. Supera i limiti di sicurezza dell'ECB, ma un errore in un blocco (es. consequenza di un problema di trasmissione) può propagarsi ai blocchi successivi.

[6]Cosa non eccessivamente importante perché lo scopo principale dell'uso di algoritmi crittografici è la confidenzialità

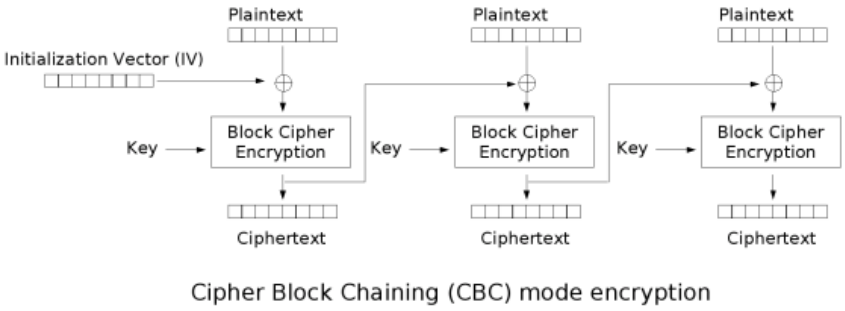


Figura 3.5: Modalità CBC. (Fonte: Wikipedia)

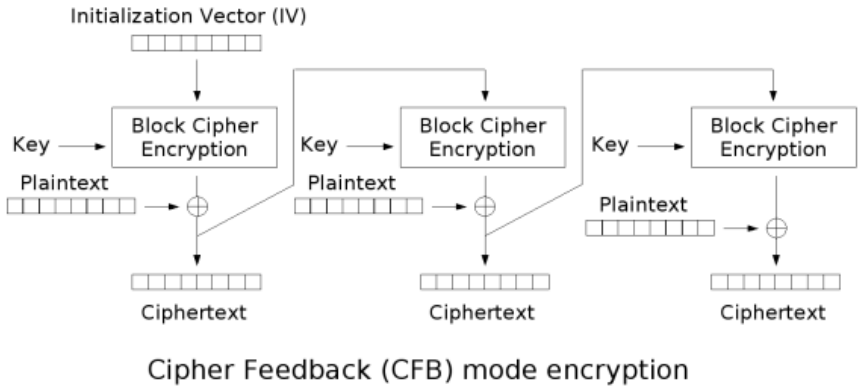


Figura 3.6: Modalità CFB. (Fonte: Wikipedia)

La complessità di calcolo di questa modalità è maggiore rispetto a quella di ECB, poiché ogni blocco dipende dal precedente, rendendo impossibile la parallelizzazione. Pertanto, il tempo di cifratura aumenta linearmente con il numero di blocchi. Tuttavia, offre una sicurezza significativamente superiore, rendendo più difficile per un attaccante analizzare il testo cifrato e risalire al testo in chiaro.

Modalità CFB (Cipher FeedBack)

La modalità CFB trasforma idealmente un cifrario a blocchi in un cifrario stream. Consiste nel cifrare l'IV ed eseguire un XOR con un blocco di testo in chiaro. Il blocco di testo cifrato diventa "feedback" del blocco successivo. Anche questa modalità propaga gli errori ai blocchi successivi e non è parallelizzabile.

Modalità OFB (Output FeedBack)

Anche la modalità OFB trasforma idealmente un cifrario a blocchi in un cifrario stream. A differenza della modalità CFB, è l'IV cifrato a diventare il "feedback" del blocco successivo. Questa modalità impedisce la propagazione degli errori verso i blocchi successivi.

Modalità CTR (Counter)

La modalità CTR è stata introdotta per essere applicata a ATM (Asynchronous Transfer Mode) e IPsec (IP security). Consiste nel concatenare un nonce e un counter, cifrare il risultato ed effettuare un XOR con il blocco di testo in chiaro, aumentando il contatore per ogni blocco successivo. È semplice, parallelizzabile e molto efficiente.

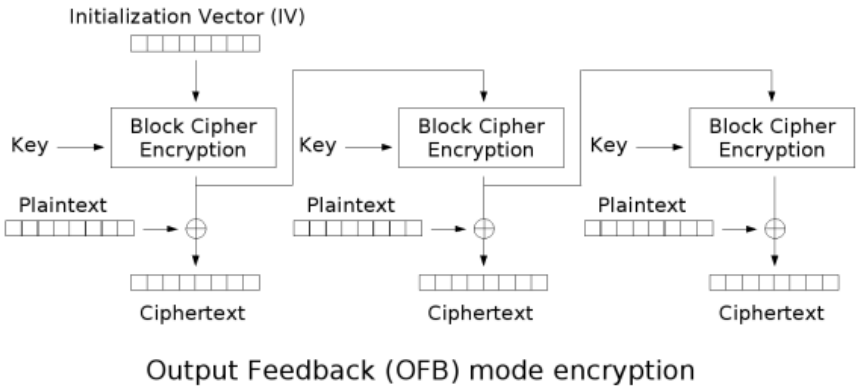


Figura 3.7: Modalità OFB. (Fonte: Wikipedia)

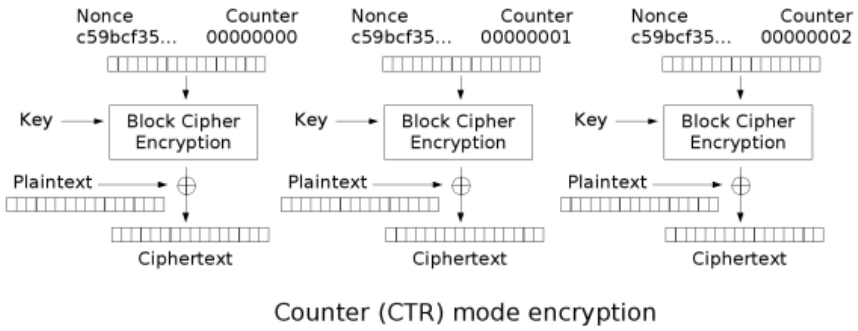


Figura 3.8: Modalità CTR. (Fonte: Wikipedia)

Padding

Il padding è una tecnica di riempimento utilizzata per adattare la lunghezza dei dati da cifrare a un multiplo della dimensione del blocco prevista dall'algoritmo. I valori dei bit aggiunti dipendono dallo standard di padding adottato. Quello più comune, propone di aggiungere al fondo del blocco, se n byte non sono usati, n byte che codificano il valore n in binario puro.

servirebbe un'immagine o un esempio

È indispensabile, per una corretta gestione del padding, aggiungere un intero blocco di padding anche quando il messaggio è già un multiplo della dimensione del blocco. Questo serve a evitare ambiguità durante la decifratura, poiché non è possibile distinguere un ultimo blocco di dati che termina con una sequenza di byte che potrebbe essere interpretata come padding. Ad esempio, se si utilizza il padding proposto dal PKCS#5, non è possibile gestire correttamente un plaintext il cui ultimo blocco termini con un byte che codifica 1.

3.3 Integrità

L'obiettivo primario della crittografia simmetrica, come trattato in precedenza, è la **riservatezza** (o confidenzialità) dei dati, garantendo che un attaccante che intercetti una comunicazione cifrata non possa comprenderla. Tuttavia, un MITM che intercetti tale comunicazione, pur non potendo decifrarla, **potrebbe comunque modificarla in modo imprevedibile**. Tale modifica può rendere il messaggio incomprensibile al destinatario o far credere a quest'ultimo di averlo ricevuto più volte (ad esempio, si potrebbe duplicare un bonifico cifrato). Per prevenire tali manipolazioni e garantire

che i dati arrivino a destinazione così come sono stati originati, è indispensabile garantire anche la proprietà di **integrità**.

3.3.1 Principi di Controllo di Integrità

Il controllo di integrità si basa su un principio fondamentale che coinvolge il mittente e il ricevente:

1. **Generazione della Prova**: Il mittente utilizza una funzione deterministica per generare una **prova** della struttura dei dati in un momento in cui sono ritenuti corretti (prima di uscire dall'ambiente sicuro e di spedirli).
2. **Ricalcolo e Confronto**: Il destinatario ricalcola la stessa funzione sui dati ricevuti. Se i risultati delle due computazioni (quella lato mittente, ricevuta dopo la trasmissione, e quella ricalcolata localmente) coincidono, i dati sono considerati **integri**.

Questo meccanismo è alla base di molte soluzioni di sicurezza, inlcusa la **firma digitale**.

3.3.2 Digest e Funzioni di Hash Crittografiche

La prova di integrità è tipicamente realizzata tramite il calcolo di un **digest** (che può intendersi come un "riassunto" di lunghezza fissa) del messaggio da proteggere, utilizzando una **funzione di hash crittografica**. Una funzione di hash prende in input dati di qualsiasi lunghezza e produce in output un risultato di lunghezza fissa. Ad esempio, un file di 4 GB (come un DVD) genererà lo stesso numero fisso di bit (es. 256 bit) di un file di 1 byte.

Affinché la funzione di hash sia considerata resistente ad attacchi deliberati, essa deve possedere i seguenti requisiti:

- **Non invertibilità**: Deve essere **impossibile** (o computazionalmente infattibile) risalire al messaggio originale a partire dal digest.
- **Pseudocasualità**: L'output (il digest) deve sembrare, a tutti gli effetti, una stringa **pseudocasuale**. Una modifica di un singolo bit nel messaggio originale deve provocare un cambiamento radicale (circa il 50% dei bit) nel digest (proprietà di diffusione).
- **Efficienza**: Deve essere **veloce da calcolare**.
- **Resistenza alle collisioni**: Deve essere difficile generare due messaggi con lo stesso digest (con una lunghezza di 256 bit ci sono 2^{256} digest possibili, è davvero molto improbabile che due messaggi abbiano lo stesso digest).

Le funzioni di hash sono tipicamente **più veloci** rispetto agli algoritmi simmetrici, poiché non sono progettate per essere invertibili. Recentemente, però, alcuni algoritmi di tipo stream hanno raggiunto velocità paragonabili, proprio perché usano operazioni semplici e velocissime.

3.3.3 Resistenza alle Collisioni e Lunghezza del Digest

Poiché il dominio dei messaggi possibili è virtualmente infinito (composto da una infinità numerabile di messaggi) e il codominio dei digest è limitato (ad esempio 2^N valori, dove N è la lunghezza del digest in bit), è matematicamente certo che esisteranno infinite collisioni (messaggi diversi con lo stesso digest). La sicurezza di un algoritmo di hash si misura dalla difficoltà statistica di trovare collisioni.

Si definiscono due tipi principali di collisioni che sono interessanti per diversi tipi di attacco:

1. **Collisione di Tipo 1 (*First Preimage Attack*)**: Dato un messaggio fisso M, l'attaccante deve trovare un messaggio diverso $M' \neq M$ tale che $H(M) = H(M')$ (esempio di attacco: intercetto un documento firmato digitalmente, ne calcolo il digest, e cerco/genmero un altro documento che produca lo stesso digest per poterlo sostituire).
2. **Collisione di Tipo 2 (Collision Attack)**: A partire da una funzione di hash, l'attaccante deve creare **due messaggi qualsiasi** M e M' tali che $H(M) = H(M')$ (esempio di attacco: creo due contratti di acquisto, uno per 10 euro e uno per 1000 euro, cercando di scriverli in modo che abbiano lo stesso digest. Se hanno lo stesso digest, ai fini della firma digitale diventano

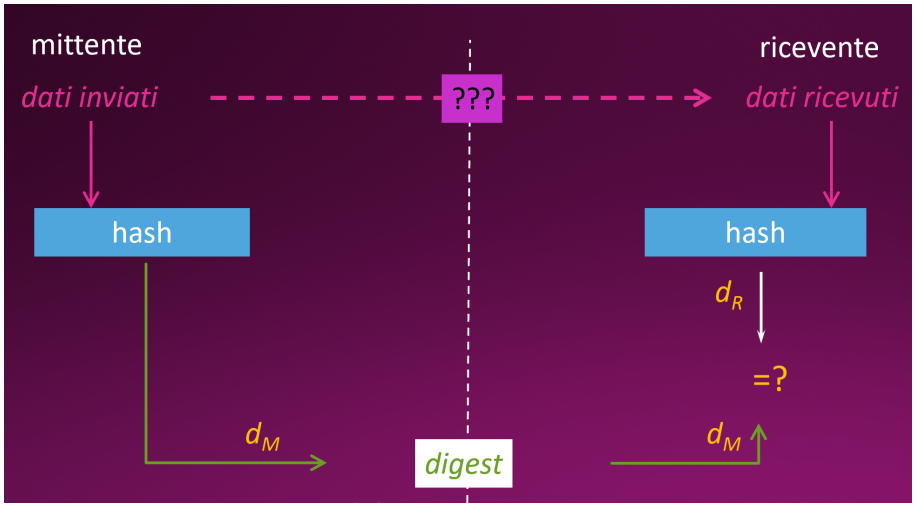


Figura 3.9: Schema di utilizzo di un digest.

interscambiabili. Posso far firmare il contratto da 10 euro, ma poi usare quello da 1000 euro al momento di incassare il valore.)

La resistenza di un algoritmo di hash è misurata in base al numero medio di tentativi necessari per forzare una collisione:

- Per la **Collisione di Tipo 1**, sono necessari in media 2^N tentativi, dove N è la lunghezza del digest.
- Per la **Collisione di Tipo 2**, sono necessari in media $2^{N/2}$ tentativi. Questo drastico dimezzamento è spiegato dal **Paradosso del Compleanno** (Birthday Paradox), che in statistica dimostra che la probabilità di trovare una corrispondenza (in questo caso, un compleanno o un digest) in un insieme in crescita è molto più alta di quanto sembrerebbe intuitivamente.[7]

Dato che gli attacchi di Tipo 2 sono computazionalmente meno onerosi per l'attaccante, su di essi si misura la **forza effettiva** dell'algoritmo. Convenzionalmente, un algoritmo di hash è ritenuto sufficientemente sicuro se la sua resistenza è pari ad almeno **80 bit**. Per garantire 80 bit di sicurezza contro gli attacchi di Tipo 2, la lunghezza del digest (N) deve essere di **almeno 160 bit** (poiché $160/2 = 80$). Tuttavia, i digest da 160 bit (come SHA-1) sono ormai considerati obsoleti o indeboliti (nel 2005, SHA-1 è stato violato in 2^{69} tentativi, compromettendo la sua sicurezza a 80 bit). Per i moderni standard di sicurezza, si considerano **digest** ≥ 256 **bit**.

Un criptosistema è considerato "**equilibrato**" quando l'algoritmo di cifratura simmetrica e la funzione di digest hanno la stessa resistenza sia per attacchi di tipo 1 che di tipo 2, ad esempio, AES-128 (forza 2^{128}) si abbina a un digest di 256 bit (forza contro collisioni 2^{128}), e AES-256 (forza 2^{256}) si abbina a un digest di 512 bit (forza contro collisioni 2^{256}).

Algoritmi di Hash Crittografici

Storicamente sono stati utilizzati diversi algoritmi di hash, tra cui **MD** (Message Digest), di cui fanno parte MD2, MD4 ed MD5, tutti progettati dal matematico Ron Rivest. Questi algoritmi, caratterizzati da digest relativamente brevi (128 bit), sono ormai considerati completamente **obsoleti** a causa di gravi vulnerabilità crittanalitiche che consentono di generare collisioni in tempi molto brevi.

Esistono poi altri algoritmi **SHA** (Secure Hash Algorithm), con SHA-1, SHA-2 e SHA-3. **SHA-1** (160 bit) è un algoritmo che è stato ampiamente utilizzato, ma è ormai deprecato. La famiglia attualmente ritenuta robusta è **SHA-2** (che include SHA-256 e SHA-512), che ne ha

[7] Per approfondire: Wikipedia, 2025

allungato il digest e integrato qualche funzione[8]. Il vincitore della competizione pubblica indetta dal NIST tra il 2008 e il 2012 è stato **Keccak** (pronunciato "catch-ack"), standardizzato come **SHA-3**. SHA-3, grazie alla sua costruzione multidimensionale, è considerato particolarmente robusto e più veloce di SHA-2. È stato anche molto apprezzato per la sua intuitività e per la facilità con cui è stato possibile verificare le proprietà richieste.

3.3.4 Message Integrity Code (MIC)

In crittografia, per garantire l'integrità dei messaggi, si aggiunge un codice chiamato **MIC (Message Integrity Code)**[9]. Il MIC è un **valore calcolato a partire dal messaggio** che viene trasmesso insieme ad esso, e che consente al destinatario di verificare che il contenuto non sia stato alterato durante la trasmissione.

Nel caso più semplice, il MIC può coincidere con il digest di una funzione di hash (es. SHA-256), calcolato come $\text{MIC} = H(M)$. Tuttavia, tale costruzione non è sicura contro un attaccante attivo: chiunque può modificare il messaggio, ricalcolare un nuovo MIC valido per il messaggio alterato e inviarlo al destinatario. In questo modo, la verifica di integrità risulterebbe positiva anche se il contenuto fosse stato manomesso.

Il semplice invio di un messaggio M e del suo digest D su un canale non protetto è insufficiente, poiché se un attaccante (MITM) modifica M in M', può ricalcolare facilmente il nuovo digest D' utilizzando la stessa funzione di hash (pubblica) e ingannare il ricevente, che accetterà il messaggio modificato come integro. Questo tipo di scenario si chiama attacco di *manomissione + ricalcolo*: per garantire l'integrità in un ambiente ostile, il digest deve essere **protetto**. In altre parole, un digest o MIC "non keyed" protegge solo da errori accidentali di trasmissione, non da modifiche intenzionali.

Inserire figura: figura slide 42 e/o 46 critto1.

3.3.5 Message Authentication Code (MAC) e Message IDentifier (MID)

Per garantire realmente l'integrità in un ambiente ostile, il calcolo del MIC deve dipendere da un elemento segreto condiviso tra mittente e destinatario, ad esempio, una chiave K. In questo caso, il MIC diventa un vero e proprio **MAC (Message Authentication Code)**, cioè un digest autenticato. Può, ad esempio, essere protetto mediante l'uso di un segreto crittografico, ad es. una chiave, in modo che solo chi conosce la chiave possa calcolare un MAC valido, e quindi solo messaggi generati da entità legittime verranno accettati.

Oltre all'attacco di manomissione e ricalcolo, un'altra minaccia è costituita dagli attacchi di tipo **replay**. In questo scenario, l'attaccante non modifica i dati, ma li riutilizza in un momento successivo per ottenere lo stesso effetto più volte. Un attaccante può infatti intercettare un messaggio legittimo completo del suo MIC (o MAC) valido — ad esempio un ordine "paga 1000€" — e ritrasmetterlo successivamente, anche più volte. Poiché il messaggio e il suo codice di integrità sono entrambi autentici, il destinatario li accetterà ogni volta come se fossero nuovi. Il problema non è che l'integrità fallisca, ma che il destinatario non possa sapere se il messaggio sia *nuovo* o una *copia* di uno già ricevuto.

Questo tipo di attacco può avere conseguenze gravi: un ordine di pagamento eseguito più volte, un comando di apertura di una porta di sicurezza ripetuto in momenti non autorizzati, o un'operazione di login replicata per mantenere accessi abusivi.

Per prevenire gli attacchi replay, i protocolli sicuri includono, tra i dati autenticati, un elemento di "freschezza" concatenato al messaggio, come un contatore, un timestamp o un valore casuale

[8] tra le migliorie ci sono, ad esempio, un maggior numero di round di elaborazione e funzioni di mixing più complesse, permettendo una migliore *diffusione* dei bit

[9] Il digest è un tipo di MIC.

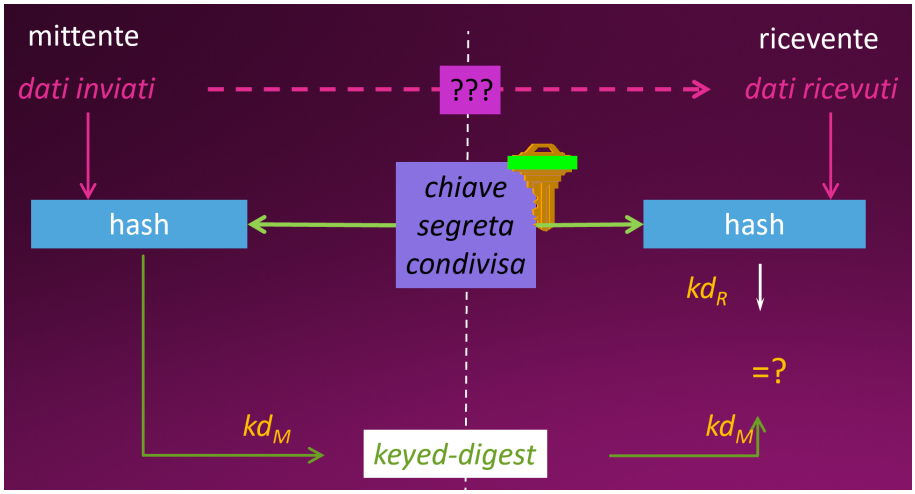


Figura 3.10: Schema di utilizzo di un keyed digest.

(nonce). Questo valore, chiamato in generale **MID (Message IDentifier)**, viene incluso nel calcolo del MIC (o MAC), ad esempio come MIC $= H(M \,||\, \text{MID})$, in modo che il destinatario possa riconoscere e scartare i messaggi duplicati o fuori sequenza.

Dunque l'integrità è in stretto legame con l'autenticazione, e spesso vi è bisogno di garantire entrambe queste proprietà, per cui in queste situazioni viene utilizzato il MAC.

Keyed Digest: Autenticazione basata sulla Conoscenza

Un MAC particolarmente efficiente e semplice per proteggere il digest è il **Keyed Digest**. Questo metodo, invece di cifrare il digest (operazione computazionalmente onerosa), si basa sulla concatenazione del messaggio M a una chiave segreta K condivisa solo tra mittente e ricevente:

$$kd = H(K \,||\, M) \quad \text{o} \quad kd = H(M \,||\, K)$$

Solo chi conosce la chiave segreta K è in grado di ricalcolare e confrontare il keyed digest trasmesso. Un attaccante che voglia modificare il messaggio M in M' e generare un kd' valido dovrebbe prima **indovinare la chiave segreta** K, rendendo l'attacco equivalente a una ricerca esaustiva della chiave, con una resistenza pari a 2^N, dove N è la lunghezza della chiave.

Questo meccanismo non solo garantisce l'integrità, ma fornisce anche **autenticazione dei dati (o del mittente)**: se il keyed digest è valido, il ricevente può dedurre sia che il messaggio provenga dall'entità che possiede la chiave segreta K (autenticazione basata sulla conoscenza di un segreto condiviso), sia che il messaggio non sia stato alterato (integrità).

Si tenga a mente che il Keyed Digest da solo non pretende assolutamente di fornire **riservatezza**, e non può garantire il **non ripudio**, poiché la chiave K è condivisa tra mittente e destinatario e non consente di dimostrare in modo univoco chi abbia generato il messaggio.

Attacchi e Implementazioni Standard del Keyed Digest

La semplice concatenazione $H(K \,||\, M)$ o $H(M \,||\, K)$ è vulnerabile a specifici attacchi, noti come *length extension attacks*. Questo tipo di attacco sfrutta il funzionamento interno di alcune funzioni di hash (come SHA-1 e SHA-2, costruzioni di Merkle-Damgård), che elaborano il messaggio in blocchi sequenziali e propagano uno stato interno. Quindi, un attaccante che conosca $H(K \,||\, M)$ (ma non la chiave K) potrebbe procedere come se avesse accesso al contesto interno e calcolare $H(K \,||\, M \,||\, M')$ senza conoscere K; analogamente, sarebbe possibile anteporre dei dati aggiuntivi M' nel caso in cui la chiave fosse alla fine: $H(M' \,||\, M \,||\, K)$.

SHA-3/Keccak, invece, utilizza una costruzione diversa (di tipo *sponge*) immune a questo tipo di attacco, poiché una parte dello stato resta "segreta".

Per mitigare questi attacchi, esistono vari tipi di soluzioni:
- Definire il keyed digest come $kd = H(K||M||K)$, concatenando la chiave sia prima che dopo il messaggio.
- Inserire la lunghezza di M tra i dati.
- Utilizzare implementazioni standardizzate, come HMAC.

HMAC

HMAC (Hash-based Message Authentication Code) è il metodo standardizzato (RFC 2104) più diffuso e sicuro per generare un keyed digest. HMAC applica la funzione di hash due volte in modo annidato, utilizzando chiavi interne ed esterne derivate dalla chiave segreta condivisa K.

Il funzionamento di HMAC si basa su due costanti predefinite: *ipad* (inner padding), una sequenza di byte con valore 0x36 ripetuta per la lunghezza del blocco della funzione hash, e *opad* (outer padding), una sequenza di byte con valore 0x5C ripetuta per la stessa lunghezza.

La chiave K viene adattata alla lunghezza del blocco della funzione hash: se è più lunga, viene prima accorciata con algoritmi di hash ($K' = H(K)$); se è più corta, viene riempita con zeri fino a raggiungere la lunghezza del blocco ($K' = H(K \,||\, \text{zero-padding})$); altrimenti viene utilizzata così com'è ($K' = K$).

La formula completa di HMAC è:

$$\text{HMAC}(K', M) = H\Big((K' \oplus \text{opad}) \,||\, H\big((K' \oplus \text{ipad}) \,||\, M\big)\Big)$$

Grazie a questa struttura annidata, HMAC risulta immune ai length extension attack e fornisce una protezione robusta dell'integrità e dell'autenticazione dei messaggi, mantenendo al contempo un'elevata efficienza computazionale. È utilizzato in numerosi protocolli di sicurezza come TLS, IPsec e SSH.

CBC-MAC

Per sistemi con risorse computazionali limitate (come gli IoT), che potrebbero non avere spazio sufficiente per implementare librerie di digest dedicate, esiste il **CBC-MAC**. Questo metodo sfrutta un algoritmo di cifratura simmetrico a blocchi (come AES in modalità CBC con IV nullo), utilizzando il ciphertext dell'ultimo blocco come MAC, poiché l'ultimo blocco è influenzato da tutti i blocchi precedenti. Tuttavia, oltre a essere più lento rispetto all'HMAC, è anche attaccabile in molti modi. Per questo motivo, sarebbe preferibile usare delle varianti del CBC-MAC che usano più di una chiave (es. OMAC).

Per il prof: Perché solo per lunghezza fissa? Inoltre, perché è necessario che IV$= 0$?

3.4 Funzioni combinate per Integrità e Riservatezza

Quando sono richieste sia la **riservatezza** (ottenuta tramite cifratura simmetrica con chiave K_1) che l'**integrità/autenticazione** (ottenuta tramite MAC con chiave K_2), è necessario combinare le primitive. Una combinazione errata di algoritmi sicuri può generare vulnerabilità. Si lascia infatti troppa libertà ai programmatori, che potrebbero usare primitive forti in modo scorretto, rendendole di fatto molto deboli.

Sono state definite diverse modalità di composizione, ognuna con pro e contro:

1. **Authenticate-And-Encrypt (A&E):** $\text{enc}(K_1, P) \,||\, \text{MAC}(K_2, P)$. Il MAC è calcolato sul plaintext P ed è inviato in chiaro, in parallelo al ciphertext. È vulnerabile ad attacchi ad oracolo[10] ed attacchi DoS, poiché il destinatario deve **sempre decifrare** il messaggio per ottenere il P necessario a ricalcolare il MAC. È utilizzato in SSH.

[10] Un attacco ad oracolo consiste nel sottoporre ripetutamente messaggi modificati al sistema per osservarne le risposte e dedurre informazioni sulla chiave o sul plaintext.

2. **Authenticate-Then-Encrypt (AtE):** $\text{enc}(K_1, P \,||\, \text{MAC}(K_2, P))$. Il MAC è calcolato sul plaintext, concatenato al messaggio e **poi tutto viene cifrato**. Sebbene il MAC sia protetto (riducendo gli attacchi oracolo), richiede ancora la decifratura preventiva per la verifica del MAC, esponendo a DoS. Utilizzato in SSL/TLS.
3. **Encrypt-Then-Authenticate (EtA):** $\text{enc}(K_1, P) \,||\, \text{MAC}(K_2, \text{enc}(K_1, P))$. Il MAC è calcolato sul **ciphertext** C. Questo consente di verificare l'integrità **prima della decifratura**, bloccando attacchi DoS mirati a sprecare risorse nella decifratura di messaggi alterati. Utilizzato in IPsec.

3.4.1 Authenticated Encryption (AE) e AEAD

Per evitare errori di composizione ed ottimizzare le prestazioni, è stato introdotto il concetto di **Authenticated Encryption (AE)**, un unico algoritmo ("black box") che fornisce contestualmente riservatezza e autenticazione dei dati, utilizzando un'unica chiave e una chiamata a procedura.

Inserire figura: AEAD, pag 55.

Nei contesti di rete, spesso è necessario proteggere anche i metadati che non possono essere cifrati (come l'header di un pacchetto IP, che i router utilizzano per l'instradamento). In questi casi, si ricorre all'AE con Dati Associati (AEAD), in cui l'algoritmo protegge il payload cifrato e i **Dati Associati (AD, Associated Data)** che restano in chiaro.

forse andrebbe spiegato che significa on-line e single/double-pass.
Gli algoritmi AEAD(?) si possono classificare in base a due caratteristiche: on/off-line e single/double-pass. direi che conviene aggiungere quelle info

L'algoritmo AEAD più diffuso e standardizzato è il **GCM** (Galois Counter Mode). GCM è particolarmente veloce (di tipo on-line single-pass) perché esegue la cifratura del payload e il calcolo del MAC (o Tag) in **parallelo** tramite operazioni nel campo di Galois, risultando significativamente più rapido e meno prono agli errori rispetto alla doppia operazione di cifratura simmetrica + HMAC. Nel caso di cifratura, questa funzione dato plaintext e chiave restituisce ciphertext e MAC, nel caso di decifratura invece controlla prima l'integrità attraverso il MAC, e solo in caso positivo decifra il ciphertext. È utilizzato da TLS.

Altri algoritmi della stessa classe sono:

- OCB 2.0 (on-line single-pass), che è il più rapido in assoluto, ma essendo stato brevettato GPL si è diffuso poco.
- EAX (on-line double-pass), più lento ma di dimensioni ridotte, quindi utile ad esempio su sistemi con risorse limitate (ad es. embedded).
- CCM (off-line double pass), il più lento in assoluto.

Grazie a una competizione chiamata CAESAR (Competition for Authenticated Encryption: Security, Applicability, and Robustness), nel 2019 sono stati scelti anche altri algoritmi per l'Authenticated Encryption, ma nessuno di essi è davvero usato nella pratica, dove GCM regna sovrano.

4. Crittografia parte 2

In questo capitolo proseguiamo l'analisi della crittografia, concentrandoci su aspetti più avanzati e sulle tecniche specifiche utilizzate per garantire la sicurezza delle comunicazioni digitali. Tratteremo argomenti quali la crittografia asimmetrica e le firme digitali.

4.1 Limitazioni della crittografia simmetrica

La crittografia simmetrica, pur essendo efficiente e veloce nel cifrare e nel decifrare grandi quantità di dati, presenta una limitazione fondamentale: **la necessità di condividere una chiave segreta** tra mittente e destinatario.

Questo apparente vantaggio della semplicità nasconde, in realtà, diverse problematiche critiche. Innanzitutto, quando la stessa chiave deve essere condivisa tra più partecipanti, la sicurezza del sistema si deteriora progressivamente. Ogni nuova entità che ottiene accesso alla chiave rappresenta un potenziale punto di vulnerabilità: maggiore è il numero di persone che condividono la chiave, maggiore è la probabilità che venga compromessa, perduta o utilizzata impropriamente. Questo accade perché ogni partecipante deve conservare la chiave in modo sicuro, trasmetterla attraverso canali protetti e proteggerla da accessi non autorizzati: un singolo errore o debolezza in qualsiasi punto della catena può compromettere l'intera sicurezza del sistema.

Un secondo problema, ancora più insidioso, riguarda la **crescita quadratica del numero di chiavi necessarie**. In un sistema dove n entità devono comunicare in modo sicuro a coppie, il numero di chiavi necessarie è dato dalla formula:

$$N_K = \frac{n(n-1)}{2}$$

Questo significa che, in una rete con soli 10 utenti, servono già 45 chiavi diverse da memorizzare, mentre con 100 utenti ne servono ben 4.950. La gestione diventa rapidamente impraticabile, rendendo la crittografia simmetrica inadeguata per sistemi di comunicazione su larga scala, in cui è necessaria una soluzione più scalabile ed efficiente.

Infine, il problema sicuramente maggiore è la **distribuzione delle chiavi**, ovvero come far sì che mittente e destinatario concordino sull'uso di una chiave segreta prima che possano iniziare

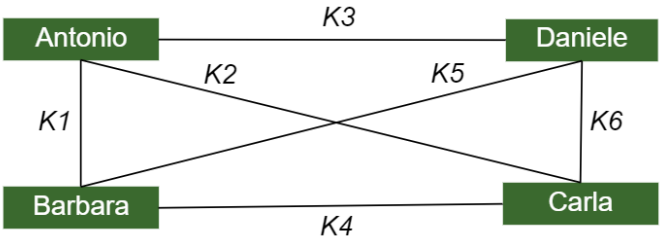


Figura 4.1: Rappresentazione grafica di come la relazione tra il numero di chiavi e di utenti sia quadratica in un sistema di crittografia simmetrica.

a comunicare in maniera cifrata. Una soluzione apparentemente semplice è la **distribuzione Out-Of-Band (OOB)**, che consiste nell'utilizzare un canale di comunicazione completamente diverso e indipendente da quello che verrà poi utilizzato per lo scambio dei messaggi cifrati. Ad esempio, si potrebbe consegnare la chiave di persona, inviarla tramite corriere fidato o comunicarla telefonicamente. Tuttavia, questa soluzione presenta diverse criticità che la rendono poco praticabile e sconsigliabile nella maggior parte degli scenari reali. In primo luogo, richiede la disponibilità di un canale alternativo sicuro, che potrebbe non essere sempre accessibile o affidabile: la consegna fisica delle chiavi è costosa, lenta e non scalabile; le comunicazioni telefoniche possono essere intercettate; i corrieri possono essere compromessi. Inoltre, la distribuzione OOB introduce un overhead logistico significativo: ogni volta che è necessario stabilire una nuova comunicazione sicura o rinnovare una chiave, si deve ricorrere nuovamente al canale alternativo, il che rende il processo macchinoso e inefficiente. La mancanza di automazione limita drasticamente l'applicabilità di questo approccio in contesti dinamici in cui le chiavi vanno distribuite o aggiornate con frequenza. Per questi motivi, la distribuzione out-of-band, pur essendo teoricamente sicura, risulta inadeguata per i sistemi moderni che richiedono scalabilità, velocità e gestione automatizzata delle chiavi.

Vedremo quindi, nel corso di questo capitolo, come la crittografia asimmetrica possa offrire soluzioni efficaci per risolvere questi problemi.

4.2 Crittografia asimmetrica

La crittografia asimmetrica, nota anche come crittografia a chiave pubblica, rappresenta una svolta significativa rispetto alla crittografia simmetrica. In questo paradigma, ogni utente possiede una coppia di chiavi: una **chiave pubblica**, che può essere condivisa liberamente con chiunque, e una **chiave privata**, che deve essere mantenuta segreta e conosciuta solo dal proprietario. Questa distinzione fondamentale tra le due chiavi consente di superare molte delle limitazioni della crittografia simmetrica. Questo approccio risolve il problema della distribuzione delle chiavi, poiché la chiave pubblica può essere condivisa liberamente senza compromettere la sicurezza della comunicazione.

Inoltre, la crittografia asimmetrica facilita la gestione delle chiavi in ambienti con molti utenti. Poiché ogni utente ha una sola coppia di chiavi, il numero totale di chiavi necessarie cresce linearmente con il numero di utenti, anziché quadraticamente come nella crittografia simmetrica. Questo rende la crittografia asimmetrica molto più scalabile e adatta a sistemi di comunicazione su larga scala, come Internet.

Tuttavia, la crittografia asimmetrica introduce anche nuove sfide, come la gestione delle chiavi private e la necessità di garantire l'autenticità delle chiavi pubbliche. Per affrontare queste problematiche, sono stati sviluppati vari protocolli e infrastrutture, tra cui le Certification Authority (CA), che svolgono un ruolo cruciale nella verifica dell'identità degli utenti e nella distribuzione delle chiavi pubbliche.

4.2.1 Chiavi con funzione reciproca

La crittografia asimmetrica offre un duplice vantaggio: può essere utilizzata sia per garantire la **riservatezza** dei messaggi sia per fornire **autenticazione** dell'identità del mittente. Questa versatilità deriva dalla natura duale della relazione tra le due chiavi.

Riservatezza

Per garantire la **riservatezza** di un messaggio, *il mittente utilizza la chiave pubblica del destinatario per cifrare il contenuto*. Poiché solo il destinatario possiede la chiave privata corrispondente, sarà l'unico in grado di decifrare e leggere il messaggio. Questo garantisce che, anche se il messaggio cifrato venisse intercettato durante la trasmissione, nessun attaccante potrebbe accedere al contenuto originale.

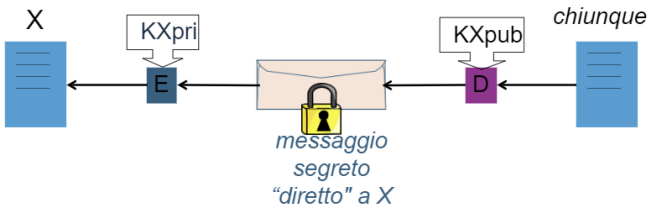


Figura 4.2: Esempio generico di riservatezza in crittografia asimmetrica.

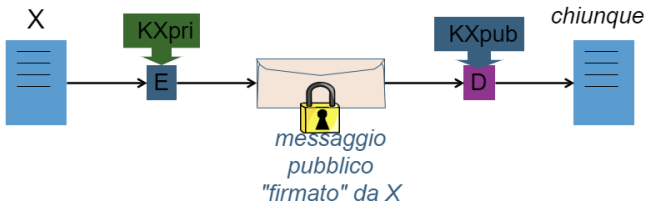


Figura 4.3: Esempio generico di autenticazione in crittografia asimmetrica.

Esempio (Figura 4.2): Alice vuole inviare un messaggio confidenziale a Bob.

1. Alice recupera la chiave pubblica di Bob ($K_{Bob,pub}$)
2. Alice cifra il messaggio M usando $K_{Bob,pub}$: $C = enc(M, K_{Bob,pub})$
3. Alice invia il messaggio cifrato C a Bob
4. Bob decifra il messaggio usando la sua chiave privata: $M = dec(C, K_{Bob,priv})$

In questo scenario, la riservatezza è garantita, ma Bob non può essere certo che il messaggio provenga effettivamente da Alice, poiché chiunque conosca la sua chiave pubblica potrebbe aver cifrato il messaggio.

Autenticazione

Per garantire l'**autenticazione**, il processo viene invertito: *il mittente cifra il messaggio (o un suo riassunto) con la propria chiave privata*. Chiunque può decifrare il messaggio usando la chiave pubblica del mittente, ma il fatto che la decifratura abbia successo prova che il messaggio è stato cifrato con la chiave privata corrispondente, autenticando quindi l'identità del mittente.

Esempio (Figura 4.3): Alice vuole dimostrare a Bob di essere davvero lei.

1. Alice cifra il messaggio M con la sua chiave privata: $C = enc(M, K_{Alice,priv})$
2. Alice invia il messaggio cifrato C a Bob
3. Bob recupera la chiave pubblica di Alice ($K_{Alice,pub}$)
4. Bob decifra il messaggio: $M = dec(C, K_{Alice,pub})$

Se la decifratura è riuscita e produce un messaggio sensato o il dato atteso, Bob può essere certo che il messaggio proviene da Alice, poiché solo lei possiede la chiave privata usata per cifrarlo. Tuttavia, in questo caso non c'è riservatezza: chiunque con accesso alla chiave pubblica di Alice può leggere il messaggio.

Combinazione di Riservatezza e Autenticazione

Per ottenere sia **riservatezza che autenticazione**, è possibile combinare i due approcci applicando una doppia cifratura. Alice vuole inviare a Bob un messaggio riservato e autenticato: prima cifra il messaggio M con la sua chiave privata ($C_1 = enc(M, K_{Alice,priv})$) per garantirne l'autenticazione, poi cifra C_1 con la chiave pubblica di Bob ($C_2 = enc(C_1, K_{Bob,pub})$) per garantirne la riservatezza. Bob, ricevuto C_2, decifra prima con la sua chiave privata ($C_1 = dec(C_2, K_{Bob,priv})$) e poi con la chiave pubblica di Alice ($M = dec(C_1, K_{Alice,pub})$). In questo modo, Bob può essere certo che il messaggio proviene da Alice e che nessun altro ha potuto leggerlo durante la trasmissione.

4.3 Algoritmi di crittografia asimmetrica

4.3.1 Generazione delle chiavi

Un aspetto fondamentale della crittografia asimmetrica è che le due chiavi non vengono generate in modo indipendente o casuale, ma sono **matematicamente correlate** tra loro. La coppia di chiavi (K_{pub}, K_{priv}) è generata da un algoritmo che garantisce una relazione matematica precisa: ciò che viene cifrato con una chiave può essere decifrato solo con l'altra, e viceversa.

Il processo di generazione parte tipicamente dalla scelta di parametri matematici specifici. La sicurezza dell'intero sistema si fonda sull'impossibilità di risalire dalla chiave pubblica alla chiave privata. Per questo motivo, il processo che un attaccante dovrebbe seguire per riuscire a ricavare la chiave privata dalla chiave pubblica deve necessariamente essere un problema matematico *computazionalmente difficile*. Informalmente, un problema è computazionalmente difficile se è esponenziale, cioè l'aumento dello sforzo che un attaccante deve sostenere cresce esponenzialmente con le dimensioni del problema. In pratica, un problema ritenuto computazionalmente difficile è idoneo all'uso nella crittografia asimmetrica, se è possibile scegliere le dimensioni del problema (es. numero di bit della chiave pubblica/privata) in modo da rendere altamente improbabile per un attaccante risolverlo nel tempo a sua disposizione.

I due problemi matematici complessi su cui si basano gli algoritmi di crittografia asimmetrica più diffusi al momento sono:

- **Fattorizzazione di numeri primi**: dato un numero n, prodotto di due numeri primi molto grandi p e q (cioè $n = p \cdot q$ di almeno 2048 bit), è estremamente difficile risalire a p e q conoscendo solo n. Mentre moltiplicare due numeri primi è un'operazione banale, fattorizzare il risultato richiede un tempo esponenziale all'aumentare di n.
- **Logaritmo discreto**: dato un numero primo p molto grande (es. di almeno 1024 bit), un generatore g e un valore y, è computazionalmente intrattabile determinare l'esponente x tale che $y = g^x \mod p$. Mentre calcolare l'elevamento a potenza modulare è un'operazione efficiente[1], il problema inverso, cioè trovare x dato g, y e p, richiede un tempo esponenziale all'aumentare della dimensione di p, rendendolo praticamente irrealizzabile con le risorse computazionali attuali.

Entrambi questi problemi, ed in particolare il logaritmo discreto, sono ritenuti intrattabili su computer tradizionali. Sono invece semplici da risolvere con computer quantistici, che però non sono ancora disponibili.

La generazione tipica di una coppia di chiavi segue questi passaggi concettuali:

1. **Costruzione del contesto matematico**. Utilizzando i parametri primi, si costruisce una struttura algebrica basata su operazioni modulari. I dettagli specifici variano a seconda dell'algoritmo utilizzato, ma il principio generale è quello di creare una relazione matematica tra parametri pubblici e privati che sia facile da calcolare in una direzione ma computazionalmente intrattabile da invertire.
   In questo contesto, è spesso importante usare numeri primi per le loro proprietà matematiche uniche. Si generano uno o più numeri primi molto grandi, tipicamente dell'ordine di centinaia o migliaia di bit. Questo processo richiede l'uso di test di primalità probabilistici per verificare se un numero casualmente generato sia primo con probabilità elevata. La scelta di numeri primi sufficientemente grandi è cruciale: numeri troppo piccoli renderebbero il sistema vulnerabile ad attacchi di forza bruta.
2. **Derivazione delle chiavi**: La chiave pubblica viene derivata dai parametri iniziali e può essere condivisa liberamente. La chiave privata viene calcolata a partire dai parametri e/o risolvendo equazioni specifiche. La separazione tra informazioni pubbliche e private

[1] L'elevamento a potenza modulare è un'operazione che calcola $a^b \mod n$, ovvero il resto della divisione di a^b per n. Esistono algoritmi efficienti che permettono di eseguire questa operazione in tempo logaritmico rispetto all'esponente.

garantisce che la conoscenza della chiave pubblica non consenta di derivare quella privata in tempi ragionevoli.

La relazione matematica tra le chiavi garantisce che le operazioni di cifratura e di decifratura siano inverse l'una dell'altra. Deve, quindi, valere che:

$$dec(K_{priv}, enc(K_{pub}, M)) = M \quad \text{e} \quad dec(K_{pub}, enc(K_{priv}, M)) = M$$

È importante notare, però, che non per tutti gli algoritmi asimmetrici le due chiavi sono intercambiabili ed è solo una questione di scegliere quale destinare all'uso pubblico e quale a quello privato. Inoltre, non è detto che le prestazioni siano le stesse usando una chiave pubblica o privata. Per alcuni algoritmi, le operazioni matematiche e le strutture algebriche sottostanti possono differire significativamente, pur mantenendo la proprietà fondamentale che le chiavi sono correlate in modo tale da permettere operazioni inverse. Ciò implica che alcuni algoritmi possono essere ottimizzati per specifiche operazioni (ad esempio, solo cifratura con la chiave pubblica per garantire la riservatezza o solo cifratura con la chiave privata, es. per calcolare una firma digitale), limitandone la versatilità rispetto ad altri algoritmi che supportano entrambe le funzionalità.

Questa correlazione matematica di invertibilità introduce però un **rischio intrinseco**: se un attaccante riuscisse a ottenere la chiave pubblica e avesse sufficienti risorse computazionali, potrebbe teoricamente derivare la chiave privata. La sicurezza del sistema si basa quindi sulla **difficoltà computazionale** di questo problema: fattorizzare numeri molto grandi (di almeno 2048 bit) richiede un tempo talmente elevato da rendere l'attacco impraticabile con le tecnologie attuali. Per garantire un livello di sicurezza adeguato, a differenza delle chiavi simmetriche alle quali bastano 128 o 256 bit, le chiavi asimmetriche devono essere significativamente più lunghe, spesso nell'ordine di **2048 bit** o più, a seconda dell'algoritmo utilizzato e del livello di sicurezza desiderato. Questo rende la crittografia asimmetrica più lenta e computazionalmente intensiva rispetto alla crittografia simmetrica, ma offre vantaggi significativi in termini di gestione delle chiavi e di scalabilità.

Panoramica degli algoritmi

Il concetto di crittografia asimmetrica è stato proposto per la prima volta nel 1976 da **Whitfield Diffie** e **Martin Hellman** nel loro articolo rivoluzionario *"New Directions in Cryptography"*. Questo lavoro ha introdotto l'idea di chiavi pubbliche e private, aprendo la strada allo sviluppo di numerosi algoritmi che hanno trasformato radicalmente il panorama della sicurezza informatica.

Il primo algoritmo pratico di crittografia asimmetrica fu **RSA (Rivest-Shamir-Adleman)**[2], sviluppato nel 1977. RSA si basa sulla difficoltà di fattorizzare il prodotto di due numeri primi molto grandi e rimane ancora oggi uno degli algoritmi più utilizzati.

Nel 1985 è stato introdotto **ElGamal**, basato sul problema del logaritmo discreto. Questo algoritmo ha ispirato lo sviluppo di **DSA (Digital Signature Algorithm)**, standardizzato dal NIST nel 1991 e utilizzato per le firme digitali.

L'avvento dei computer quantistici rappresenta una minaccia concreta per gli algoritmi di crittografia asimmetrica tradizionali, poiché sono in grado di risolvere i problemi associati alla crittografia asimmetrica tradizionale con algoritmi di computazionalità meno che esponenziale. Per affrontare questa sfida, il NIST ha avviato nel 2016 un processo di standardizzazione per gli algoritmi di crittografia **post-quantum**. Questi algoritmi si basano su problemi matematici ritenuti difficili da risolvere anche con computer quantistici; ad esempio, alcuni problemi sui reticoli matematici sono al momento ritenuti difficili da risolvere anche con computer quantistici. Sebbene i computer quantistici pratici siano ancora lontani e la transizione verso la crittografia post-quantum sia già iniziata, questi algoritmi sono ancora in fase di valutazione da parte della

[2]Ronald Rivest ha contribuito significativamente sia alla crittografia asimmetrica, creando RSA, sia a quella simmetrica, progettando algoritmi come RC2, RC4, RC5 e RC6, affermandosi come uno dei principali matematici nel campo della crittografia moderna.

comunità matematica. Data la necessità di un background matematico molto elevato, non verranno affrontati nel dettaglio in questo testo.

4.3.2 DSA

DSA (Digital Signature Algorithm) è un algoritmo sviluppato specificamente per la generazione e la verifica di firme digitali e si basa sul problema del logaritmo discreto.

La generazione delle chiavi in DSA prevede i seguenti passaggi:

1. **Scelta dei parametri di dominio**: vengono scelti tre parametri pubblici condivisi:
   - p: un numero primo molto grande
   - q: un numero primo più piccolo che divide $(p-1)$
   - g: un generatore del sottogruppo calcolato come $g = h^{(p-1)/q} \mod p$ dove h è un numero casuale tra 1 e $p-1$
2. **Generazione della chiave privata**: si sceglie un numero casuale x tale che $0 < x < q$. Questo valore deve essere mantenuto strettamente segreto e generato con un generatore di numeri casuali crittograficamente sicuro.
3. **Calcolo della chiave pubblica**: la chiave pubblica y viene calcolata come:

   $$y = g^x \mod p$$

   Questo valore può essere condiviso liberamente. La sicurezza si basa sul fatto che, conoscendo y, g e p, è computazionalmente intrattabile risalire a x (problema del logaritmo discreto).

DSA è stato progettato specificamente per la generazione e la verifica di firme digitali e non supporta operazioni di cifratura. La struttura matematica delle chiavi e le operazioni di *signature* dell'algoritmo sono ottimizzate esclusivamente per la firma digitale, rendendo impossibile il loro utilizzo per cifrare messaggi. Questo perché DSA si basa su una costruzione matematica che consente solo di generare e verificare firme, non di invertire l'operazione per recuperare un messaggio originale[3].

4.3.3 RSA

RSA è l'algoritmo di crittografia asimmetrica più diffuso e utilizzato. La sua sicurezza si fonda sulla difficoltà computazionale della fattorizzazione dei numeri primi.

Generazione delle chiavi

Il processo di generazione delle chiavi in RSA segue questi passaggi:

1. **Scelta dei numeri primi segreti**: si generano due numeri primi distinti, p e q, tipicamente di 1024 bit ciascuno (per chiavi totali di 2048 bit) o più.
2. **Calcolo del modulo**: si calcola il modulo n come prodotto dei due primi:

   $$n = p \cdot q$$

   Il valore n sarà parte sia della chiave pubblica che di quella privata. La sicurezza di RSA dipende dal fatto che, conoscendo n, sia computazionalmente intrattabile risalire a p e q.
3. **Calcolo della funzione toziente di Eulero**: si calcola $\phi(n)$, che rappresenta il numero di interi positivi minori di n e coprimi con esso[4]. ϕ può essere calcolato con formule semplici:

   $$\phi(n) = (p-1)(q-1)$$

   Questo valore è fondamentale per il calcolo della chiave privata e deve rimanere segreto. Conoscere $\phi(n)$ equivale a conoscere la fattorizzazione di n.

[3] Per approfondire: Geeks4Geeks, 2025

[4] Due numeri sono coprimi se il loro massimo comune divisore è 1

4. **Scelta dell'esponente pubblico**: si sceglie un esponente pubblico e tale che:
   - $1 < e < \phi(n)$
   - $\text{MCD}\big(e, \phi(n)\big) = 1$ (cioè e deve essere coprimo con $\phi(n)$)[5]

   Comunemente si usa $e = 65537 = 2^{16} + 1$, che offre un buon compromesso tra sicurezza ed efficienza computazionale. Questo valore è un **primo di Fermat** e consente operazioni di cifratura veloci grazie al numero ridotto di bit a 1 nella sua rappresentazione binaria (10000000000000001_2).
   Storicamente sono stati utilizzati anche altri primi di Fermat più piccoli:
   - $e = 3 = 2^{2^0} + 1$ (in binario: 11_2) - il più veloce computazionalmente
   - $e = 17 = 2^{2^2} + 1$ (in binario: 10001_2)
   - $e = 65537 = 2^{2^4} + 1$ (in binario: 10000000000000001_2) - lo standard attuale

   La scelta di questi valori specifici è dovuta a due proprietà fondamentali:
   (a) Sono **primi di Fermat** della forma $F_n = 2^{2^n} + 1$, garantendo che siano sempre coprimi con $\phi(n) = (p-1)(q-1)$;
   (b) Hanno solo **due bit a 1** nella rappresentazione binaria (il primo e l'ultimo), il che rende l'elevamento a potenza estremamente efficiente. Il bit più significativo (MSB) deve essere 1 per definire la lunghezza del numero, mentre il bit meno significativo (LSB) a 1 garantisce che il numero sia dispari, condizione necessaria per essere primo (eccetto il 2). Avere solo questi due bit a 1 minimizza il numero di moltiplicazioni necessarie durante l'elevamento a potenza modulare.

   Tuttavia, $e = 3$ ed $e = 17$ sono stati abbandonati progressivamente perché troppo piccoli e vulnerabili a determinati attacchi (es. *low public exponent attack*). Il valore $e = 65537$ è diventato lo standard attuale mantenendo l'efficienza computazionale grazie ai soli due bit a 1, ma è anche sufficientemente grande da resistere agli attacchi noti contro esponenti piccoli. Un esempio storico è l'attacco DoS rivolto contro alcuni server che validavano firme crittografiche. Firme generate con chiavi private, le cui corrispondenti chiavi pubbliche (contenute in certificati appositi) presentavano molti bit a 1 nella rappresentazione binaria, richiedevano tempi di verifica molto più lunghi e permettevano agli attaccanti di sovraccaricare i server. Era sufficiente generare certificati appositamente realizzati.
5. **Calcolo dell'esponente privato**: la chiave d viene calcolata determinando l'inverso modulare di e modulo $\phi(n)$, ovvero rispettando la seguente congruenza:

   $$d \equiv e^{-1} \mod \phi(n) \Rightarrow d \cdot e \equiv 1 \mod \phi(n)$$

   Questa congruenza implica che sia il prodotto $d \cdot e$ che 1, quando divisi per $\phi(n)$, lasciano lo stesso resto. Ciò rende computazionalmente intrattabile derivare d a partire da e e n. Ad esempio, un metodo per ottenere d da e è fattorizzare n per calcolare $\phi(n)$.
   Inoltre, questa formula garantisce che le operazioni di cifratura e decifratura siano inverse l'una dell'altra.
6. **Definizione delle chiavi finali**:
   - **Chiave pubblica**: (n, e) - può essere condivisa liberamente
   - **Chiave privata**: (n, d) - deve rimanere segreta.

Operazioni di cifratura e decifratura

Le operazioni di cifratura e decifratura in RSA si basano sull'elevamento a potenza modulare:

Cifratura: $C = M^e \mod n$	usando la chiave pubblica(n, e)
Decifratura: $M = C^d \mod n$	usando la chiave privata(n, d)

[5] MCD sta per Massimo Comune Divisore

dove M è il messaggio in chiaro ($0 \leq M < n$) e C è il messaggio cifrato.

Per messaggi più lunghi di n, è necessario suddividerli in blocchi per evitare la perdita di informazioni dovuta al modulo.

Dimostrazione
La correttezza di RSA si basa sul teorema di Eulero, che afferma: per ogni intero a coprimo con n, vale $a^{\phi(n)} \equiv 1 \mod n$. Inoltre, come già presentato, poiché d è l'inverso modulare di e modulo $\phi(n)$, per definizione vale che $e \cdot d \equiv 1 \mod \phi(n)$. Questo significa che esiste un intero k tale che $e \cdot d = k \cdot \phi(n) + 1$. Verifichiamo quindi che decifrando C otteniamo il messaggio originale M:

$$C^d \equiv (M^e)^d \equiv M^{ed} = M^{k\phi(n)+1} = M \cdot (M^{\phi(n)})^k \equiv M \cdot 1^k \equiv M \mod n$$

Operazioni di Firma e Verifica

Simmetricamente, RSA può essere utilizzato per la firma digitale:

$$\begin{aligned} &\text{Firma: } S = M^d \mod n && \text{usando la chiave privata}(n,d) \\ &\text{Verifica: } M = S^e \mod n && \text{usando la chiave pubblica}(n,e) \end{aligned}$$

dove M è il messaggio originale e S è la firma digitale.

Ottimizzazione Computazionale con il Teorema Cinese del Resto

In RSA le operazioni con la chiave privata (decifratura e firma digitale) sono significativamente più lente rispetto a quelle con la chiave pubblica (cifratura e verifica della firma). Questo è dovuto al fatto che l'esponente privato d è generalmente molto più grande di quello pubblico e, il che rende l'elevamento a potenza modulare più computazionalmente intensivo. Per mitigare questo svantaggio, viene spesso utilizzata un'ottimizzazione basata sul **Teorema Cinese del Resto** (Chinese Remainder Theorem, CRT), standardizzata come PKCS#1. Questa è una tecnica matematica che ottimizza la decifratura RSA, accelerando il tempo di calcolo fino a 4 volte. L'idea è sfruttare la fattorizzazione nota $n = p \cdot q$ per eseguire i calcoli modulari con due computazioni separate modulo p e modulo q (numeri più piccoli), per poi ricostruire il risultato finale[6]. Genera una rappresentazione diversa della chiave privata, rappresentabile concettualmente come:

$$f(x) \mod n \approx f(x) \mod p \quad \& \quad f(x) \mod q$$

Questo approccio è standardizzato e ampiamente utilizzato nelle implementazioni pratiche di RSA. Tuttavia, è importante notare che l'uso del CRT introduce una vulnerabilità agli **attacchi di tipo *fault injection***: se un attaccante riesce a indurre un errore controllato in una delle due computazioni in cui viene scomposto il calcolo (ad esempio tramite variazioni di tensione, temperatura o bombardamento con radiazioni), si ottenere un risultato parzialmente corretto che permette di fattorizzare n e quindi recuperare la chiave privata. Per questo motivo, le implementazioni critiche devono includere contromisure specifiche contro questi attacchi fisici.

RSA: debolezze

RSA, nonostante la sua robustezza teorica e la diffusione capillare, presenta alcune vulnerabilità intrinseche che vanno comprese e mitigate attraverso implementazioni corrette.

Small Encryption Exponent / low public exponent e Håstad broadcast L'uso di esponenti pubblici molto piccoli, come $e = 3$, introduce una vulnerabilità: se un mittente invia lo stesso messaggio M a tre destinatari diversi (nel caso di $e = 3$), ciascuno con la propria chiave pubblica ma tutti con lo stesso esponente $e = 3$, un attaccante che intercetta i tre messaggi cifrati può recuperare

[6] Per approfondire: Ben Lynn, Stanford

il testo in chiaro senza conoscere alcuna chiave privata. Questo accade perché i tre messaggi cifrati $C_1 = M^3 \mod n_1$, $C_2 = M^3 \mod n_2$ e $C_3 = M^3 \mod n_3$ forniscono informazioni sufficienti per ricostruire M^3 mediante tecniche matematiche che consentono di combinare le tre congruenze. Il punto cruciale è che, lavorando su moduli diversi e sufficientemente grandi, è possibile ricostruire il valore di M^3 non più come il resto di una divisione, bensì come un numero intero completo. Poiché i moduli n_1, n_2 e n_3 sono tutti molto grandi, il loro prodotto è enormemente superiore a M^3. Di conseguenza, quando si ricostruisce M^3 considerando un modulo pari al prodotto dei tre moduli originali, il risultato non subisce alcuna riduzione modulare: M^3 rimane semplicemente M^3 come numero intero. A questo punto, per recuperare il messaggio originale M, è sufficiente calcolare la radice cubica di tale valore, un'operazione matematica elementare che non richiede la conoscenza di alcuna chiave privata. Questo tipo di attacco evidenzia come la scelta di parametri crittografici apparentemente innocui possa compromettere gravemente la sicurezza del sistema.

La mitigazione principale consiste nell'adozione del parametro pubblico $e = 65537$ che riduce il rischio, pur mantenendo un'efficienza computazionale accettabile.

Uso della stessa coppia di chiavi per cifratura e firma Un errore comune nelle implementazioni consiste nell'utilizzare la stessa coppia di chiavi RSA sia per la cifratura che per la firma digitale. Questa pratica introduce rischi significativi di confusione tra le operazioni, rendendo possibile attacchi di tipo *cross-protocol*. Un attaccante potrebbe sfruttare la simmetria matematica di RSA per indurre un utente a firmare un messaggio che in realtà è un testo cifrato, o viceversa, compromettendo la sicurezza del sistema e recuperando il plaintext (la firma ed il cifrare sono operazioni opposte).

La mitigazione consiste nell'utilizzare coppie di chiavi diverse per le operazioni di cifratura e di firma.

Oracoli di decifratura Uno dei più insidiosi attacchi contro RSA è l'attacco dell'oracolo di decifratura. Un oracolo di decifratura è una tecnica per ottenere informazioni sulla chiave privata o sul testo in chiaro, che richiede operazioni da eseguire con la chiave privata. In un attacco di questo tipo, l'attaccante invia a un sistema legittimo una serie di testi cifrati appositamente costruiti e osserva le risposte ottenute. Queste risposte possono rivelare informazioni critiche sulla chiave privata o sul testo in chiaro, permettendo all'attaccante di ricostruire la chiave privata o di decifrare messaggi senza possederla.

Le mitigazioni principali includono:

- **Eliminazione degli oracoli**: garantire che il sistema non riveli mai informazioni sulla validità del padding o sulla correttezza della decifratura attraverso tempi di risposta, messaggi di errore o comportamenti osservabili;
- **Utilizzo di OAEP**: usare padding sicuri come Optimal Asymmetric Encryption Padding (OAEP) che introducono casualità e rendono ogni cifratura unica, prevenendo attacchi basati su testi cifrati ripetuti. **In generale, mai firmare o cifrare dati non opportunamente pre-processati con schemi di padding sicuri.**

Sfide RSA

Le RSA Factoring Challenges furono una serie di sfide pubbliche lanciate dai laboratori di RSA nel 1991 e concluse nel 2007 per testare la robustezza dell'algoritmo RSA e stimare il tempo necessario a fattorizzare numeri di diverse dimensioni. Queste sfide hanno fornito dati empirici fondamentali per determinare le dimensioni minime delle chiavi necessarie a garantire un livello di sicurezza adeguato.

Questi risultati empirici hanno portato alle raccomandazioni attuali: le chiavi da 1024 bit sono considerate obsolete e attaccabili entro alcuni mesi, mentre **2048 bit** rappresentano lo standard

minimo per applicazioni sicure, offrendo una protezione di vari anni, con **4096 bit** raccomandati per proteggere informazioni particolarmente sensibili o che devono rimanere riservate per decenni.

Le RSA Factoring Challenges furono ufficialmente ritirate nel 2007. La decisione fu motivata principalmente dal fatto che gli obiettivi educativi e di ricerca erano stati ampiamente raggiunti: la comunità scientifica aveva acquisito dati sufficienti per stimare la sicurezza di RSA e determinare le dimensioni appropriate delle chiavi. Inoltre, il crescente interesse per l'avvento di nuove sfide crittografiche (curve ellittiche prima, post-quantum poi) rese meno rilevante il focus sulla fattorizzazione RSA.

Twinkle & Twirl

Un aspetto interessante della storia della crittografia asimmetrica riguarda due algoritmi meno noti ma significativi: **Twinkle**[7] e **Twirl**[8]. Questi sono due progetti di dispositivi hardware specializzati proposti da Adi Shamir (uno dei tre ideatori di RSA). Questi dispositivi erano progettati appositamente per accelerare la fattorizzazione di numeri grandi mediante hardware specializzato.

Twinkle prometteva di accelerare significativamente la fase di raccolta dei dati, riducendo i costi e i tempi necessari per fattorizzare numeri di grandi dimensioni.

Twirl rappresentava un'evoluzione del concetto di Twinkle, con un'architettura completamente elettronica basata su ASIC. Secondo le stime di Shamir, un dispositivo Twirl con un costo stimato di circa 10 milioni di dollari avrebbe potuto fattorizzare chiavi RSA a 1024 bit in circa un anno di calcolo continuo, o significativamente meno nel caso fosse stata implementata una parallelizzazione nel calcolo delle fattorizzazioni.

Questi progetti non furono mai commercializzati né realizzati su larga scala, ma il loro impatto sulla comunità crittografica fu significativo. L'eredità di Twinkle e Twirl risiede nell'aver evidenziato l'importanza di considerare non solo gli algoritmi matematici, ma anche le potenziali minacce derivanti da implementazioni hardware specializzate a basso costo (relativo, almeno per i governi), stimolando la ricerca sia nel campo degli acceleratori crittografici sia nello sviluppo di standard di sicurezza più robusti.

4.4 Applicazioni della Crittografia Asimmetrica

La crittografia asimmetrica, o a chiave pubblica, si basa sull'utilizzo di coppie di chiavi, una privata (K_{priv}) mantenuta segreta e una pubblica (K_{pub}) ampiamente diffusa. Sebbene questi algoritmi, come **RSA** e **DSA**, offrano funzionalità cruciali come l'autenticazione e la riservatezza, sono caratterizzati da un **alto carico di elaborazione**. Per questo motivo, sono considerati inutili per la cifratura diretta di grandi quantità di dati. Ad esempio, un algoritmo come RSA (tipicamente usato con chiavi di 2048 bit) non può cifrare dati che, codificati come un numero in binario puro, siano superiori al modulo n. Un messaggio di 2048 bit non è sufficientemente lungo da essere interessante dal punto di vista informativo.

Gli algoritmi a chiave pubblica sono impiegati principalmente per due attività che richiedono l'uso di piccole quantità di dati: la **distribuzione di chiavi segrete** (chiavi simmetriche) e la creazione di **firme digitali**. La loro funzione principale è superare le limitazioni intrinseche della crittografia simmetrica, in particolare il problema della distribuzione sicura della chiave unica.

4.4.1 Scambio di Chiavi Simmetriche

Poiché gli algoritmi simmetrici (come AES) sono molto efficienti e in grado di cifrare terabyte di dati in pochi minuti, si combinano i vantaggi di entrambi i sistemi crittografici (simmetrico e non) attraverso un approccio ibrido, solitamente noto come **enveloped data**.

[7] Shamir, 2000
[8] Shamir, 2002

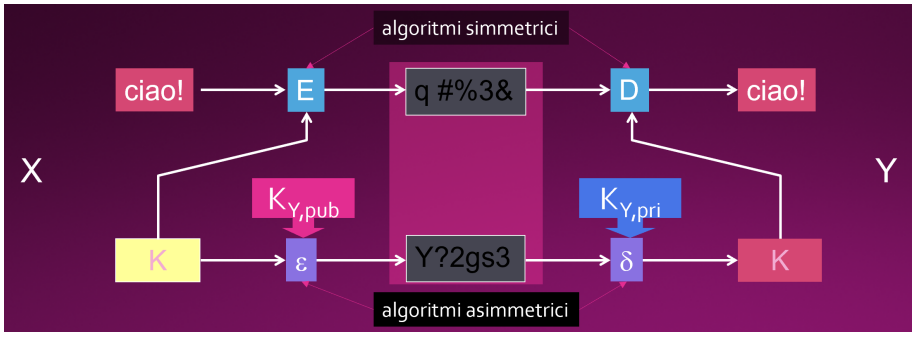


Figura 4.4: Scambio di chiavi simmetriche con *enveloped data*.

Cifratura Ibrida (Enveloped Data)

L'obiettivo è inviare un messaggio di grandi dimensioni e la relativa chiave simmetrica K in un'unica operazione, garantendo sempre la confidenzialità. Facendo riferimento alla Figura 4.4, in cui Alice deve inviare un messaggio confidenziale a Bob:

- Il mittente (Alice) inventa una chiave di messaggio simmetrica K (ad esempio, una chiave AES da 128 o 256 bit) per quella sola comunicazione.
- Alice cifra il messaggio M con l'algoritmo simmetrico e la chiave di messaggio K (operazione E), ottenendo il ciphertext C.
- Alice cifra la chiave di messaggio K utilizzando l'algoritmo asimmetrico e la **chiave pubblica del destinatario** ($K_{Bob,pub}$) (operazione ε).
- Alice invia il messaggio cifrato C e la chiave cifrata $\varepsilon_{K_{Bob,pub}}(K)$.
- Il destinatario (Bob) utilizza la propria **chiave privata** ($K_{Bob,pri}$) per decifrare la chiave di messaggio K.
- Infine, Bob utilizza K per decifrare il messaggio C.

Questo meccanismo è efficiente poiché la crittografia asimmetrica (più lenta) è applicata solo alla chiave di sessione K, di piccole dimensioni (quindi compatibili con i limiti matematici degli algoritmi asimmetrici). Inoltre, la chiave K viene generata *ex novo* per ogni messaggio, evitando il problema dell'"usura" (aging) delle chiavi nel tempo. Un potenziale rischio di questo approccio è che la chiave K generata localmente dal mittente **non sia veramente random** (ad esempio, influenzata da un attacco locale allo Pseudo-Random Number Generator) e quindi vulnerabile a un attacco di forza bruta se la sua entropia è insufficiente. Pertanto, in ambito militare si preferisce usare l'algoritmo di Diffie-Hellman, che non è soggetto a tale problema.

Algoritmo Diffie-Hellman

L'algoritmo **Diffie-Hellman (DH)** è stato il primo algoritmo noto a chiave pubblica, utilizzato in particolare per il *key agreement* o *key exchange*, ovvero per concordare una chiave segreta tra due parti in modo interattivo. La sua sicurezza si basa sulla difficoltà computazionale di risolvere il **problema del logaritmo discreto**.

Il procedimento standard prevede i seguenti passaggi (cfr. Figura 4.5):

1. Le parti (Alice e Bob) scelgono o concordano due interi pubblici: un numero primo grande p (ad esempio, 1024 bit) e un generatore g (solitamente $2, 3$ o 5).
2. Alice sceglie un numero segreto x e calcola $X = g^x \mod p$.
3. Bob sceglie un numero segreto y e calcola $Y = g^y \mod p$.
4. Alice e Bob si scambiano i valori pubblici X e Y.
5. Alice calcola la chiave $K_{Alice} = Y^x \mod p$.
6. Bob calcola la chiave $K_{Bob} = X^y \mod p$.

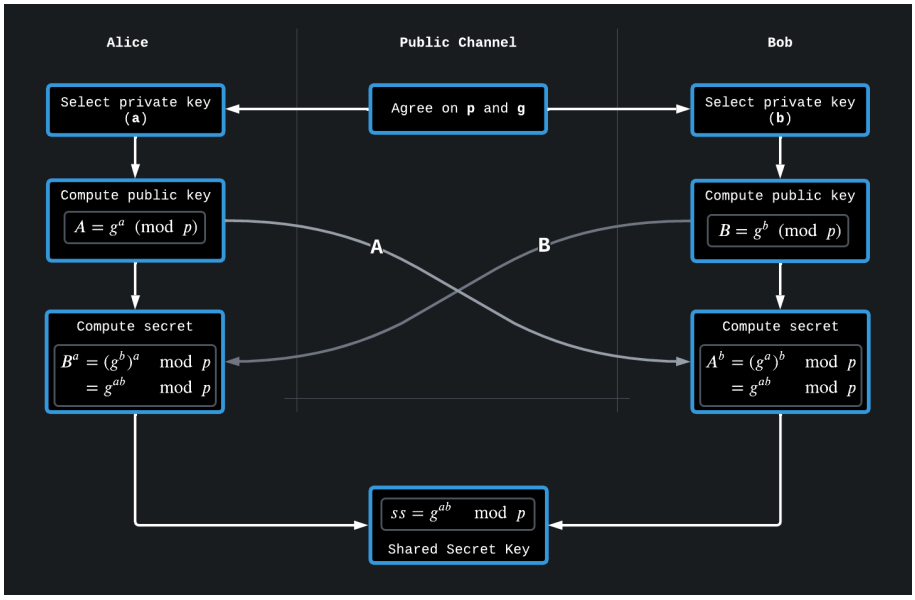


Figura 4.5: Scambio di chiavi con Diffie-Hellman.

Il risultato è una chiave segreta condivisa $K_{Alice} = K_{Bob} = g^{xy} \mod p$. L'attaccante che intercetta solo X e Y (sniffing) non può risalire a x e y in un tempo computazionalmente ragionevole.

Infatti, il fondamento di sicurezza dell'algoritmo di Diffie-Hellman risiede proprio nel problema del logaritmo discreto, già accennato al Paragrafo 4.3.1 e qui ripreso. Tale problema consiste nel determinare l'esponente x a partire dal valore di $g^x \mod p$. A differenza del logaritmo "classico" in aritmetica reale, che si definisce su un dominio continuo e può essere calcolato con metodi analitici, il logaritmo discreto opera in un contesto discreto, ossia nel gruppo finito dei numeri interi modulo p. In questo dominio non esiste una funzione inversa efficiente dell'esponenziazione modulare: calcolare $g^x \mod p$ è un'operazione semplice e veloce (grazie ad algoritmi di esponenziazione rapida), ma trovare x conoscendo solo g e $g^x \mod p$ è un'operazione che, allo stato attuale della conoscenza, non può essere risolta in tempo polinomiale. Gli algoritmi noti per affrontare il problema del logaritmo discreto, come il metodo di **Pohlig-Hellman** o il **number field sieve**, hanno complessità sub-esponenziale e diventano impraticabili per numeri primi di grandi dimensioni. Per questo motivo si dice che la funzione esponenziale modulare è una funzione a senso unico (*one-way function*): facile da calcolare in una direzione, ma estremamente difficile da invertire.

Tuttavia, DH è **vulnerabile agli attacchi Man-in-the-Middle** se l'attaccante è in grado di manipolare i dati scambiati. In un attacco MITM, l'attaccante intercetta X e Y e li sostituisce con valori derivati da un proprio segreto z, riuscendo a negoziare una chiave segreta diversa con entrambe le parti e a decifrare e ricifrare tutte le comunicazioni. Per mitigare questo rischio, DH richiede la **pre-autenticazione** delle parti, spesso realizzata tramite l'uso di certificati digitali.

4.4.2 Autenticazione mediante Cifratura Asimmetrica

La crittografia a chiave pubblica, sebbene non adatta alla cifratura diretta di grandi quantità di dati per via dell'elevato carico computazionale, riveste un ruolo fondamentale nell'implementazione dell'autenticazione. Il meccanismo di base che permette l'autenticazione si fonda sulla **proprietà reciproca** delle chiavi asimmetriche: se un messaggio o un dato viene cifrato con la chiave privata (K_{priv}) del mittente, può essere decifrato correttamente solo con la chiave pubblica (K_{pub}) corrispondente. Questo successo nella decifratura da parte del destinatario dimostra inequivocabilmente che i dati sono stati generati dal possessore legittimo della chiave privata, confermando così l'identità del mittente.

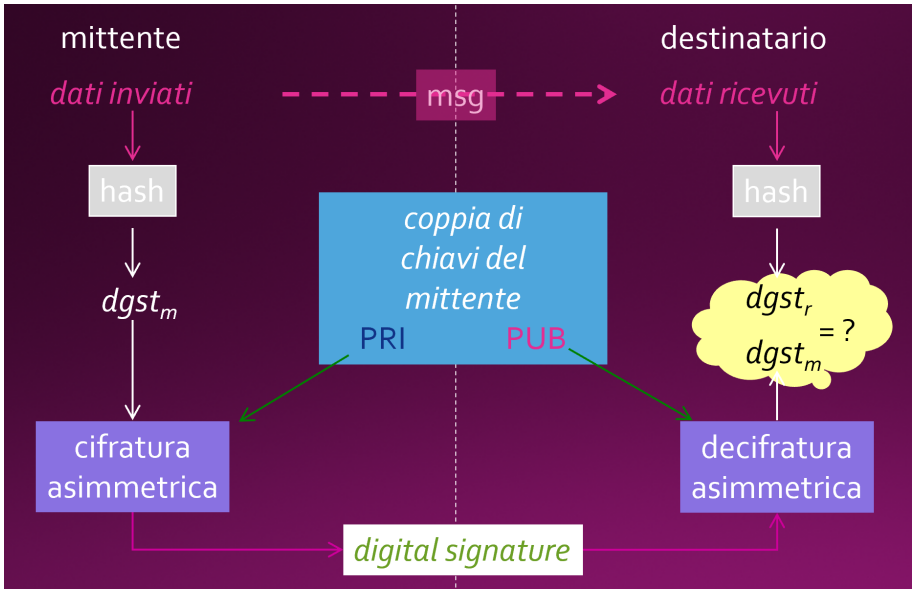


Figura 4.6: Schema di utilizzo della firma digitale.

La Firma Digitale (Digital Signature)

Poiché gli algoritmi asimmetrici (come RSA) sono lenti e non possono cifrare dati di dimensione superiore a quella del modulo n (tipicamente 2048 bit o più), l'operazione di autenticazione non viene mai applicata all'intero messaggio (M). La prassi standard, nota come **Firma Digitale** (propriamente detta), adotta un approccio ibrido che combina la velocità delle funzioni di hash con la sicurezza della crittografia asimmetrica.

La *Firma Digitale* è definita come la **cifratura del digest** (o hash) calcolato sui dati da inviare, utilizzando la chiave privata del firmatario. Questo digest, essendo un riassunto a lunghezza fissa dei dati originali, è piccolo e può essere cifrato rapidamente con un algoritmo asimmetrico. Nella pratica, per aumentarne la sicurezza, è necessario aggiungere anche un *padding* alla firma.

Questa tecnica è stata concepita per garantire due proprietà di sicurezza fondamentali: **l'Autenticazione** e l'**Integrità** dei dati e del mittente. È cruciale ricordare che la firma digitale **non garantisce la Riservatezza** (Confidenzialità), poiché il messaggio originale (M) transita in chiaro. Se si desidera anche la riservatezza, occorre applicare metodi di cifratura simmetrica in combinazione (come nell'approccio *enveloped data*). L'algoritmo principale utilizzato storicamente per la generazione di firme digitali è il **DSA** (Digital Signature Algorithm) e la sua variante basata sulle curve ellittiche, ECDSA (Elliptic Curve DSA).

Procedura di Generazione e Verifica della Firma

Il processo di generazione e verifica di una firma digitale è strutturato in passaggi precisi, riportati anche in Figura 4.6:

1. **Generazione della Firma (Mittente S)**: Il mittente calcola il digest, $H(M)$, del messaggio M utilizzando una funzione di hash crittografico sicuro. Successivamente, cifra questo digest con la sua chiave privata ($K_{S_{priv}}$) per ottenere la firma. Il mittente invia il messaggio in chiaro M insieme alla firma cifrata.
2. **Verifica (Destinatario)**: Il destinatario, che ha ricevuto i dati M, esegue due operazioni parallele. Per prima cosa, calcola il digest ($H(M)$) sui dati ricevuti (in figura dgst_r). In secondo luogo, utilizza la chiave pubblica del mittente ($K_{S_{pub}}$) per decifrare la firma ricevuta, estraendo così il digest originale generato dal mittente.

3. **Verifica**: Il destinatario confronta il digest calcolato localmente (sul messaggio ricevuto, dgst_r) con quello estratto dalla firma (calcolato dal mittente, dgst_m). Se i due valori coincidono, si ha la certezza che il messaggio è integro (non è stato modificato durante il transito) ed è autentico (solo il possessore di $K_{S_{priv}}$ avrebbe potuto cifrare correttamente il digest).

Firma Digitale vs. Keyed-Digest: Il Non Ripudio

La caratteristica più rilevante della firma digitale, che la distingue da altre tecniche di autenticazione come l'uso di Keyed-Digest (MAC simmetrico), è la garanzia del **Non Ripudio**.

Un Keyed-Digest si basa su un segreto condiviso (K) tra mittente e destinatario. Sebbene ciò garantisca integrità e autenticazione per le parti coinvolte e sia molto efficiente dal punto di vista computazionale, in caso di controversia legale non è possibile dimostrare chi tra i due abbia effettivamente generato il messaggio, poiché entrambi erano a conoscenza del segreto K.

Al contrario, la firma digitale, utilizzando la chiave privata che è **nota solo al firmatario**, fornisce una **prova formale** e innegabile della paternità del documento, anche in assenza di un segreto condiviso con il destinatario. Questo la rende uno strumento con **valore legale**. A differenza della firma autografa (manuale), che garantisce solo l'autenticazione, la firma digitale garantisce anche l'integrità del documento, poiché è indissolubilmente legata al digest dei dati.

L'affidabilità di questo sistema dipende criticamente dalla **protezione della chiave privata**. Se la chiave privata venisse compromessa, un attaccante potrebbe firmare documenti per conto del titolare, invalidando potenzialmente tutte le firme precedenti generate con quella chiave. Per prevenire ciò, le chiavi private sono tipicamente memorizzate in ambienti protetti, come smart card crittografiche o Hardware Security Module (HSM).

Per visualizzare la differenza tra i due concetti, si può pensare al **Keyed-Digest** come a due persone che utilizzano lo stesso timbro segreto per autenticare un messaggio in una comunicazione privata: entrambi sanno che è autentico, ma nessuno può dimostrare in seguito chi l'abbia generato. La **Firma Digitale**, invece, è come un sigillo personalizzato e unico, come i sigilli reali, creato con uno stampo (la chiave privata) che solo una persona possiede: chiunque veda il sigillo autenticato (la firma) sa esattamente chi l'ha applicato.

4.4.3 Crittografia a Curve Ellittiche (ECC)

La *Crittografia a Curve Ellittiche* (ECC, Elliptic Curve Cryptography) rappresenta un approccio alternativo alla crittografia a chiave pubblica che, invece di operare sull'aritmetica modulare degli interi, si basa sulla matematica delle **curve ellittiche**, strutture algebriche bidimensionali definite da equazioni del tipo $y^2 = x^3 + ax + b$.

Il problema crittografico fondamentale alla base di ECC è l'**Elliptic Curve Discrete Logarithm Problem (ECDLP)**: dati due punti P e Q su una curva ellittica, il punto base P e il punto $Q = kP$ (dove kP rappresenta l'addizione ripetuta del punto P con se stesso k volte), determinare il valore dell'intero k. Sebbene l'operazione sia formalmente un'addizione iterata di punti sulla curva, viene chiamata "logaritmo discreto" per analogia con il problema classico: così come nel logaritmo discreto tradizionale si cerca l'esponente x tale che $y \equiv g^x \mod p$ (dove l'esponenziazione è una moltiplicazione ripetuta), nell'ECDLP si cerca il moltiplicatore scalare k tale che $kP = Q$ (dove la moltiplicazione scalare è un'addizione ripetuta). Inoltre, è stato formalmente dimostrato che i due problemi sono computazionalmente equivalenti. In entrambi i casi, l'operazione diretta è computazionalmente efficiente, mentre l'inversione risulta intrattabile. La denominazione "logaritmo" riflette quindi la natura del problema matematico sottostante: trovare l'operando che, mediante un'operazione di gruppo iterata, produce un determinato risultato.

Questo problema si dimostra particolarmente difficile da risolvere: mentre il logaritmo discreto classico può essere attaccato con algoritmi sub-esponenziali avanzati come il *Number Field Sieve*, per l'ECDLP non sono note tecniche più efficienti degli algoritmi generici a complessità esponenziale, come il metodo Pollard's Rho adattato alle curve ellittiche. Questa maggiore resistenza

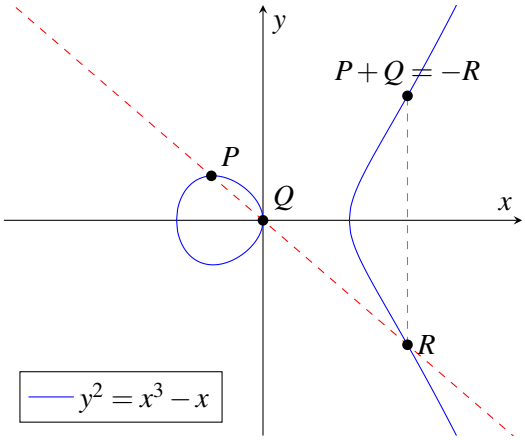


Figura 4.7: Addizione di punti su una curva ellittica: $P+Q$ si ottiene tracciando la retta passante per P e Q, trovando il terzo punto di intersezione R e rifettendolo rispetto all'asse x.

computazionale è dovuta all'assenza, nel gruppo additivo dei punti di una curva ellittica, delle proprietà aritmetiche e delle strutture algebriche che rendono possibili le ottimizzazioni utilizzate contro il logaritmo discreto tradizionale. Di conseguenza, l'ECDLP risulta significativamente più difficile da risolvere a parità di dimensione delle chiavi, consentendo a ECC di garantire un elevato livello di sicurezza con parametri crittografici molto più ridotti.

Vantaggi e Operazioni

Il vantaggio principale di ECC è che, a parità di livello di sicurezza, consente l'utilizzo di **chiavi molto più corte** (circa 1/10 della lunghezza) rispetto ai sistemi basati su RSA o DH tradizionale. Ad esempio, una chiave ECC da 163 bit offre una sicurezza paragonabile a quella di una chiave RSA da 2048 bit. Questo rende ECC particolarmente vantaggioso in termini di prestazioni e di utilizzo delle risorse computazionali. I numeri più piccoli richiedono meno risorse computazionali, il che è ideale per dispositivi con bassa potenza, come quelli dell'IoT.

Le operazioni su una curva ellittica, definita dall'equazione $y^2 \equiv (x^3 + ax + b) \mod p$, sono complesse. Ad esempio, l'addizione di due punti P e Q richiede passaggi geometrici per determinare il punto risultante $P+Q$. L'operazione cruciale in ambito crittografico è la **moltiplicazione scalare di un punto**. ECC è utilizzato in diverse applicazioni come le firme digitali (ECDSA), l'accordo di chiave (ECDH) e la distribuzione di chiavi (ECIES).

EC Diffie-Hellman (ECDH)

ECDH è la versione di Diffie-Hellman che sfrutta le curve ellittiche:

1. Alice e Bob scelgono una curva ellittica specifica, γ, e un punto base G (generatore) su di essa.
2. Alice sceglie un segreto x e calcola il punto $X = xG$.
3. Bob sceglie un segreto y e calcola il punto $Y = yG$.
4. Si scambiano i punti pubblici X e Y.
5. La chiave segreta condivisa K è calcolata come $K = xY = yX = xyG$.

Questo meccanismo di scambio di chiavi è più veloce per gli utilizzatori legittimi e più difficile da attaccare rispetto al DH classico, pur mantenendo un livello di sicurezza equivalente con chiavi più piccole.

Sicurezza e Vulnerabilità di Implementazione

La sicurezza di un sistema basato su ECC dipende criticamente dalla **scelta delle curve** matematiche utilizzate. Organismi come il NIST sono preposti a selezionare le curve ritenute sicure. Storicamen-

te, alcune curve proposte si sono rivelate deboli, sollevando dubbi sull'integrità di tali istituzioni, poiché curve non robuste rendono risolubili i problemi crittografici in tempo polinomiale.

È fondamentale anche l'implementazione corretta degli algoritmi e di tutte le specifiche crittografiche richieste. Ad esempio, la generazione di una firma digitale **ECDSA** richiede l'utilizzo di un **numero casuale (nonce) diverso** per ogni operazione di firma. Se questo nonce viene riutilizzato o risulta prevedibile, la chiave privata può essere calcolata a partire da due firme diverse generate con lo stesso nonce, compromettendo completamente la sicurezza del sistema.

Un caso emblematico di questa vulnerabilità si è verificato con la **PlayStation 3 di Sony**. La console incorporava Linux con un bootloader che verificava la firma ECDSA dei binari prima di eseguirli, impedendo l'esecuzione di software non autorizzato. Tuttavia, gli implementatori commisero un errore critico: utilizzarono lo stesso numero casuale (nonce) per tutte le operazioni di firma. Questo errore permise ad alcuni ricercatori di sicurezza di recuperare la chiave privata di Sony analizzando le firme pubblicamente disponibili. Una volta nota la chiave privata, chiunque poteva generare firme valide per qualsiasi programma, consentendo l'esecuzione di software arbitrario sulla console e vanificando completamente il sistema di protezione.

Questa vicenda evidenzia come il mancato rispetto delle specifiche crittografiche, anche quando l'algoritmo matematico è teoricamente sicuro, possa annullare completamente la sicurezza del sistema. La corretta implementazione è tanto cruciale quanto la solidità teorica dell'algoritmo scelto.

4.4.4 Key Derivation Function (KDF)

Le chiavi crittografiche, per garantire un livello di sicurezza accettabile, devono essere intrinsecamente **casuali** (random), ovvero ogni bit deve avere una probabilità del 50% di essere 0 o 1, al fine di renderle imprevedibili. Tuttavia, gli utenti sono spesso tenuti a utilizzare *password* o, preferibilmente, *passphrase*, che sono sequenze di caratteri facilmente memorizzabili, ma che risultano prevedibili o poco casuali, e quindi **non sono adatte** come chiavi crittografiche.

Per risolvere questa dicotomia tra la necessità umana di memorizzare sequenze semplici di caratteri e il requisito crittografico di avere chiavi ad alta entropia, sono state sviluppate le **KDF**. Una KDF è un algoritmo che prende in input una password prevedibile e genera in output una o più chiavi crittografiche che appaiono come sequenze di bit casuali. Le KDF sono tipicamente **basate su funzioni di hash crittografico**.

La struttura generale di una KDF è definita dalla seguente formula:

$$K = KDF(P, S, I)$$

Dove:

- K rappresenta la chiave o l'insieme delle chiavi crittografiche generate.
- P è la **Password** o passphrase fornita dall'utente.
- S è il **Sale** (*salt*), un numero casuale pubblico che viene utilizzato per rendere K imprevedibile dato P. Il sale deve essere **distinto e cambiato ogni volta** per garantire la "freschezza" della chiave derivata. Il suo uso, combinato con la password tramite operazioni come lo XOR, mira a rendere il dato risultante di **distribuzione piatta**, simile a un dato veramente random.
- I è il **Numero di Iterazioni**, ovvero il numero di volte in cui la funzione base viene applicata.

Per comprendere più concretamente il funzionamento di una KDF, si consideri il seguente esempio. Supponiamo che un utente inserisca la password:

$P =$ "password123".

Prima della derivazione della chiave, il sistema genera un *salt* casuale, il cui scopo è impedire che due password identiche portino alla generazione della stessa chiave derivata. Assumiamo un salt di

128 bit, ad esempio:

$S =$ `3f a9 1c 72 0b e4 55 9d 88 22 17 6c 00 fa 91 3c`.

La funzione di derivazione adottata è PBKDF2, standardizzata nella RFC 8018. PBKDF2 (*Password-Based Key Derivation Function 2*) combina la password, il salt e un elevato numero di iterazioni mediante una funzione crittografica interna (tipicamente HMAC-SHA256[9]) incrementando deliberatamente il costo computazionale dell'operazione al fine di contrastare attacchi a forza bruta. Nel nostro esempio si utilizza PBKDF2-HMAC-SHA256 con:

$I = 100\,000$ iterazioni.

Il procedimento concreto della derivazione si può riassumere come segue:

1. La password viene combinata con il salt tramite concatenazione:

   $$P \parallel S.$$

2. Si calcola HMAC-SHA256 della concatenazione, ottenendo un primo blocco pseudo-casuale.
3. L'HMAC viene ripetuto per un totale di I cicli, utilizzando come input di ogni iterazione l'output di quella precedente:

   $$K = \mathrm{HMAC}(P,\ \mathrm{HMAC}(P,\ \ldots\ \mathrm{HMAC}(P,\ P \parallel S))).$$

4. Il valore risultante dall'ultima iterazione costituisce la chiave derivata K.

Il valore finale, espresso in forma esadecimale, può essere ad esempio:

$K =$ `5b 37 91 f3 88 14 1c 05 9d 20 6a 41 cc fa 7e 9c`
`48 91 02 76 ab 33 89 1a 7e c9 f6 12 aa 99 3d 5b`

Tale chiave dipende fortemente sia dalla password sia dal salt, e viene completamente modificata nel caso in cui il salt venga rigenerato. L'impiego congiunto di un salt sufficientemente lungo e di un numero considerevole di iterazioni rende la funzione di derivazione resistente a tentativi di indovinare la password tramite attacchi a dizionario o forza bruta.

Meccanismi di Rallentamento e Mitigazione degli Attacchi

Il parametro relativo alle iterazioni (I) è fondamentale per la sicurezza. L'obiettivo primario delle KDF è quello di introdurre una lentezza deliberata nel calcolo della chiave. Per un utente legittimo che digita la password manualmente, attendere poche centinaia di millisecondi o anche un secondo per la derivazione della chiave è un ritardo tollerabile.

Al contrario, per un attaccante che tenta un attacco *brute force* o a dizionario, questa lentezza, moltiplicata per migliaia o decine di migliaia di iterazioni e per le potenziali password da testare, comporta un enorme dispendio di tempo. Ad esempio, aumentare il tempo di calcolo di 10.000 volte può trasformare un attacco che richiederebbe un giorno in uno che richiederebbe più di 27 anni.

Inizialmente, l'uso di un elevato numero di iterazioni (nell'ordine delle migliaia o decine di migliaia) era sufficiente per scoraggiare gli attacchi. Tuttavia, con l'evoluzione tecnologica, gli attaccanti hanno iniziato a sfruttare in modo massivo le Graphics Processing Unit (GPU) e i circuiti Application-Specific Integrated Circuit (ASIC). Grazie alla loro elevata capacità di parallelizzazione, tali dispositivi permettono di testare simultaneamente milioni di password, aumentando drasticamente la velocità degli attacchi rispetto all'utilizzo esclusivo della CPU. In

[9] Questa nomenclatura indica l'uso di HMAC con SHA256 come funzione hash sottostante.

questo modo, l'effetto di rallentamento basato unicamente sul numero di iterazioni della CPU viene parzialmente vanificato.

Per affrontare questa sfida, le nuove generazioni di KDF hanno introdotto un ulteriore requisito di sicurezza: la saturazione intenzionale della memoria RAM (almeno quella delle GPU e dei circuiti ASIC). Questi algoritmi, detti *memory-hard*, sono progettati per lavorare su grandi quantità di dati in memoria (anche diversi gigabyte) prima di completare l'operazione di derivazione. L'idea alla base è che la KDF non si limiti a eseguire molte iterazioni di una funzione crittografica, ma costruisca e manipoli una grande struttura di memoria, tipicamente una matrice o un vettore di blocchi pseudocasuali, che vanno tutti generati e memorizzati prima del calcolo degli hash. Ogni blocco dipende dai blocchi precedenti in modo non lineare e non predicibile, e la funzione richiede di accedere ripetutamente alla struttura in punti determinati pseudo-casualmente durante l'esecuzione. In altre parole, la KDF obbliga il dispositivo ad avere:

- **molta RAM disponibile**, per poter costruire la struttura dati di grandi dimensioni;
- **accesso ripetuto e irregolare** a quella RAM, impedendo ottimizzazioni come il caching;
- **latenza elevata negli accessi**, che diventa parte integrante del tempo totale della derivazione.

Sfruttando la limitata memoria disponibile sulle GPU rispetto alla RAM di sistema, anche se un attaccante dispone di numerosi core, solo pochi thread possono operare contemporaneamente. La saturazione della RAM riduce quindi drasticamente l'efficacia della parallelizzazione e ripristina il fattore di lentezza previsto dalle KDF tradizionali. In questo modo, mentre le KDF tradizionali rallentano il calcolo aumentando il numero di iterazioni, le KDF memory-hard rallentano gli attacchi aumentando i requisiti di memoria e imponendo un modello di accesso inefficiente per hardware massivamente parallelo, rendendo impraticabile l'uso simultaneo di migliaia di core per testare password.

Algoritmi Standard di Key Derivation

Tra gli algoritmi KDF più noti si ricordano:

- **Password-Based Key Derivation Function 2 (PBKDF2)**: Storicamente basato su SHA-1, prevedeva l'uso di un sale (S) di almeno 64 bit e un numero di iterazioni (I) superiore a 1.000. È stato standardizzato come parte di PKCS#5 v2.1 (RFC-8018) e ha sostituito PBKDF1, che limitava la generazione di chiavi a un massimo di 160 bit. PBKDF2 è oggi considerato superato in contesti ad alta sicurezza a causa della sua vulnerabilità agli attacchi basati su GPU, non richiedendo un grande consumo di memoria.
- **HKDF**: Basato sull'algoritmo HMAC (Keyed-Digest).
- **Argon2**: È stato selezionato come KDF vincitore di una competizione pubblica (la Password Hashing Competition) avviata nel 2013 e conclusasi nel 2015. Argon2 incarna la filosofia moderna delle KDF, combinando il lavoro computazionale con la **saturazione della RAM** al fine di resistere efficacemente agli attacchi basati su hardware parallelo.

Per comprendere l'importanza delle KDF, si può pensare a un **piatto segreto** (la chiave crittografica) che deve essere prodotta a partire da ingredienti comuni (la password). Sebbene chiunque possa conoscere gli ingredienti di base, la KDF agisce come una ricetta che **richiede una quantità enorme di tempo e risorse specifiche** (le iterazioni e la RAM satura) per trasformare gli ingredienti comuni nel prodotto finale. Questo garantisce che solo il cuoco legittimo (l'utente) possa produrre la ricetta in un tempo ragionevole, mentre un concorrente (l'attaccante), anche se dispone di molti assistenti (le GPU), deve provare tante combinazioni di ingredienti, ma viene paralizzato dalla scarsità di spazio di lavoro (la RAM limitata).

4.4.5 Crittografia Post-Quantum PQC

La PQC (Post-Quantum Cryptography) è l'area di studio che si occupa di sviluppare e standardizzare algoritmi crittografici resistenti agli attacchi che potrebbero essere lanciati da un computer quantistico sufficientemente potente.

La Minaccia Quantistica e l'Algoritmo di Shor

Gli attuali sistemi crittografici a chiave pubblica (asimmetrici), come RSA, DSA (Digital Signature Algorithm) e le varianti basate sulle Curve Ellittiche (ECC), ad esempio ECDH ed ECDSA, fondano la loro sicurezza sulla difficoltà computazionale di risolvere problemi matematici ben definiti: per RSA si tratta del problema della fattorizzazione di numeri interi, mentre per DH e ECC la base è il problema del logaritmo discreto. Questi problemi sono ritenuti "difficili" per i computer classici perché gli algoritmi più efficienti conosciuti richiedono un tempo che cresce in modo superpolinomiale (o esponenziale) in funzione della dimensione della chiave.

Tuttavia, avere a disposizione un computer quantistico introduce una seria minaccia a questo paradigma: l'algoritmo di Shor, teorizzato da Peter Shor nel 1994, consente di risolvere sia la fattorizzazione di grandi numeri sia il calcolo del logaritmo discreto in tempo polinomiale su un computer quantistico. Oggi, però, non esistono computer quantistici sufficientemente avanzati da eseguire l'algoritmo di Shor[10] su numeri abbastanza grandi da minacciare la crittografia moderna con chiavi reali. I prototipi esistenti sono ancora limitati dal numero di qubit, dalla coerenza quantistica e dalla necessità di correzione degli errori. Tuttavia, la comunità crittografica considera sempre più imminente la necessità di migrare verso soluzioni post-quantistiche, poiché un futuro computer quantistico in grado di attaccare la crittografia potrebbe compromettere la riservatezza delle comunicazioni registrate oggi e l'integrità delle firme digitali.

Urgenza della Transizione e "Collect Now, Decrypt Later"

La transizione verso la PQC è considerata urgente a causa del rischio di **data harvesting** (o "raccogli adesso, decifra poi"). Anche se un attaccante non può decifrare oggi i dati cifrati con RSA o DH, può intercettarli e salvarli, con l'intento di decifrarli in futuro, quando i computer quantistici saranno disponibili. Tutti i dati che devono essere tenuti segreti per più di 10-15 anni sono a rischio di decifrazione non appena arriverà il primo computer quantistico sufficientemente potente.

È quindi necessario, con urgenza, individuare una nuova classe di problemi matematici la cui risoluzione non sia possibile in tempo polinomiale nemmeno con un computer quantistico.

Approcci e Standardizzazione NIST

Per identificare questi nuovi algoritmi resistenti, il NIST statunitense ha lanciato competizioni pubbliche per selezionare i futuri standard di PQC. Inizialmente sono stati proposti sei approcci principali basati su diverse sfide matematiche:

- **Lattice-based** (Basati sui reticoli).
- **Multivariate** (Equazioni multivariate).
- **Hash-based** (Basati sulle funzioni di hash).
- **Code-based** (Basati sulla teoria dei codici).
- **Isogeny-based** (Basati sulle isogenie).
- **Symmetric key quantum resistance**.

Questi problemi sono ritenuti molto complessi e concettualmente equivalenti, e possono essere usati per costruire sistemi crittografici in cui un attaccante deve risolverli per ottenere un vantaggio (es. trovare la chiave privata a partire dalla chiave pubblica), resistendo al calcolo quantistico. Va notato che la ricerca è ancora in una fase volatile: alcune famiglie di algoritmi, come quelle basate sui codici, sono state ritenute promettenti, ma poi sono state rese inutilizzabili in seguito alla scoperta di nuovi teoremi matematici.

[10] Per i curiosi: L'algoritmo sfrutta le proprietà della sovrapposizione e dell'interferenza quantistiche per trovare in modo efficiente il periodo di una funzione modulare associata al numero da fattorizzare o al logaritmo discreto da calcolare. In termini pratici, Shor riduce il problema della fattorizzazione a quello della ricerca di un periodo, che può essere risolto rapidamente con un quantum Fourier transform, operazione eseguibile in tempo polinomiale se in possesso di abbastanza qubit (1783 qubit effettivi secondo l'articolo qui `https://doi.org/10.1007/978-3-032-01878-6_13`, meno di un milione di *noisy qubit* secondo questo `https://arxiv.org/abs/2505.15917`)

Tra gli algoritmi selezionati (o finalisti) per la standardizzazione dal NIST, si ricordano:

- **CRYSTALS-Kyber**: un algoritmo *lattice-based* destinato alla cifratura e allo scambio di chiavi (key exchange).
- **CRYSTALS-Dilithium**: un algoritmo *lattice-based* destinato alle firme digitali.
- **Sphincs+**: un algoritmo *hash-based*, selezionato come backup nel caso in cui venissero scoperte debolezze negli algoritmi basati sui reticoli (lattice-based).

Nonostante queste selezioni, il NIST ha già lanciato una nuova call per proposte algoritmiche (a partire dal 2023), sottolineando la persistente incertezza sull'effettiva resistenza dei candidati attuali a futuri attacchi quantistici.

Impatto sulla Crittografia Simmetrica e Hash

A differenza degli algoritmi asimmetrici, la **crittografia simmetrica** e le **funzioni di hash crittografico** non sembrano essere influenzate in modo significativo dalla tecnologia quantistica.

L'algoritmo quantistico più noto contro questa famiglia è l'**Algoritmo di Grover**, che, in teoria, può dimezzare la velocità di un attacco brute force contro gli algoritmi simmetrici a blocchi.

Questa minaccia comporta che la **sicurezza effettiva** di una chiave venga dimezzata. Ad esempio, una chiave AES da 256 bit risulterebbe avere, contro un attacco quantistico basato sull'algoritmo di Grover, una forza equivalente a una chiave classica da 128 bit.

La mitigazione per la crittografia simmetrica è relativamente semplice: è sufficiente aumentare la dimensione delle chiavi utilizzate (ad esempio, passare da chiavi con una forza di 128 bit a chiavi con una forza di 256 bit o superiore) per mantenere un livello di sicurezza accettabile. Pertanto, la tecnologia sottostante per gli algoritmi simmetrici e di hash può essere riutilizzata, eliminando la necessità di reinventare completamente nuovi problemi matematici per queste classi di algoritmi.

4.5 Certificati Digitali

Un sistema basato sulla crittografia asimmetrica si fonda su due presupposti fondamentali:

- la chiave privata deve rimanere segreta e non essere mai condivisa;
- la chiave pubblica deve essere resa disponibile al maggior numero possibile di soggetti.

La sola presenza di una chiave pubblica, però, non consente di verificare l'identità di chi la utilizza: chiunque può generare una coppia di chiavi senza alcuna garanzia sulla propria identità. Per risolvere questo problema esistono diversi metodi.

Una prima soluzione consiste nello scambio diretto delle chiavi, al di fuori del canale di comunicazione principale, ad esempio, incontrandosi di persona. In questo modo l'identità del proprietario può essere verificata tramite procedure del mondo reale, riducendo il rischio di impersonificazione. Tuttavia, questa strategia non è scalabile e risulta poco praticabile in contesti di rete ampi e dinamici.

Un metodo molto più efficiente è quello dei certificati digitali. Un **certificato digitale** (o **certificato a chiave pubblica**) è un documento elettronico usato per attestare l'identità di una persona, un'organizzazione o un dispositivo su una rete, come Internet. Può essere paragonato a un documento d'identità fisico: il suo compito è associare in modo affidabile una chiave pubblica a un soggetto.

Un certificato digitale è quindi una struttura dati che collega in modo sicuro una chiave pubblica ad attributi identificativi. Oltre al nome del titolare, questi attributi possono includere l'indirizzo IP, l'indirizzo email o i record DNS. Come nei documenti d'identità tradizionali, i certificati riportano la firma digitale dell'emittente, chiamata **autorità di certificazione** (**CA — Certification Authority**), e hanno una validità limitata nel tempo, generalmente di un anno per i certificati acquistati. Inoltre, possono essere revocati sia dal titolare sia dall'autorità che li ha emessi.

4.5.1 Formati per certificati a chiave pubblica

Il formato più diffuso per i certificati digitali è lo standard internazionale **X.509**. La sua versione più recente e più comunemente adottata, la versione 3, è composta da tre parti principali: il corpo del certificato, l'identificatore dell'algoritmo di firma e la firma applicata dall'autorità di certificazione. Il corpo del certificato raccoglie un insieme strutturato di informazioni, tra cui la versione, il numero di serie, l'algoritmo di firma, l'autorità emittente, il soggetto a cui il certificato è rilasciato, il periodo di validità, i dati relativi alla chiave pubblica del soggetto e una serie di estensioni opzionali. La sezione dedicata alla chiave pubblica specifica l'algoritmo crittografico utilizzato e la chiave stessa; la validità del certificato è definita da una data di inizio e una data di scadenza, che ne determinano la validità temporale.

Accanto allo standard X.509 esistono altre soluzioni, seppure molto meno utilizzate. Tra queste, **Pretty Good Privacy (PGP)**, che adotta un modello di fiducia decentrato nel quale più utenti possono firmare una stessa chiave; **Simple Public-Key Infrastructure (SPKI)**, basato su un modello di delega della fiducia in cui i certificati contengono essenzialmente una chiave pubblica e specifiche autorizzazioni limitate; e **PKCS#6**, storicamente impiegato per la rappresentazione e il trasporto di certificati estesi, oggi considerato deprecato e sostanzialmente rimpiazzato dallo standard X.509.

4.5.2 Public-Key Infrastructure

Una **PKI** è l'infrastruttura composta da tecnologie, processi, persone e regole che consente di gestire in modo sicuro l'intero ciclo di vita dei certificati digitali: dalla loro generazione, alla distribuzione, fino alla loro eventuale revoca.

La verifica di una firma digitale può essere riassunta nei seguenti passaggi:

1. si calcola l'impronta (digest) del messaggio utilizzando l'algoritmo indicato nella firma;
2. si ricava la chiave pubblica del firmatario dal certificato digitale associato;
3. si utilizza tale chiave pubblica per decifrare il valore della firma, ottenendo il digest originale prodotto dal firmatario;
4. si confrontano i due digest: se coincidono, la firma è valida e sono garantite integrità e autenticità del messaggio; in caso contrario, la firma risulta non valida, indicando una possibile alterazione o falsificazione.

Per recuperare correttamente la chiave pubblica del firmatario, è necessario verificare la validità del suo certificato digitale, ossia la firma rilasciata dall'autorità di certificazione (CA). A sua volta, il certificato della CA deve essere verificato tramite la firma della CA di livello superiore, e così via. Si rende quindi necessario un sistema gerarchico di fiducia che culmina nelle **Root CA** (o **autorità di certificazione radice**), i cui certificati sono auto-firmati e considerati affidabili per convenzione.

Un certificato può essere revocato prima della naturale scadenza, su richiesta del titolare o per decisione dell'autorità che lo ha emesso. Durante la verifica di una firma, spetta al destinatario verificare che il certificato fosse valido al momento della sottoscrizione. A tal fine esistono due principali meccanismi:

- **Certificate Revocation List (CRL)**: elenco dei certificati revocati prima della loro naturale scadenza, firmato digitalmente dalla CA. I certificati sono identificati dal *serial number* (ID univoco all'interno di una CA). Ogni voce nell'elenco include la data e il motivo della revoca, che consente di verificare lo stato di un certificato nel passato. Un certificato può essere revocato per diversi motivi, tra cui la compromissione o la perdita della chiave privata, errori nella sua emissione o la cessazione del rapporto con il soggetto titolare. Le CRL vengono pubblicate periodicamente dalla CA e rese disponibili tramite uno o più punti di distribuzione (CRL Distribution Points), generalmente accessibili tramite HTTP. Un client che deve verificare un certificato scarica l'intera lista e controlla se il numero di serie del certificato in esame è presente. Le principali caratteristiche delle CRL sono:

  - distribuzione periodica e non in tempo reale, che comporta il rischio che le ultime informazioni sulle revoche non siano state ancora pubblicate(es. prendiamo per buono qualche certificato già revocato solo perché non è ancora pubblica tale informazione);
  - dimensione potenzialmente elevata, soprattutto per CA con molti certificati revocati (raro nella pratica);
  - dopo averle scaricate, sono consultabili anche *off line*, senza necessità di connessione ad Internet.
- **Online Certificate Status Protocol (OCSP)**: protocollo che consente di ottenere, in tempo quasi reale, lo stato di un singolo certificato tramite una risposta firmata da un server dedicato. Con l'OCSP, il client invia una richiesta a un *OCSP Responder*, specifico di una CA, indicando il serial number del certificato di cui vuole conoscere lo stato. Il responder restituisce una risposta firmata che indica se il certificato è valido, revocato o in uno stato sconosciuto. A differenza delle CRL, OCSP non richiede il download di una lista completa, ma fornisce una risposta puntuale per ciascun certificato. Le principali caratteristiche del protocollo OCSP sono:
  - verifica puntuale dello stato del certificato;
  - riduzione del traffico di rete rispetto alle CRL;
  - dipendenza dalla disponibilità del responder OCSP.

4.6 Prestazioni ed acceleratori

Le operazioni crittografiche, pur essendo essenziali per garantire sicurezza, autenticità e integrità nelle comunicazioni digitali, comportano costi computazionali che variano sensibilmente in base agli algoritmi utilizzati e al contesto in cui vengono eseguite. Le prestazioni dipendono principalmente dalla potenza di calcolo del processore, in particolare dall'architettura, dall'insieme di istruzioni supportate, dalle dimensioni e dalla velocità della cache, mentre la memoria RAM ha un impatto marginale. Nella maggior parte dei casi, le prestazioni crittografiche non rappresentano un limite significativo sui sistemi client moderni; diventano invece critiche in ambienti server, nei nodi di rete e nei dispositivi embedded, dove vanno gestite un elevato numero di operazioni crittografiche o dove le risorse hardware sono fortemente limitate. Per far fronte a tali esigenze, vengono impiegati acceleratori crittografici, ovvero componenti hardware specializzati progettati per eseguire in modo più efficiente operazioni quali la cifratura, la decifratura e la firma digitale, migliorando significativamente le prestazioni e la scalabilità dei sistemi che li utilizzano.

4.6.1 Personal Security Environment

Il **Personal Security Environment (PSE)** è l'ambiente personale di sicurezza in cui vengono memorizzati e gestiti i dati crittografici sensibili di un utente, come le chiavi private e i certificati digitali. Il suo scopo è proteggere queste informazioni da accessi non autorizzati e garantire che le operazioni che le utilizzano avvengano in un contesto sicuro. Il PSE può essere implementato sia in software, ad esempio tramite file crittografati nel sistema operativo, sia in hardware, come smart card, token USB o moduli sicuri integrati nei dispositivi.

Nel caso dei PSE hardware, si distinguono due categorie: quelli **passivi**, che si limitano a custodire le chiavi private in modo protetto, rendendole disponibili al sistema per le operazioni crittografiche; e quelli **attivi**, che, oltre a conservare le chiavi, eseguono internamente le operazioni crittografiche, impedendo che la chiave privata venga mai esposta all'esterno. Quest'ultima soluzione offre un livello di sicurezza superiore, in quanto riduce drasticamente il rischio di estrazione o compromissione delle chiavi e accelera le operazioni, se vengono implementate le ottimizzazioni adeguate.

4.6.2 Smart card

Una **smart card crittografica** è un dispositivo hardware sicuro, di dimensioni paragonabili a una carta di credito, progettato per generare, conservare e utilizzare in modo protetto chiavi crittografiche e certificati digitali. Al suo interno è presente un microprocessore dotato di memoria protetta e, nei modelli più avanzati, di un coprocessore crittografico dedicato, che consente di eseguire operazioni come firma digitale, autenticazione e cifratura direttamente sulla carta, senza mai esportare la chiave privata all'esterno.

La smart card rappresenta un ambiente di sicurezza attivo: le chiavi vengono generate e custodite internamente e tutte le operazioni crittografiche sensibili vengono eseguite al suo interno. L'accesso alle funzionalità della carta è generalmente protetto da un PIN o da meccanismi analoghi, così da prevenire l'uso non autorizzato anche nel caso di furto o smarrimento fisico del dispositivo. Grazie a queste caratteristiche, le smart card crittografiche trovano largo impiego in contesti che richiedono elevati livelli di sicurezza, come sistemi di autenticazione forte, firma digitale qualificata, controllo degli accessi, identità elettronica e servizi e-government.

4.6.3 Hardware Security Module

Un **HSM** è un acceleratore crittografico progettato per generare, proteggere e gestire chiavi crittografiche e per eseguire operazioni di sicurezza ad alta affidabilità, come la cifratura, la decifratura e la firma digitale. Un HSM è tipicamente installato in data center, server aziendali o ambienti cloud e offre capacità di calcolo elevate, scalabilità e funzioni avanzate di protezione fisica e logica.

Gli HSM sono dotati di meccanismi anti-manomissione e di ambienti sicuri isolati dal sistema operativo ospitante, in modo che le chiavi private non vengano mai esposte o memorizzate in chiaro al di fuori del modulo. Questi sistemi possono assumere diversi fattori di forma a seconda del contesto d'impiego: schede **PCI/PCIe** installate direttamente nei server, dispositivi **esterni** collegati tramite USB o interfacce dedicate, e **netHSM**, unità autonome collegate in rete e progettate per un utilizzo centralizzato e ad alta disponibilità in ambienti enterprise o cloud.

Le chiavi possono essere generate e utilizzate esclusivamente all'interno dell'HSM, riducendo drasticamente il rischio di attacchi e compromissioni. Questi dispositivi sono fondamentali nelle infrastrutture critiche, nei sistemi bancari e di pagamento, nei servizi cloud e in tutte le applicazioni che richiedono elevati standard di sicurezza, continuità operativa e gestione centralizzata delle chiavi.

4.7 Documenti elettronici

Nel mondo della sicurezza informatica è bene fare una distinzione: un **documento elettronico** è una qualunque rappresentazione elettronica di un documento, gestita tramite strumenti informatici, ad esempio una versione digitalizzata di un documento analogico; un **documento digitale** è un documento nativamente digitale, creato fin dall'origine tramite sistemi informatici, che può contenere metadati strutturati che ne definiscono contenuto, contesto e formato.

Un documento elettronico o digitale deve soddisfare alcune fondamentali proprietà di sicurezza:

- *integrità*: il contenuto non deve poter essere modificato senza che la manomissione venga rilevata;
- autenticazione dell'autore: deve essere possibile identificare in modo certo il soggetto che ha prodotto o firmato il documento;
- *usabilità*: il formato scelto deve permettere sia la lettura del contenuto sia la verifica delle eventuali firme digitali, garantendo interoperabilità e accessibilità;
- *archiviazione a lungo termine*: il documento deve rimanere consultabile e verificabile anche dopo molti anni, mantenendo i requisiti di integrità, autenticità e riservatezza nel tempo.

4.7.1 Formati dei documenti firmati

I formati dei documenti firmati possono essere classificati in:

- **enveloping signature**: il documento originale è contenuto all'interno di una struttura dati che costituisce il documento firmato. La firma "avvolge" il contenuto. Per leggere questo tipo di formato è necessario utilizzare un software specializzato in grado di verificare la firma digitale. Un esempio è **CMS Advanced Electronic Signatures (CAdES)**, che si basa sullo standard crittografico CMS/PKCS7 ed è adatto a firmare qualsiasi tipo di file, come documenti PDF, immagini o archivi. Il formato più comune presenta estensione .p7m;
- **enveloped signature**: la firma è incorporata nel documento stesso, che quindi "contiene" la firma. In questo caso, non è necessario utilizzare un software specializzato per leggere i dati. Degli esempi sono **XML Advanced Electronic Signatures (XAdES)**, uno standard pensato per documenti in formato XML, e **PDF Advanced Electronic Signatures (PAdES)**, che è lo standard specifico per i documenti PDF;
- **detached signature**: la firma è separata dal documento. Firma e documento costituiscono file separati. La firma fa riferimento al documento tramite un hash. I file possono essere contenuti in un archivio **Associated Signature Container (ASiC)**, basato su ZIP.

4.7.2 Firme multiple

In molti contesti, come nel mondo reale, può essere necessario che più soggetti appongano una firma digitale sullo stesso documento. Esistono due modelli principali per la gestione delle firme multiple.

Il primo approccio consiste nel far firmare a ciascun attore il documento originale in modo indipendente dagli altri. Questo metodo consente di eseguire le operazioni di firma in parallelo, riducendo i tempi. Tuttavia, presenta un limite: le firme risultano scollegate tra loro, quindi un'eventuale firma potrebbe essere rimossa o sostituita senza che gli altri firmatari possano rilevarlo. Inoltre, non si presta bene ai casi in cui l'ordine di firma deve essere fisso e ogni firmatario deve anche garantire che ci siano le firme dei responsabili precedenti (es. workflow aziendali).

Il secondo approccio prevede che ogni firmatario sottoscriva non solo il documento, ma anche le firme già presenti. In questo modo si crea una catena di firme dipendenti, in cui ciascuna firma "protegge" le precedenti. Questo metodo garantisce maggiore sicurezza e impedisce eventuali manomissioni, ma richiede un processo sequenziale e quindi non consente la parallelizzazione.

4.7.3 Timestamping

La semplice firma digitale attesta chi ha firmato un documento e ne garantisce l'integrità, ma non dimostra quando il documento è stato firmato. Questo può essere un problema in molti casi legali e amministrativi, in cui è necessario provare che un'operazione sia avvenuta entro un certo termine (es. scadenze, contratti, atti ufficiali). Il timestamping colma questa lacuna, creando una prova temporale opponibile a terzi.

Il **timestamping** (marcatura temporale) è un meccanismo che permette di associare a un documento digitale una data e un'ora certe e verificabili, garantite da un'entità fidata. Serve a dimostrare che un determinato contenuto esisteva in un preciso momento nel tempo e che non è stato modificato successivamente.

Il **Time-Stamp Protocol (TSP)** si articola nelle seguenti fasi:

1. si calcola l'hash del documento;
2. l'hash viene inviato a un'autorità di marcatura temporale (**Time Stamping Authority (TSA)**);
3. la TSA genera un **Time-Stamp Token (TST)**, che contiene l'hash ricevuto, la data e l'ora certi e la propria firma digitale su questi dati;
4. il TST viene restituito all'utente e allegato al documento.

In questo modo, chiunque in futuro potrà verificare non solo che il documento è rimasto integro, ma anche che esisteva già al momento indicato e che la data non può essere stata falsificata. È un meccanismo fondamentale, ad esempio, per i documenti legali, la conservazione digitale a lungo termine, la fatturazione elettronica e la certificazione di log o transazioni.

4.7.4 Qualified Certificate

I **Qualified Certificate (QC)**, ovvero certificati qualificati, sono una particolare categoria di certificati digitali a chiave pubblica che rappresentano il livello più alto di affidabilità legale nell'identificazione elettronica e nella firma digitale.

Un certificato qualificato contiene dati che collegano in modo univoco una persona fisica (o, in alcuni casi, giuridica) alla sua chiave pubblica, ma, a differenza dei certificati standard, può essere rilasciato solo da un **Qualified Trust Service Provider (QTSP)** accreditato e sottoposto a controlli e certificazioni ufficiali. Inoltre, il dispositivo che custodisce la chiave privata associata, come una smart card, un token USB o un modulo HSM certificato, deve essere un **Qualified Signature Creation Device (QSCD)**, cioè un dispositivo protetto che impedisce l'estrazione o l'uso improprio della chiave privata.

Grazie a questi requisiti rigorosi, i QC garantiscono un elevato livello di certezza sull'identità del titolare e sul corretto uso delle chiavi. In ambito giuridico, una firma elettronica apposta tramite un certificato qualificato è equiparata alla firma autografa su documento cartaceo, ed è pienamente valida e opponibile davanti alla legge. In sostanza, forniscono un meccanismo sicuro e riconosciuto per autenticare persone e documenti in contesti pubblici, aziendali e amministrativi.

4.7.5 Il problema delle "macro"

Molti formati di documenti elettronici consentono di incorporare al loro interno **macro**, cioè piccoli programmi in linguaggi di scripting (come VBA, Visual Basic for Applications) capaci di eseguire operazioni automatiche.

Se da un lato le macro sono utili per automatizzare procedure ripetitive, generare report o interagire con le basi dati, dall'altro rappresentano un potenziale rischio per la sicurezza. Infatti, una macro può alterare il contenuto di un documento, compromettendone l'integrità, oppure eseguire comportamenti malevoli. Chi legge il documento pensando che il contenuto sia (semanticamente) esattamente lo stesso di quello firmato data la validità della firma, potrebbe essere ingannato. Ad esempio, la macro `date()` potrebbe ingannare sulla data di creazione del documento, inducendo il lettore a credere che sia stato creato nello stesso giorno.

Per questo motivo, molte organizzazioni e amministrazioni pubbliche adottano politiche restrittive, disabilitando l'uso delle macro o utilizzando formati che non le supportano.

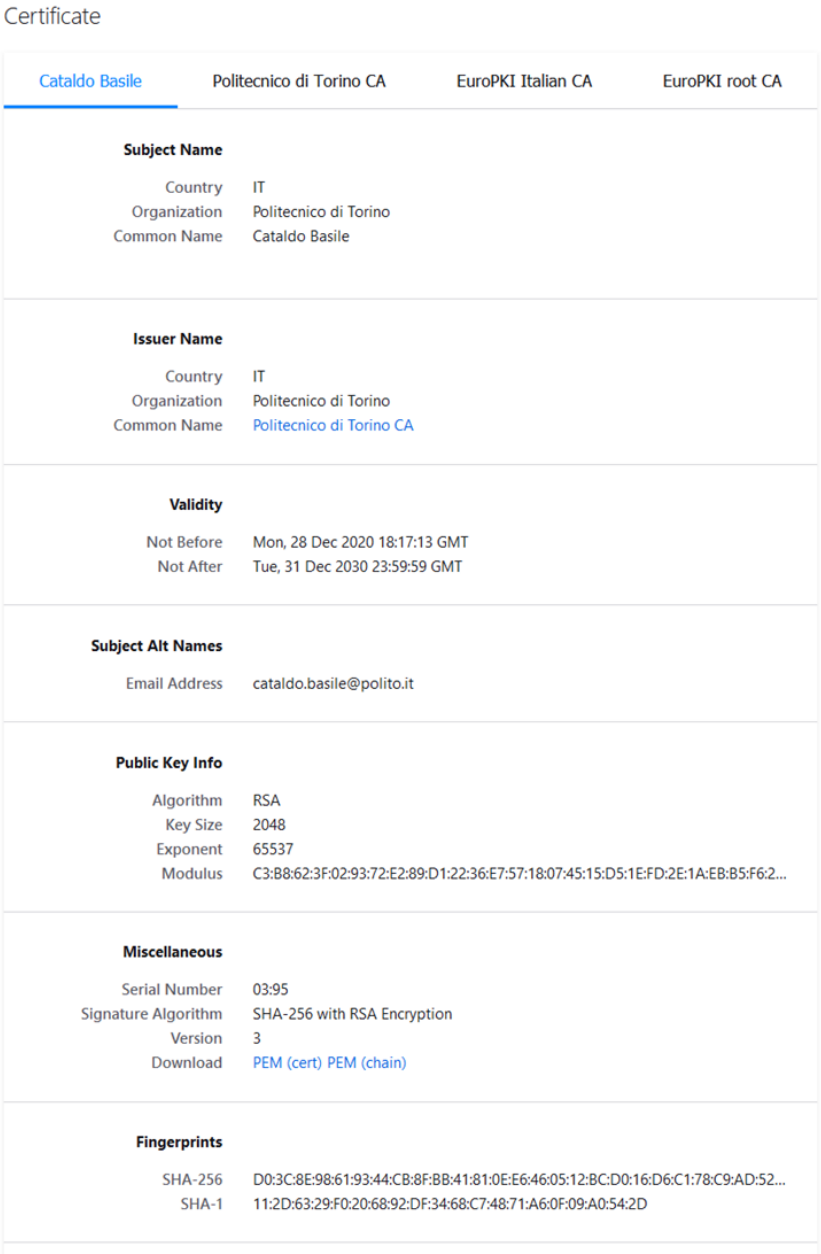


Figura 4.8: Esempio di un certificato X.509.

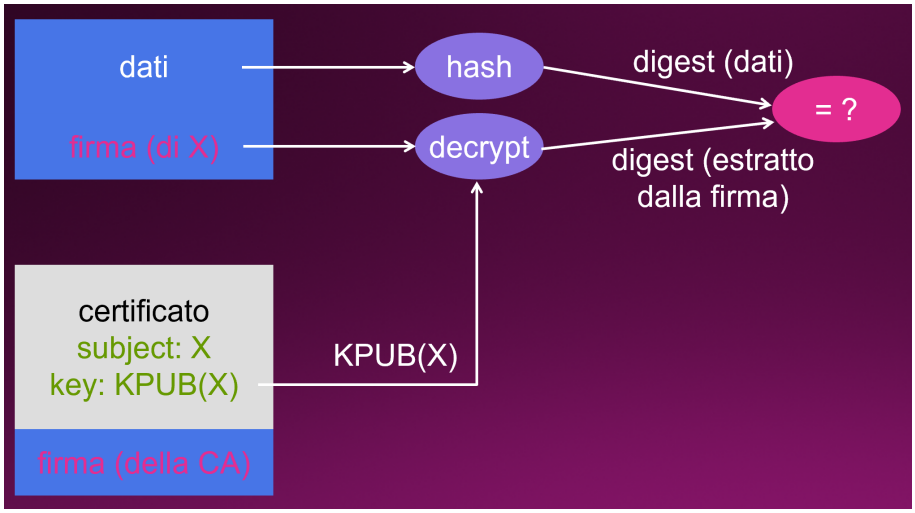


Figura 4.9: Utilizzo di un certificato per l'autenticazione di un messaggio.

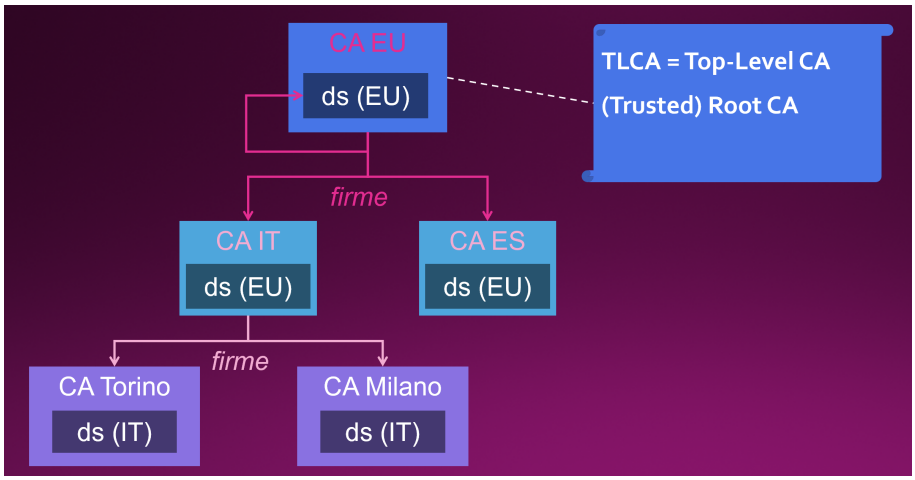


Figura 4.10: Esempio di gerarchia delle Certificate Authority.

Bibliografia

[1] David Lemire, Murugiah Souppaya e Kelley Dempsey. *Automation Support for Security Control Assessments: Volume 1 — Overview*. NIST Interagency or Internal Report (NISTIR) 8011, Volume 1. National Institute of Standards e Technology, apr. 2017. URL: https://nvlpubs.nist.gov/nistpubs/ir/2017/NIST.IR.8011-1.pdf.

Acronimi

AE	Authenticated Encryption	57
AEAD	Authenticated Encryption with Associated Data	57
AES	Advanced Encryption Standard	47
APT	Advanced Persistent Threat	23
ARP	Address Resolution Protocol	33
ARX	Add, Rotate, XOR	47
ASiC	Associated Signature Container	81
ASIC	Application-Specific Integrated Circuit	67, 74
ATM	Asynchronous Transfer Mode	50
BEC	Business Email Compromise	12
BSOD	Blue Screen of Death	40
C.I.A.	Confidentiality, Integrity, Availability	9
CA	Certification Authority	59, 78
CAdES	CMS Advanced Electronic Signatures	81
CBC	Cipher Block Chaining	49
CDB	Computer Data Breach	12
CFB	Cipher FeedBack	50
CMS	Cryptographic Message Syntax	81
CRL	Certificate Revocation List	78
CTR	Counter	50
CVE	Common Vulnerabilities and Exposures	29
CVSS	Common Vulnerability Scoring System	29
CWE	Common Weakness Enumeration	30
DDoS	Distributed Denial of Service	16
DES	Data Encryption Standard	46
DH	Diffie-Hellman	68, 72
DNS	Domain Name System	16
DoS	Denial of Service	16
DSA	Digital Signature Algorithm	62 s.
ECB	Electronic Code Book	48
ECC	Elliptic Curve Cryptography	71 s.
ECDH	Elliptic Curve Diffie-Hellman	72
ECDLP	Elliptic Curve Discrete Logarithm Problem	71 s.
ECDSA	Elliptic Curve Digital Signature Algorithm	73
EDE	Encrypt-Decrypt-Encrypt	47
EPM	Enterprise Patch Management	41
EPSS	Exploit Prediction Scoring System	30
GCM	Galois Counter Mode	57

(Continued)

GDPR	General Data Protection Regulation	11
GPU	Graphics Processing Unit	74 s.
HKDF	HMAC-based Key Derivation Function	75
HMAC	Hash-based Message Authentication Code	56
HSM	Hardware Security Module	71, 80, 82
ICMP	Internet Control Message Protocol	33
IDS	Intrusion Detection System	31
IoT	Internet of Things	11, 23
IPsec	IP security	50
ISCM	Information Security Continuous Monitoring	37
ISP	Internet Service Provider	14
IV	Initialization Vector	48
KD	Keyed Digest	55
KDF	Key Derivation Function	73
KISS	Keep It Simple, Stupid	15
MAC	Message Authentication Code	54, 71
MD	Message Digest	53
MFA	Multi-Factor Authentication	20
MIC	Message Integrity Code	54
MID	Message IDentifier	54 s.
MITM	Man In The Middle	17
NIST	National Institute of Standards and Technology	27
NVT	Network Vulnerability Test	34
OAEP	Optimal Asymmetric Encryption Padding	66
OCSP	Online Certificate Status Protocol	79
OFB	Output FeedBack	50
OOB	Out-Of-Band	59
OWASP	Open Web Application Security Project	37
PAdES	PDF Advanced Electronic Signatures	81
PBKDF2	Password-Based Key Derivation Function 2	74 s.
PGP	Pretty Good Privacy	78
PKCS	Public-Key Cryptography Standards	75, 78, 81
PKI	Public Key Infrastructure	78
PM	Patch Manager	38
PQC	Post-Quantum Cryptography	75
PSE	Personal Security Environment	79
QC	Qualified Certificate	82
QSCD	Qualified Signature Creation Device	82
QTSP	Qualified Trust Service Provider	82
RBAC	Role-Based Access Control	26
RMF	Risk Management Framework	32
RSA	Rivest-Shamir-Adleman	62 s.
SFM	Software Flaw Manager	38
SHA	Secure Hash Algorithm	53
SPKI	Simple Public-Key Infrastructure	78
TCP	Transmission Control Protocol	33
TSA	Time Stamping Authority	81
TSP	Time-Stamp Protocol	81

(Continued)

TST Time-Stamp Token 81
UDP User Datagram Protocol 33
XAdES XML Advanced Electronic Signatures 81
ZDI Zero Day Initiative 31